API Engineering

NHS Pathways Transformation

5 November, 2026

Table of Contents

1 API 111O Concern list 8
1.1 Context 8
1.2 Triage API Concerns 9
2 API 111O queries on the API spec 20
2.1 Actions 20
2.2 Queries 22
3 API Clinical version policy 31
4 API Data Ingestion 32
4.1 API Data Ingestions format 32
4.1.1 Currently used by Triage API 32
4.1.2 AlternateText 32
4.1.3 BodyMap 32
4.1.4 BodyMapOverrides 33
4.1.5 CareAdvice 33
4.1.6 CareAdviceExclude 34
4.1.7 CareAdviceItems 34
4.1.8 CCSConditions 34
4.1.9 CCSConditionSupportingInformation 35
4.1.10 CCSJumps 35
4.1.11 CCSTemplateConditions 36
4.1.12 CCSTemplates 36
4.1.13 Dispomap 37
4.1.14 Dispositionitems 37
4.1.15 Dispositions 38
4.1.16 Dxrank 39
4.1.17 Pathways 39
4.1.18 Skillsets 40
4.1.19 SymptomDiscriminators 40
4.1.20 SymptomGroups 40

4.1.21 Systemflags 41
4.1.22 SystemVariables 41
4.1.23 Triagelogic 41
4.1.24 TxAnswers 42
4.1.25 TxQuestions 43
5 API Development 44
5.1 Development and Release Processes 44
5.1.1 Overview 44
5.1.2 Versioning Strategy 44
5.1.3 Environment Promotion Flow 46
5.1.4 Development Process 47
5.1.5 Release process 54
5.2 JWT and OAuth2.0 with APIM 57
5.2.1 Overview 57
5.2.2 Onboarding a New Client 58
5.2.3 Using our Service 59
5.2.4 Preparing an Environment 61
5.3 Local environment 62
5.3.1 Mac 62
5.3.2 Windows (tbd later) 65
5.4 Triage Engine Architecture 65
5.4.1 Provided ports 65
5.4.2 Required ports 67
5.4.3 Data structures 68
5.5 Working with dev workspaces 75
5.5.1 Log in 75
5.5.2 Get the endpoint URL 76
5.5.3 Example using aws configure sso 76
6 API Documentation 78
6.1 API Certificates 78
6.1.1 Proxygen keys 79
6.1.2 Handling notes 79

6.2 Triage API documentation guidelines 79
6.3 Triage API Specification 81
7 API Environments 82
7.1 Overview 82
7.2 Environments 82
7.2.1 Core Environments 82
7.2.2 Development Environment 84
7.3 Next steps 84
7.4 Provisioning Environments 85
7.4.1 Develop 85
7.4.2 Int 88
7.4.3 Test 88
7.4.4 Ref 88
7.4.5 Production 88
8 API Logging 89
8.1 Events 89
8.1.1 Deployment 89
8.1.2 Triage 92
9 APIM Proxygen Environment setup 95
9.1 APIM Onboarding Questions 96
10 API Pipeline 99
10.1 Commit stage 99
10.2 Acceptance stage 99
10.3 QA stage 99
10.4 Production stage 99
11 API Release 100
12 API Tech stack 101
12.1 Build 101
12.2 Code analysis 102
12.3 Testing 103
12.4 Infrastructure 103
12.5 Application Layer 105

12.6 Frontend 106
12.7 Integration 106
12.8 Support / Observability 106
12.9 Analytics 107
12.10 Managing Dependencies with Dependabot 107
13 API Testing 108
13.1 Unit testing 108
13.1.1 Core 108
13.1.2 Inbound adapters 109
13.1.3 Outbound adapters 109
13.2 Acceptance testing 109
13.2.1 Integration Test Scenarios 110
13.3 Conformance testing 168
13.4 Testing Strategy Gap Analysis 169
13.4.1 1. Purpose 169
13.4.2 2. Evidence Reviewed 169
13.4.3 3. Headline Assessment 170
13.4.4 4. Detailed Analysis by Test Type 170
13.4.5 5. Cross-Cutting Findings 178
13.4.6 6. Action points 179
13.4.7 7. Summary 181
13.5 Performance testing 182
13.5.1 Patient Triage API - Performance Test Results [05/05/2026] 182
13.5.2 Patient Triage API - Performance Test Results [28/10/2026] 199
13.5.3 Patient Triage API - Performance Test Results [30/10/2025 - 28/01/2026] 214
13.5.4 Patient Triage API - Performance Test Results [DD/MM/YYYY] 218
13.6 Private Beta Testing Summary 232
13.6.1 Test Execution Summary (Private Beta) 232
13.6.2 Appendix 233
14 Red Lines Assessment 238
14.1 Scope 238
14.2 SoftEngs Template 238

14.2.1 Using Shared Services 242
14.2.2 Software Development Lifecycle 247
14.2.3 Software Management 250
14.2.4 Team-Specific Red Lines 253
15 Triage Engine API Alpha 254
15.1 Architectural approach to minimise latency in the engine 254
15.1.1 Overview of potential architecture 254
15.1.2 Deconstructing the request 255
15.1.3 Triage engine optimisation 258
15.1.4 Constructing the response 260
15.2 Conceptual Design and Data Requirements 260
15.2.1 In-memory OOP approach 260
15.2.2 Off-the-shelf neo4j approach 267
15.2.3 Lambda with DynamoDB approach 269
15.2.4 Recommendation 274
15.3 Triage API architecture review 274
15.3.1 Status codes key 275
15.3.2 Architecture checklist (discovery phase) 275
15.3.3 Design checklist (alpha phase) 284
15.3.4 Governance 288
15.4 Triage API Alpha goals and success criteria 289
15.4.1 Alpha technical PoC build goals 289
15.4.2 Success criteria 289
15.5 Triage API Run cost estimate - pre-Alpha 290
15.5.1 Architecture key components (to feed into cost calc) 290
15.5.2 Usage assumptions, scenarios 291
15.5.3 Caveats 291
15.5.4 Exclusions 291
15.5.5 Cost table 292
15.6 Future infrastructure and role permissions 293
15.6.1 Overview 293
15.6.2 Principles 294

15.6.3 Architecture / tooling 294
15.6.4 AWS Accounts 294
15.6.5 External environments 295
15.6.6 Role definitions 295
15.6.7 Role Access Matrix by Environment 296
15.6.8 SSO Permission Sets 298
15.7 Failure Scenarios 300
15.7.1 Context 300
15.7.2 Infrastructure 300
15.7.3 Connectivity 300
15.7.4 Security 300
15.7.5 App / logic bug 300
15.7.6 Data – corrupted resource 300
15.7.7 Performance 301
15.7.8 Testing / Deployment failure 301
15.7.9 Integration 301
15.7.10 Process & human failures 301
15.8 Service Map 302
15.9 APIM FMEA 303
15.9.1 Redundancy pattern options 303

1 API 111O Concern list

1.1 Context

This page is to collate responses to the concerns raised by 111O as part of the move to consuming the Patient Triage API. Initially as part of a Private Beta on the emergency prescription pathway, due to launch in Spring 2026 (current target end of April).

Link to word doc that was shared: NHS Triage API Concerns.docx[1]

1. https://nhs.sharepoint.com/:w:/s/msteams_4bd8b1/IQApCj1gzHXkTpsDztcVWoG9AbIvgjJY2YDrQznAWzBHuzQ?e=1ZjcrM

1.2 Triage API Concerns

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Single point of failure between online and telephony	No ability to triage (telephone or online)	Where do users go if both services are unavailable? Do we (HFYS) need a backup should this happen?	Better understanding to how resilience and approach to failure will be handled. A health check endpoint for us to check whether the service is up.	After private beta	Valid concern that needs a position writing down. We are working on establishing a longer term position on BC. Current policy is that we are not removing distribution of CSV files in the short-medium term so telephony providers will still have up to date clinical content to operate their BC tools. The Solo tool will be frozen at decommission time and not disabled. Still probably an action to document or update failure modes (from 111O and PT API perspective). For PT API: • API consumers should be reacting on latency timeouts to trigger failure modes. We should have some recommendations as part of our onboarding. Worth clarifying these as part of 111O EP (when to fail-over). • APIM has a health check endpoint / we can get added to a notify list? - Add an action to do this • Investigate use of Pingdom or similar as an external health check we could react upon (heartbeat check)	API	Kept 111O informed in BC discussions as a key stakeholder.

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
					• Check what other teams do e.g. FTRS / other APIs • People should be running off latency • What to do in partial degradation, how do we notify of issues etc still need to think this through		Pick up other actions as written in comments.
We will be reliant on both AWS and Azure at the same time	Increase risk of an online outage as now dependant on two cloud providers. We've seen service outages across both in the last few months	What is the approach to DR?		After private beta	Valid worry.. Question should be what is the approach to BC 1. Use cached content 2. After PB - look into offline content same as telephony. Could focus on busiest / most acute pathways first. E.g. Dx cut off. Anything after Primary Care dispo uses that fallback. 3. 111O Fallback to telephony should remain. BC content in time may become decoupled from current releases and update on a lower frequency. Authoring tool may enable a separate BC API?	API	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Versioning	Wrong clinical content is shown to the patient, Issue around stem removal could become more of a problem if the data is affected instantly.	What if we are pointed at the wrong version? Depending where the API gets it's data, could the stem issue happen instantly? How is the API working in the background? What does a 'go-live' entail? PAT does not version Digital content currently.	A better understanding of versioning. Understanding the “Go Live/ release process” Learn from EP Implementation Can we get testers access to PAT?	After private beta	Versions will be static. Clients will get the version they request. Shouldn’t be any race conditions from changing versions mid journey. Should probably draw out a version update and content release process. (PT API mostly there for this now?) 111O Should have a proper test and rfc process in place before version change in prod Careful to check about refreshing any cache as part of version changes.	API	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Speed of updates to questions/ routing etc	Lack of ability to make changes quickly, dependent on a pathways release (3-6 months) Making all changes in PAT may causes issues if we have multiple changes with customisations at the same time		Test this within EP Understand how interims will work and get fed into future main versions	After private beta	This is an authoring tool requirement? • Automated testing • Clinical sign off • Quick release into prod Technical ingestion process shouldn't take long. Clinical process and safety checks will need to be followed and would be the complexity	Authoring	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Existing Customisations	Will these be supported and what work arounds/ changes might be needed.	Current customisations include What if we want to include NHS.UK[2] care advice again? Disposition overrides Dx38 Care Advice Additional / split questions Skipped questions AreYouSure is a separate type to us New ones: Interrupt page mid Pathway +Others	Make a full list of customisations and assess to see how/ where they would be mitigated.	After private beta	Need to review case by case.	API	

2. http://nhs.uk/

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
New customisations (any new ideas we want to try out or use in the future)	Slow down implementing new ideas.	· Currently we might want to implement a new interrupt mid-pathway, will this be easy? · In the past, we have implemented new types (E.g. AreYouSure), how can we do this and maintain speed going forward?		After private beta	Should be part of authoring tool rather than logic built directly into 111O ? More customisations means they are becoming decoupled from the source content. Routes back to the authoring process.	Authoring API	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
System Variables/set variables	If system variables are missing/ incorrect it will impact on the assessment	Do we still have control over our switches? We don't do cut-off at the moment... will we be able to be independent of it? We need to be able to insert set nodes Will any custom/ added set variable be persisted in the responses?	Can we create a test pathway that covers all our worry points. Useful for automated tests. This would support the previous concern	After private beta	Not sure, need more info / work it through. Set your own system variables as part of the triage context? Need to check we could support this. Needs a separate chat including Andrew etc to find out more	API	
Loss of access to files	Currently very useful during testing with the current state of release notes	Quality of release notes needs to be improved	Ways to better identify what has changed between releases and impacts on 111 online	After private beta	Authoring process? We will have all versions available on the api (at least in the test env) Interesting to know how they currently manage versions in test. Technically, the files wills till be available for now..	Authoring	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Loss of Neo4j	Currently if a someone does not have access to vPAT then they have the option of using our Neo4J visual app to work out routes to specific outcomes. We will no longer have this to use and it may become very difficult to find routes to specific places (questions/ outcomes/ care advice etc)	· Will it be possible to give more 111 online colleague access to PAT/ vPAT? · Or a read-only version? Or a whole export of PDFs etc?	· Explore how we can allow more access to this information.	After private beta	Just give access to PAT/ vPAT? Read only role? Maybe an alternate view for authoring. Authoring req? Pathway viewer mode?	Authoring	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Test Environment		Will we have a stable environment to test on? Will we be able to test against two different versions at the same time (including if they manage to allow us to make changes to old versions)? Who can import into the API? How do we manage merging multiple releases/ In to live		PB for testing. After PB for ingestion and access to multiple versions	For Private Beta, test environment will be made available with a single version. Target state is test and integration environments available with multiple versions to select from within each environment. Import into API will need to be via authoring process / tooling. - will follow ingestion process Merging releases etc all part of authoring process. Feels like an authoring requirement, Git Branching problem to solve.	API Authoring	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Documentation	If the team get even less documentation then they will be less aware of impactful pathways changes that could lead to poor user journeys		Can a way to automatically document changes be made available?	After private beta	Authoring process, requirement? Authors need to work together as a team to be aware of what is happening rather than siloed. Ideally author once for multiple channels or author for user facing first.	Authoring	
Pathway suitability *There is a concern we could be returned a Pathway that is not suitable for online. Currently we only import a subset of Pathways and if Pathways attempts to jump us to one we haven't imported we end the triage.*	Show users unsuitable Pathways. Show users Pathways that haven't been given online content.	· How will we prevent unsuitable Pathways/ questions from being returned to us?		After private beta	During early PB there will only be a single suitable pathway. Authoring / versioning discussion. Testing process, automated or otherwise should be in place. Will need to dig more into the detail of this.	Authoring	

Concern	Potential Impacts	Questions	What could help reduce the concern?	Phase (For PB or later)	Comment(s)	Workstream	Status
Potential for creating sex-less triage The API demo today 10/12/25 shows they are basing build on PWs bodymap, which is using patient (gender aka, sex) and building the same as they have before. How could be potentially change this to create sex-less PWs	Create PWs which do not need to be split into m/f unless it's clinically needed to be known	Can we over-ride the build so we don't need to know about a persons sex/gen if it's not needed for a triage		After private beta	Needs more discussion and develop some options. Could be handled by setting flags/variables? mid-point jumps into a pathway?	API Authoring	
Surveys = we get loads of rich data from the surveys, and they are built to pick up parts of the triage. Will this still work with the API?	Check that surveys will still pick up the elements of the journey that are helpful		Explore whether the surveys are affected if questions are coming from an API Can We use EP work to assess impact on Surveys?	Private beta for testing	Hopefully won't be affected, 111O manages the journey and context? Assess via EP work.	111O	

2 API 111O queries on the API spec

2.1 Actions

	Action	Owner	Status	Comments
1	Confirm that care advice is not required for Emergency Prescriptions	Roy M	DONE	We do not display care advice or worsening information for EP - RM
2	Confirm if it is possible to send a hard-coded symptom group in the body map request. Alternative is to change the API not to require it	Roy M	DONE	It would be possible for us to add a short term fix for EP for the SG group to be added to EP related requests. - RM **Note It is no longer necessary to send the symptom group details, as this was removed in** [NPT-807] API - Remove requirement for clients to provide SG etc with bodymap selection - CDT JIRA[3]
3	Understand what 'answer' fields are being used by 111O at present, and under what circumstances they are populated. This will help determine what fields need to be returned in the API.	Roy M	DONE	We use the AnswerText field (which maps to our Title in the Answer model) to display on screen and we use the AnswerReportText field (which gets mapped to our ReportText field in our Answer model) in our ITK messaging details and also in case consultation summary for breast clinics so this is required. We do also use the DispoDisplayText field but usually if this isn't populated we default to the ReportText field and so if only one was returned ReportText would be preferable. - RM Additional - see comment below in number 7. - RM

3. https://nhsd-jira.digital.nhs.uk/browse/NPT-807

	Action	Owner	Status	Comments
4	Relating to the question above - understand what answer fields will be populated in PAT for EP, as this will also determine what needs to be returned by the API	Gareth D	IN PROGRESS	• Confirmed on the call that the digital fields will be populated in PAT, so both telephony and digital fields should be returned through the API • Will need to confirm following the population of the EP Pathway in PAT exactly what fields are being populated
5	Confirm what FHIR package is required to de-serialise the responses from the API	Gareth N	DONE	
6	Provide more examples of responses. Gareth N to contact Roy to get a feel for what is required	Gareth N	DONE	

	Action	Owner	Status	Comments
7	Confirm if the variables and e.g. node values are sent in the ITK message, or if this is just question text and answer text	Roy Marler	DONE	Upon investigation this is a little more complicated than first thought. We include a Consultation Summary and a Pathways Assessment in our ITK messages. The Pathways Assessment is a list of answers given - the ReportText field of the answer. It does not include any question numbers or Ids or set/read nodes etc. The Consultation Summary filters that down and will only include the ReportText of the answer item if the DispoDisplayText is populated. This is because ReportText is almost always populated but DispoDisplayText is only populated where relevant. So, if a user answers "not sure" to questions then DispoDisplayText is blank so we don't include not sures in the summary. Much like when asking if something is present like vertigo, DispoDisplayText is only populated when it is present but blank when it isn't. This means the Summary is kept to only those items that are relevant. The Pathways Assessment will be a complete list. This means we would need both fields returned if possible. If we did only have ReportText it would mean it would never be included in the Consultation Summary however, the Pathways Assessment would be the full list.

2.2 Queries

(EP = Emergency Prescriptions)

	Query	Raised By	Required for EP?	Response
1	What is the intent of both a request / response, only "plan" looks to be allowed?	RM	**N/A**	**Yes, only "plan" is allowed.** For the intent of the Task (in both the request and response), we interpret it as the triage service aiming to return what the caller needs—either care advice or requested services (such as an ambulance). For that reason, we believe **"plan"** is the most appropriate intent to reflect the triage service interaction. In the FHIR R4 specification, "plan" is defined as: *"The request represents an intention to ensure something occurs without providing an authorization for others to act."* This definition makes "plan" broad enough to cover both self-care advice and service requests, such as an ambulance. The **intent** field is required by the FHIR specification, and for the alpha build we validate all requests to ensure correct FHIR formatting (including all required fields).
2	On the response there is a status of in-progress or completed, is that as in an asynchronous status or something else? I.e. if the response is in-progress the results are incomplete and therefore we would have to wait to potentially get completed as it was still running?	RM	**N/A**	**No, it's not asynchronous.** The status represents the triage interaction state. All requests are considered **"in-progress"**, and all responses will remain **"in-progress"** until a final disposition or the end of the pathway is reached and there are no more questions to return. At that point, the status will be set to **"completed"** as the interaction between the user and the Patient Triage API is is complete. Requests to the API are **stateless**—you provide all triage information so far (patient details and previously answered questions), and we return a response with the next question and/or care advice and/or disposition. Once you receive the response, you have everything the API can provide based on the information sent; there is no waiting for additional results or background processing. Similar to the intent field, the status field is required by the FHIR specification, and we validate all requests to ensure correct FHIR formatting (including all required fields). However, this field is not read or used by the triage API; it simply provides metadata about the request/response interaction.

	Query	Raised By	Required for EP?	Response
3	Care advice looks to consist of just a text item. Are more fields to be returned if populated? For example, we use keyword and/or alternate text to separate out care advice so all the Medication care advice is together under the heading Medication. Also, we use the CxId to identify CX1910 which is specifically worsening care advice.	RM	**No** Care advice is not required for EP	**Currently, only text is returned.** This was intentional during the alpha phase to simplify the design and focus on returning key information. Additional fields for care advice can be incorporated into the design, and we anticipate further changes as more data needs to be returned. That's very helpful information regarding keywords (which we initially thought would only be needed for logic, not for returning), alternate text, and the CX1910 code. Thank you for sharing that insight. **Next steps** • RM to check if care advice is required for EP • CX1910 not required for EP
4	It may just not be in the example but I assume WATCH FOR: care advices will also be returned?	RM	**N/A**	**Yes** If the algorithm says they should be included, we'll include them in the response.
5	Is the care advice type returned? We sometimes do different things depending upon whether it is interim or in line etc.	RM	**No** Care advice is not required for EP	**No, the type is not currently returned.** This is something we can design and add to the care advice in future builds—thank you for highlighting its use as a field by 111O. As mentioned above, the objective of the alpha build was to prove the concept of returning care advice as a communication for key fields.
6	Is ordering of answer options done via the valueCoding > code value, is that the equivalent of AnswerNo in the csv files?	RM	**N/A**	**Yes, that's correct.** For each answer (within the **item** list for the question **group** in the Questionnaire resource), the `answerOption.valueCoding` is set to the answer number provided for each option in Pathways.

	Query	Raised By	Required for EP?	Response

7	This might be my ignorance of Pathways Data but in the example of a request for a continuing triage what would this bit be in the previous answers:
	{
	"type": {
	"text": "ClinicalArea",
	"valueString": 8
	}
	},
	That probably won't display well, but it doesn't match the other format of TX code and answer code number schema.
		RM	**N/A**	**That format is outdated—here's the new approach.**
				This was an initial concept for how we would format the selection of an answer to a bodymap question if it was returned. Since starting the alpha build and gaining a better understanding of requirements, this approach has changed.
				The JSON below shows an example of how the answer for a bodymap question will look going forward. We will update this in our technical specification. The bodymap is treated as a unique case compared to other questions in Pathways due to the nature of what is being answered and how it is captured.
				{
				"type": {
				"coding": [
				{
				"code": "bodymap"
				}
				]
				},
				"valueCodeableConcept": {
				"coding": [
				{
				"code": "pw_id",
				"display": "PW516"
				},
				{
				"code": "pw_title",
				"display": "Abdominal Pain"
				},
				{
				"code": "sg_code",
				"display": "1004"
				},
				{
				"code": "sg_desc",
				"display": "Abdominal Pain"
				}
				]
				}
				}

	Query	Raised By	Required for EP?	Response

8	Following on from the answer to number 7 above; currently we replace the bodymap with our search/categories pages and this allows a user to basically select the pathway that they are going to use. This means that we would effectively be always continuing a triage instead of starting one because we would have age/ gender/party and we would supply the pathway by adding something like the following to the answers - is that correct?
	{
	"type": {
	"coding": [
	{
	"code": "bodymap"
	}
	]
	},
	"valueCodeableConcept": {
	"coding": [
	{
	"code": "pw_id",
	"display": "PW516"
	},
	{
	"code": "pw_title",
	"display": "Abdominal Pain"
	},
	}
	}
		RM	**Yes**	**Yes, that's correct.** The only two points to note with this approach are:
				1. **Pathway validation:** The pathway selected from the search/categories pages must also exist in the list of pathways returned by the API. If your search/categories pages are filtered by release and match the release selected for the Patient Triage API, this should work fine—but it's worth calling out.
				2. ~~**Symptom group requirement:** You will need to provide the symptom group when answering the bodymap in the request. This is required for returning dispositions later in the journey. The symptom group is provided for each pathway in the list of pathways included in the bodymap question response from the API.~~ ~~**We are currently looking are ways to update the API to only require the pathway ID so this likely won't be in issue later in beta but is currently needed for alpha**~~
				a. **Note this was implemented in** [NPT-807] API - Remove requirement for clients to provide SG etc with bodymap selection - CDT JIRA[4]
				Next steps
				• Roy to check if SG could be hard-coded for EP in lieu of the change being made for SG not to be required in the request

4. https://nhsd-jira.digital.nhs.uk/browse/NPT-807

	Query	Raised By	Required for EP?	Response
9	I had a question on the responses around complete journeys and what that may mean for users. For example would it mean that when a journey is marked as complete, what happens when a user changes their answers that may lead to a different outcome.	SB	**N/A**	**That's correct, changing answers can lead to different outcomes.** Given the stateless design of the API, we do not store or track any particular triage journey. If a user goes back and changes answers, it could lead to a different outcome. The API will treat each request as a standalone task and return the next or final response based on the previous questions and triage information provided in that request. However, if a user goes very far back in the triage journey and changes one answer, then sends the request, there could be issues because the series of previous answers might not logically link together. For recent questions—such as updating the last answer—you could potentially receive a new outcome.
10	ControlType (TxQuestions.csv) we map 1 > questionType 11, 3 > set node, 4 > Read node - not sure if we need this if it is handled by the API but how would Set/Read node info be passed to us and/or passed back to the API in the next request?	RM	**N/A**	**This will be handled by the API.** The Patient Triage API will only return questions (**ControlType 1**) and will internally handle variables (**ControlType 3**) and branches (**ControlType 4**). This approach minimizes clinical safety responsibilities on the client side. Therefore, information for Set/Read nodes will not be passed back. On a related note, the API currently only supports **ControlType 1 (Questions)**. When we start supporting **ControlType 2**, we can add an identifier to clarify which type it is. **Notes** • Assuming that the Pathway is authored in PAT this is OK • Note that control type 2 is not used in 111O at all

	Query	Raised By	Required for EP?	Response
11	Our Question Id example "PW915.0" built using Pathway Id and OrderNo from TriageLogic file. Used for linking together in the site. Not sure at this point if we could work with an alternative.	RM	**Yes**	**We can return both the question ID and the pathwayId-orderNo combination if required.** Changing the API to use `pathwayId.orderNumber` (which we refer to as a *coordinate* in our triage engine logic) would require significant changes to our release process and triage logic. However, we could update the logic to return the coordinate **alongside** the question ID if that's what's important. Switching the Patient Triage API to use coordinates **instead of** question IDs would be a more fundamental change. **Next steps** • Triage team to have a look to see if the IDs can be returned separately
12	Keywords and Excluded Keywords: these are in the pathways data in the care advice and care advice exclude files however the API looks like it only returns the care advice items and these do not contain keywords information.	RM	**No**	**Keywords logic is not implemented in Alpha but could be included.** Currently, the API only returns care advice items without keyword information. We plan to incorporate keyword and excluded keyword logic in the Beta phase as part of the design enhancements. This is something we identified during Alpha and realised would need to be implemented in Beta. **Notes** • No keywords or excluded keywords in EP
13	AlternateText.csv contained other text with displaytype ONLINE - would this be done automatically or visible elsewhere?	RM	**Possibly?** Need to confirm what text fields are being used by 111O	**Alternate text is not included in Alpha but but could be included.** We can configure the API in the Beta phase to return alternate text where the `displayType` is set to "**ONLINE**", based on the skillset used. **Next steps** • Roy to understand exactly what fields are being used in 111O and when • Need also to understand what fields will be populated by the authors in PAT

	Query	Raised By	Required for EP?	Response
14	Answers ReportText <- AnswerReportText in TxAnswers file we get AnswerText but not AnswerReportText (might be able to do without it but would need to check in more detail) Answers DispoDisplayText in TxAnswers	RM	**Yes**	**Yes, we can return** `AnswerReportText` **and** `DispoDisplayText` **for each answer if required** These fields already exist in our logic model for answers, as we use them to build the Pathways assessment and the consultant summary. If needed, we can update the API in the Beta phase to return `AnswerReportText` and `DispoDisplayText` for each answer. **Next steps** • ~~Roy to understand exactly what fields are being used in 111O and when - DONE~~ • ~~Need also to understand what fields will be populated by the authors in PAT - DONE~~
15	The field Positive from TxAnswers - only used in a handful of places. Mostly diagnostic clinics so probably wouldn't be an issue for EP and the alpha	RM	**No**	**Next steps** • Triage team to work out how this could be returned if required
16	Specify in the TxAnswers file however, I don't think we use it any more but needs checking (we call it SpecifyText).	RM	**No**	**Next steps** • Roy to double check whether this is needed long term
17	CareAdvice Type we pick out interim separately etc. Again it looks like only the CareAdviceItems are included?	RM	**No** Care advice is not required for EP	**This is something we can look to implement in the Beta phase.** At present, we collect all care advice items together and return them in a single **Communication** FHIR resource. In future, we can enhance this to return a separate Communication resource for each care advice type (e.g., *interim*, *in-line*, *disposition*, *home care*). These questions have been good insight for requirements, thank you for highlighting them.
18	What version of FHIR is used to deserialize responses, (should be R4)	RM	**N/A**	**Next steps** • Gareth N to verify what package should be used

	Query	Raised By	Required for EP?	Response
19	We display a list of pathways and [Roy to update] . Will the API be able to provide these e.g. through another endpoint	RM	**No**	**Next steps** • Roy to provide some more info on what could be required in the future.
20	Could we have more examples of responses e.g. continuing a triage, with care advice etc. We can then try the deserialisation at our end.	RM	**N/A**	**Next steps** • Gareth N to send Roy more responses
21	Need to understand what data is required for the ITK messages that is not being returned through the API	GD	**TBC**	**Next steps** • Roy to look at this

3 API Clinical version policy

- Target state process will be hands off to ensure clinical safety. Managed by scripts, not manual intervention. Key imperative is to not allow unsafe content into production.
- Process of clinical version export must be triggered by an authorised user - MFA, RBAC, Audit in place
- Clinical Versions will be exported to lower environments first. MUST have clinical check with auditable sign-off before a clinical-version can be promoted to production
- Backups of clinical versions must be in place before being made available to consume.
- Clients must specify the clinical version of content
- If a bug is found in a live clinical version, adopters should roll back to last known good release while a new version is worked on. This will also be the process for early adopters.
    - Q: Do we enforce this? (needs business input)
        i. No - client gets what they request
        ii. No - client gets what they request, including a warning
        iii. Yes - Halt with error (operation outcome)
        iv. Yes - Warning and give content to last known good clinical version - goes against *"you get what you request"*. Could introduce clinical risk?
        v. Yes - There is a flag to set, which enables choice - clinical content downgrade to last known good, uplift to fixed version, vs halt with error
- A new Clinical version fixing a broken clinical version gets a new clinical version number. It does NOT replace or overwrite an existing clinical version.
- We deprecate clinical versions deemed 'unsafe' (auditable process via authoring tool). There is a process and notification for deprecation, not just turned off.
- Authoring tool is the UI for managing clinical versions

4 API Data Ingestion

4.1 API Data Ingestions format

4.1.1 Currently used by Triage API

- BodyMap
- CareAdviceItems
- Dispositions
- Pathways
- Skillsets
- SymptomDiscriminators
- SymptomGroups
- Triagelogic
- TxAnswers
- TxQuestions

4.1.2 AlternateText

Name	Type	Required
DisplayType	String	
ItemType		
ItemId		
ItemNumber		
ItemText	String	

4.1.3 BodyMap

Name	Type	Required
BMGroup		
BMItem		

Name	Type	Required
Ordering		

4.1.4 BodyMapOverrides

Name	Type	Required
SkillsetID		
BodyMapPathway		
BodyMapPathwayJumpsTo		

4.1.5 CareAdvice

Name	Type	Required
CxID		
CxAge	int	
CxGender		
CxKeyword	string	
KWSpecific		
KWGroup		
Type		
Cx_Dx		
AlternativeTitleDigital		

4.1.6 CareAdviceExclude

Name	Type	Required
CxKeyword		
ExcludeKw		

4.1.7 CareAdviceItems

Name	Type	Required
CxID		
CxNo		
CxText		
include		
ActiveOOH		
ActiveAmbulance		
ActiveSPA		
ActiveFOHINT		
CxTextDigital		

4.1.8 CCSConditions

Name	Type	Required
ConditionId		
ConditionName		
SymptomGroup		

Name	Type	Required
Importance		
AmbJump		
HomeMgmtJump		
ServicesJump		
ETCJump		
Gender		
Age		

4.1.9 CCSConditionSupportingInformation

Name	Type	Required
ConditionId		
CategoryTitle	string	
CategoryId	int	
OrderNo	int	
Text	string	

4.1.10 CCSJumps

Name	Type	Required
Label	string	
OrderNo	int	

Name	Type	Required
PathwayJump		
ButtonType		

4.1.11 CCSTemplateConditions

Name	Type	Required
TemplateID		
ConditionID		
OrderNo	int	

4.1.12 CCSTemplates

Name	Type	Required
TemplateId		
Version		
TemplateName	string	
Synonyms		
SymptomGroup		
ServicesJump		
ETCJump		

4.1.13 Dispomap

Name	Type	Required
DispoCode		
PDTDisposition	string	
LocalDisposition		
DxRank		
IncludeOOH		
IncludeAmbulance		
IncludeSPA		
ActiveOOH		
ActiveAmbulance		
ActiveSPA		
CMSGroup		
AmbPriorityType		
SubstituteCode		
ActiveFOHINT		
SubstituteGroup		

4.1.14 Dispositionitems

Name	Type	Required
Dx_ID		

Name	Type	Required
DxItemNo		
DxItemText	string	
DxReportText		
DxItemRationale		
DxCode		
DispoAdviceCode		
TransferCode		
Active		
ActiveOOH		
ActiveAmbulance		
ActiveSPA		
ActiveFOHINT		

4.1.15 Dispositions

Name	Type	Required
DispoCode		
DxID		
DispoText	string	

4.1.16 Dxrank

Name	Type	Required
DxRank		
DxText	string	

4.1.17 Pathways

Name	Type	Required
PW_ID		
Version		
PathwayTitle	string	
Module		
Party		
PW_Age		
PW_Gender		
PW_ClinicalArea		
Pw_Trauma		
KP_Use		
KP_NotUse		
CxKW		
PathwayTitleHtm		
Symptom_Group		

4.1.18 Skillsets

Name	Type	Required
SkillsetID		
SkillsetDescription	string	
SkillsetUserType		
Service		
EntryPathwayId		
EntryOrderNumber		
Tbranch		
EarlyExitPathwayId		
EarlyExitOrderNumber		
Enabled		
AlternateDisplayText	string	

4.1.19 SymptomDiscriminators

Name	Type	Required
SdId		
SdDesc	string	

4.1.20 SymptomGroups

Name	Type	Required
SgId		

Name	Type	Required
SgDesc	string	

4.1.21 Systemflags

Name	Type	Required
SystemFlagCode		
SystemFlagDescription	string	

4.1.22 SystemVariables

Name	Type	Required
VariableName	string	
VarType		
Explanation		
Values		

4.1.23 Triagelogic

Name	Type	Required
PW_ID		
OrderNo	int	
Qu_ID		
AnswerNo		
JumpID		

Name	Type	Required
T1		
T2		
T3		
T4		
Rank		

4.1.24 TxAnswers

Name	Type	Required
QuID		
AnswerNo		
AnswerText	string	
AnswerReportText	string	
SystemFlag		
DispoDisplayText	string	
SnMed_description	string	
SnoMed_Procedure		
Dispo_keyDescription	string	
Dispo_key_Procedure		
AnswerRationale		
CxSpecific		

Name	Type	Required
Specify		
Positive		
SymptomDiscriminator		
AnswerTextDigital		
AnswerRationaleDigital		

4.1.25 TxQuestions

Name	Type	Required
QuID		
QuestionText	string	
Qu_Rationale		
ControlType		
QuestionTextDigital		
Qu_RationaleDigital		

5 API Development

- Development and Release Processes (see page 44)
- JWT and OAuth2.0 with APIM (see page 57)
- Local environment (see page 62)
- Triage Engine Architecture (see page 65)
- Working with dev workspaces (see page 75)

5.1 Development and Release Processes

5.1.1 Overview

This document defines the standardized process for releasing the Patient Triage API changes across **Dev**, **Test**, and **Production** environments. It ensures consistent deployments, traceability, quality assurance, and communication among engineering, QA, operations, and product stakeholders.

5.1.2 Versioning Strategy

Projects within Safe and Effective Triage will follow **semantic versioning**. In summary (from https://semver.org/):

Given a version number **MAJOR.MINOR.PATCH,** increment the:

1. **MAJOR** version when you make incompatible API changes
2. **MINOR** version when you add functionality in a backward compatible manner
3. **PATCH** version when you make backward compatible bug fixes

Additional labels are available as extensions to the MAJOR.MINOR.PATCH format:

- **release candidates:** e.g. `1.0.0-rc`
- **build metadata:** e.g. `1.0.0+sha-ab12cd3`
  - Added after `+`.
  - Purely informational (e.g. commit SHA, build number).
  - Does **not** affect version precedence.

The requirements from https://semver.org/#semantic-versioning-specification-semver are inherited for this project.

Initial Version

- Development will start at `0.1.0` and incremented following the rules above.
- First formal release intended for production: `1.0.0`.

5.1.2.1 Automation in GitHub Actions

We use **GitHub Actions** to automate versioning:

- **Pre-releases** → auto-generated on `main` merges after build and fast tests (`-pre`).
- **Release Candidates** → auto-generated on `main` merges after acceptance tests (-rc).
- **Quality Assurance** (fail/pass) → triggered manually by QA engineer after their work (-qa-fail, -qa-pass).
- **Formal Releases** → triggered manually (`vMAJOR.MINOR.PATCH`).

5.1.2.1.1 Version Calculation

Use commit messages (`feature:` , `fix:` , `BREAKING CHANGE`) to decide bumps.

The **commit** contains the following structural elements, to communicate intent to the consumers of your library:

1. **fix:** a commit of the *type* `fix` patches a bug in your codebase (this correlates with `PATCH` [5] in Semantic Versioning).
2. **feat:** a commit of the *type* `feat` introduces a new feature to the codebase (this correlates with `MINOR` [6] in Semantic Versioning).
3. **BREAKING CHANGE:** a commit that has a footer `BREAKING CHANGE:` , or appends a `!` after the type/scope, introduces a breaking API change (correlating with `MAJOR` [7] in Semantic Versioning). A BREAKING CHANGE can be part of commits of any *type*.
4. *types* other than `fix:` and `feat:` are allowed, for example @commitlint/config-conventional[8] (based on the Angular convention[9]) recommends `build:` , `chore:` , `ci:` , `docs:` , `style:` , `refactor:` , `perf:` , `test:` , and others.

https://www.conventionalcommits.org/en/v1.0.0/

5.1.2.2 Implementation

5.1.2.2.1 Formal Releases

- The "**Release Build Process**" should be followed for the creation of a formal release (i.e. not a tagged release or prerelease build).
- The trigger for physically creating a formal release should be **manual.**

---

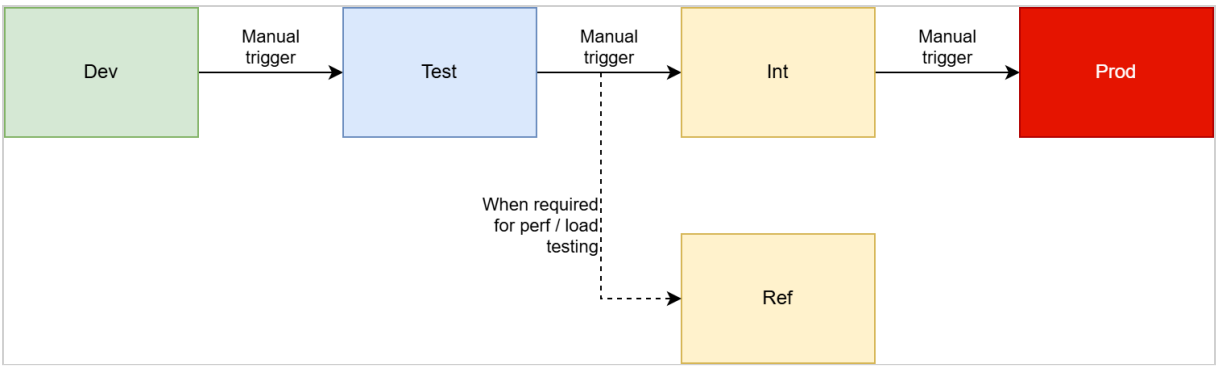
5. http://semver.org/#summary
6. http://semver.org/#summary
7. http://semver.org/#summary
8. https://github.com/conventional-changelog/commitlint/tree/master/%40commitlint/config-conventional
9. https://github.com/angular/angular/blob/22b96b9/CONTRIBUTING.md#-commit-message-guidelines

- The triggering process will occur in GitHub.

5.1.2.2.2 Pre-releases

- Pre-releases should be published as part of the **Post-merge Deployment & Pre-release** stage on **pushes** to `develop`.
- Pre-releases use the `pre` tag and build versions should be incremented for each new version published. This results in the format `v<major>.<minor>.<patch>-pre.<build-version>`.
- There should be a process that can be run to clean up old pre releases and related artefacts, which can be run manually or on a schedule.

5.1.3 Environment Promotion Flow



5.1.3.1 Environment-Based Deployment Matrix

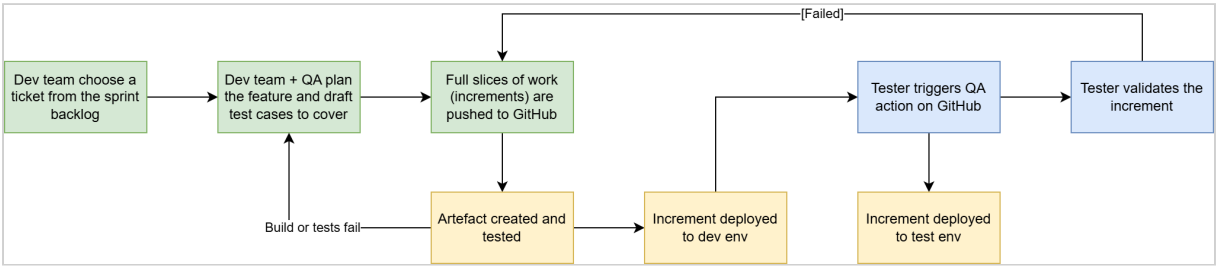
Environment	APIM Environment	AWS Environment	Source of Truth	Trigger	Deployment Frequency	Purpose
Workspace	*N/A*	*N/A*	`feature/*` or `main` branches	Automatic on push	Per push	Isolated testing of in-progress work
Dev	dev	Development (998245890657)	`main` branch	Automatic on push	After each merge to `main`	Early integration testing, regression testing, shared feedback

Environment	APIM Environment	AWS Environment	Source of Truth	Trigger	Deployment Frequency	Purpose
Test	internal-qa	Test (938674806470)	Tagged release	Manual by QA	Per feature or release	Functional QA and validation by testers, non-functional testing Regression End-to-end automated tests
Integration	int	Test (938674806470)	Tagged release	Manual	Pre-release cycle / stable	Integration testing with internal/external partners (e.g., Safe and Effective Triage) External onboarding testing
Reference	ref	Test (938674806470)	Tagged release	Manual with approval gates	Pre-release cycle	Pre-live reference, regression testing and validation, non-functional testing (baseline) Performance
Production	prod	Prod (744389574564)	Tagged release	Manual with approval gates	On release	Live usage in clinical systems

5.1.4 Development Process

The development process covers the usual **day-to-day activities of developers and testers** within teams.

5.1.4.1 Overview

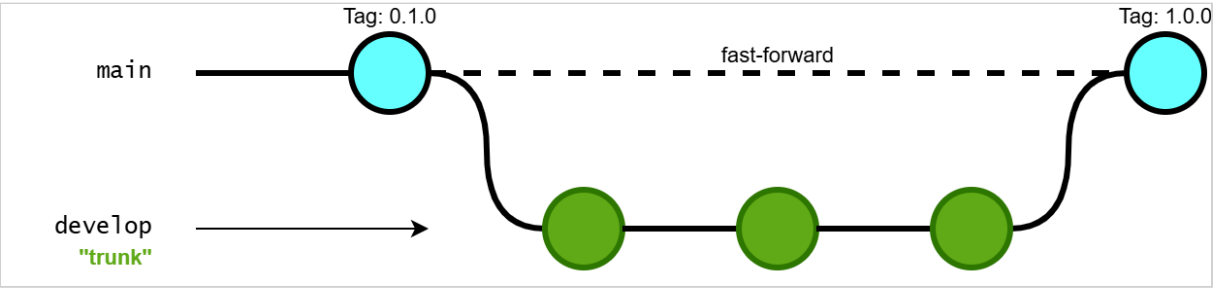


1. A ticket is included in the sprint backlog and picked up by the development team.
    a. A developer moves the ticket to the 'In Progress' status.
2. The development team (including testers) kick off the work.

   a. Requirements are discussed and confirmed. Anything unclear confirmed with PO and/or SME.
   b. Test cases are listed at both acceptance and unit level.
3. A developer pushes commits to GitHub.
   a. Commits must be signed in order to be merged back into the release branch.
   b. Each commit pushed to GitHub triggers a run of an automated pipeline, which must pass in order to publish an artefact.
   c. When an artefact is published it is deployed automatically to the dev environment.
4. Changes are further tested by a tester.
   a. Someone moves the ticket to the 'In Test' status.
   b. The tester triggers a pipeline to deploy the artefact to the QA environment.
   c. If defects are identified that require changes, the development team push subsequent changes before the ticket progresses further.
5. Test evidence should be documented and reviewed to ensure that testing has been comprehensive and all acceptance criteria are covered.
   a. If the review highlights additional testing is required, the testing is performed and documented.

5.1.4.2 Branching Process - Trunk-Based Development

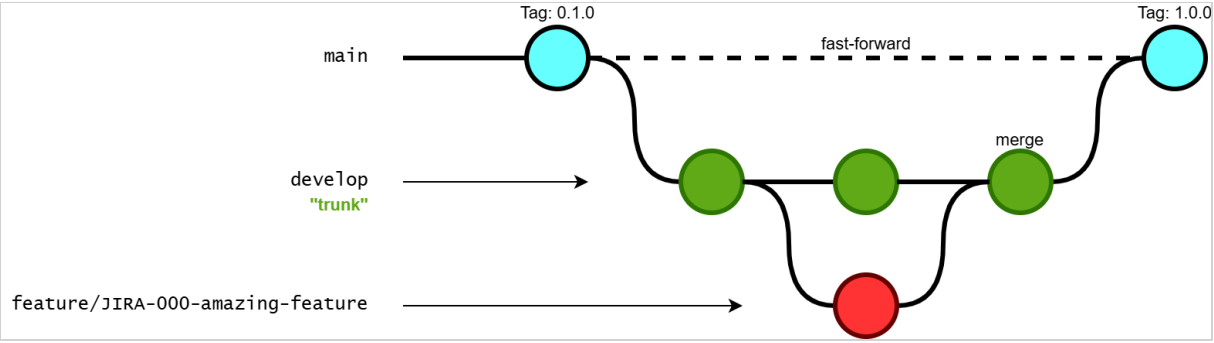
The team uses Trunk-Based Development (TBD). The trunk branch is called `develop`, and releases are referenced in `main`. In most cases, no other branches will be needed.



In the case where the team decides to do some work in a branch, the process should be something like:

- The developer creates a new task branch based from the trunk branch.
  - The developer updates their local copy of the trunk branch.
  - The new branch format should be `feature/NPT-000-example-name`
- Once the development of the feature is completed the developer creates a pull request that is peer reviewed before merging.
- If the peer review is successful the feature is merged back into the release branch
- The release branch is then deployed to the test environment where it is tested.
- If testing is successful then the release branch is deployed to the `int` environment

- Once all the sprint tickets are complete the `main` branch can be fast-forwarded and deployed to the ref and then production environments.



5.1.4.3 Acceptance Test Driven Development

ATDD is an application of TDD at the system level: encompassing potentially multiple components including UI, backend and persistence.

See Canonical TDD[10] if unfamiliar with the process.

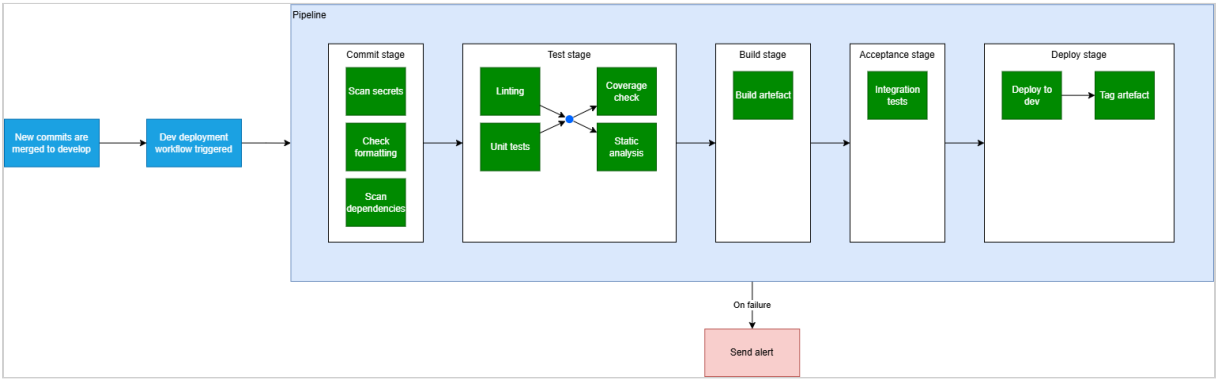
1. Dev team selects a ticket from the sprint backlog according to priorities, dependencies, capacity and preference
2. Dev team (importantly including any QA engineers) and any needed SME's create a draft list of test cases that will pass when the acceptance criteria are met
    a. This is a living document that should be amended as work is completed, lessons are learned, and new cases thought of
    b. Thus it should be shared and collaboratively editable
3. Dev team selects one such test and writes it as an executable test - verifying that it initially fails
    a. Recommendation is to pick a test that is anticipated to require the least effort to implement
4. Dev team implements the functionality to make the test pass
    a. This may involve inner "loops" of canon TDD, with the same lists of test cases
    b. Since we have a hexagonal architecture, this will be for the core business logic and the inbound and outbound adapter layers
5. Each time an acceptance test passes, the development team review the whole piece of work to identify refactoring opportunities
    a. Any time something is refactored, all automated tests should be run again to verify nothing was broken
6. If there are more test cases, start again at step 3, otherwise the ticket is done.

5.1.4.4 Automated Pipelines

The following process is carried out **every time changes to the trunk branch** are made.

10. https://tidyfirst.substack.com/p/canon-tdd

The process will run quality checks against the current state and, if passing, will update the dev environment with the latest changes and publish a prerelease that can be deployed to independent environments.



1. New commits are pushed to develop.
2. The dev build and deployment workflow is triggered. This runs a full automated pipeline against the latest changes.
    a. If any element of the pipeline fails, an alert should be sent to the development team.
3. The pipeline should create a GitHub artefact and release, deploy it to the dev environment, and tag it with a version.

5.1.4.5 Concerns and mitigations

As these approaches may be unfamiliar to many readers, we describe how to handle various potentially problematic situations here.

- **Software teaming** (the whole team work together on one thing at a time instead of splitting into solo silos and then trying to reintegrate later)
- **Test-driven development** (writing tests that describe the intended new behaviour prior to trying to solve the problem)
- **Feature slicing** (keeping tickets small; instead of asking "how big is this piece of work?", ask "can we split this piece of work into smaller bits?")
- **Robust automated pipelines** (automatic processes that will ensure only quality artefacts are allowed to progress through the environments towards prod)

5.1.4.5.1 Won't this break trunk more often?

Probably, but that's okay. The trunk (`develop`) is a workbench where the development team will be persisting work to share amongst themselves. It is allowed to get messy.

We handle this concern with:

- Software teaming: only one feature will be worked on at a time, so the "mess" will be limited
- TDD: with the test for a new behaviour written first, we guarantee that if it isn't actually delivered by a commit, it will fail

- Robust pipelines: if a commit is pushed with an incomplete implementation of a new behaviour, the pipelines will guarantee the team is notified and that the artefact won't be built as a release candidate, won't be deployed to dev, and won't be allowed to progress through the environments towards prod. The team must address the failure before an increment can be progressed.

5.1.4.5.2 Won't this result in lots of merge conflicts?

If we were using TDB alone, yes, but we won't: we'll also be restricting WIP to **one**. (In very rare cases we might allow **two**, but extra care will have to be taken to ensure integration goes smoothly.)

We handle this concern with:

- Software teaming: by only having one new feature being developed at once, there is no risk of different workstreams interfering with each other

5.1.4.5.3 What if we're due to release and there's a feature that is only partially completed?

This is a plausible concern and is mitigated by appropriate planning, tracking, and engineering discipline.

Specifically, we handle this concern with:

- Software teaming: by only having one new feature being developed at once, we have the option to release an earlier artefact built before we started the in-progress feature.
- TDD: specifically, Acceptance-test driven development, where tests are written for the broader feature first, even when smaller portions of the functionality can be delivered and verified incrementally.
- Robust pipelines: By having our acceptance tests running automatically in the pipeline, our ATDD tests will be there from the start of the new piece of work such that the pipeline will fail until the whole feature has been delivered, even if we push smaller increments in the meanwhile.
- Feature slicing: by keeping deliverables small, we are less likely to be in a partially-complete (failing) situation. We could still be in a situation where what we have delivered, functionally correct and stable, is *business-useless* until further features are also completed. Feature slicing helps us project when the larger piece of work will be complete and plan the release accordingly. By constantly evaluating this, we should know ahead of time if we are unlikely to be able to complete the feature in time. In that case, we may choose to release an earlier increment before work started on the larger deliverable, or hide the new features behind a feature toggle.

5.1.4.5.4 What if QA rejects a change when the developers have already started some new work?

The QA engineer is part of the dev team, so with actual software teaming, they will be involved with the refinement, the initial test scoping, *the development*, and the final validation of the increment; the other developers likewise will be involved in the final validation process. If for any reason the QA engineers do silo themselves off to validate an increment and the other developers start a new piece of work, a number of options are available according to the priority and urgency of the finding:

1. (Low priority or PO disagrees it's a blocking defect) Create a new ticket with the QA feedback for the next sprint, or later. If PO agrees it is needed prior to the next release, JIRA can be used to ensure it isn't forgotten for the release, as that's a manual process anyway.
2. (Moderate priority) The newly started work can be finished and then the defect addressed as top priority afterwards. This is when context switching is seen as an impediment to smooth, constant, delivery but the defect should still be fixed as soon as possible afterwards.
3. (High priority) The newly started work can be reverted and the defect addressed immediately with automated tests in place to verify it. The reverted work can then be reinstated, resolving any conflicts from the parallel work, using the automated tests to ensure the fixed behaviour and all previous expected behaviours are still in place, and then continuing as normal.

Corollary: when a defect does make its way into the production environment and a hotfix is needed:

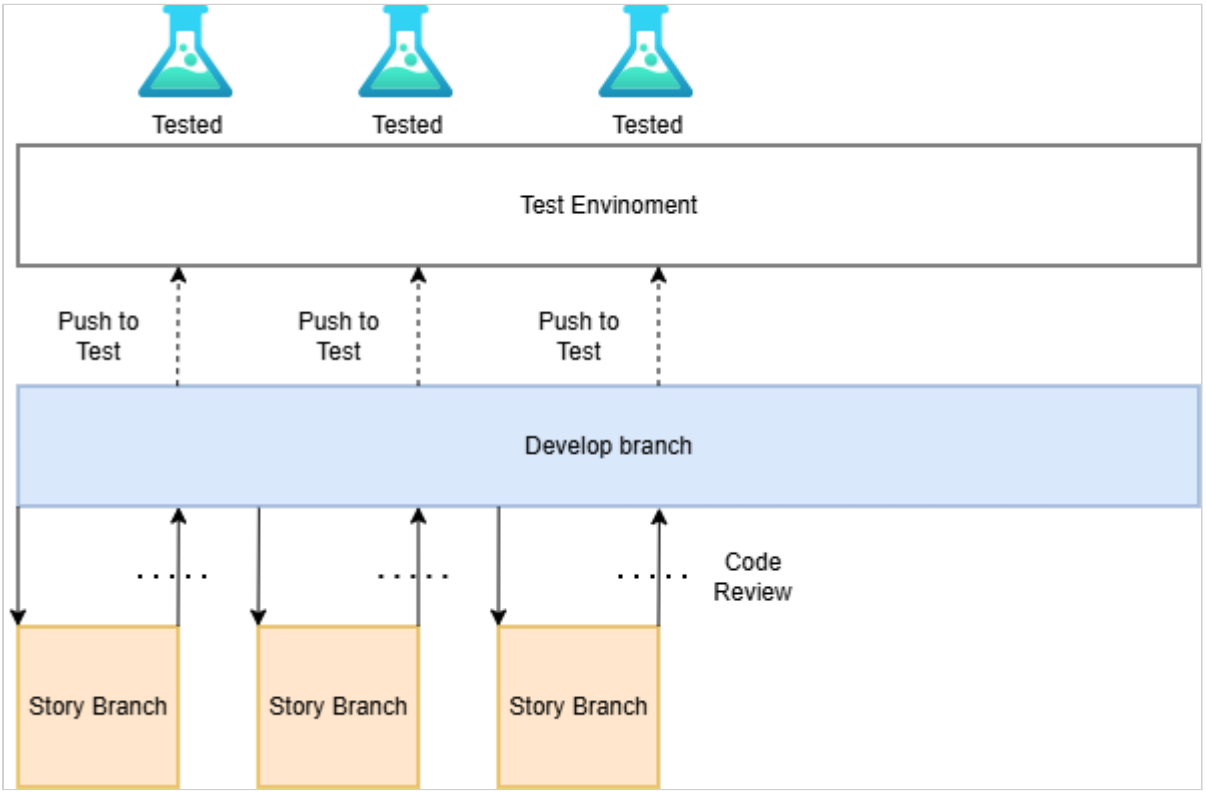
- The guiding principle is to ensure the `main` branch can be "fast-forwarded" to the next release.
- Revert all commits since the latest (defective) release.
- Patch the issue and release with all required assurance and governance.
- Revert the revert and use the automated tests to ensure no behaviour is lost.
- (Note: reverting merges is awkward, so our practices of TBD and software teaming make this safe.)

5.1.4.6 In-Sprint Development and Testing Process

5.1.4.6.1 Purpose

This process defines how a story moves from active development to tested completion within a sprint. The objective is to protect code quality through automated testing and peer review, then validate behaviour in the test environment before the story is marked complete.

5.1.4.6.2 In-Sprint Diagram



5.1.4.6.3 Roles and Responsibilities

- **Developer**: Implements the story, runs unit/system tests, opens the pull request, and addresses feedback.
- **Peer Reviewer**: Reviews the pull request and approves only when standards and quality expectations are met.
- **Tester**: Deploys to test, validates the feature, and confirms acceptance criteria.

5.1.4.6.4 Process Flow

1. Create a feature branch from **develop** when work begins.
2. Follow the branch naming standard: `feature/JIRA-123-short-description`.
3. Example: `feature/NPT-000-example-name`.
4. Complete implementation on the feature branch.
5. Run and pass all required unit and system tests.
6. Open a pull request from the feature branch into **develop**.
7. Complete peer code review.
8. Do not merge until review feedback is resolved and approval is granted.
9. Merge into **develop** only after required checks pass.
10. When the tester is ready to begin testing Push the **develop** branch to the test environment.

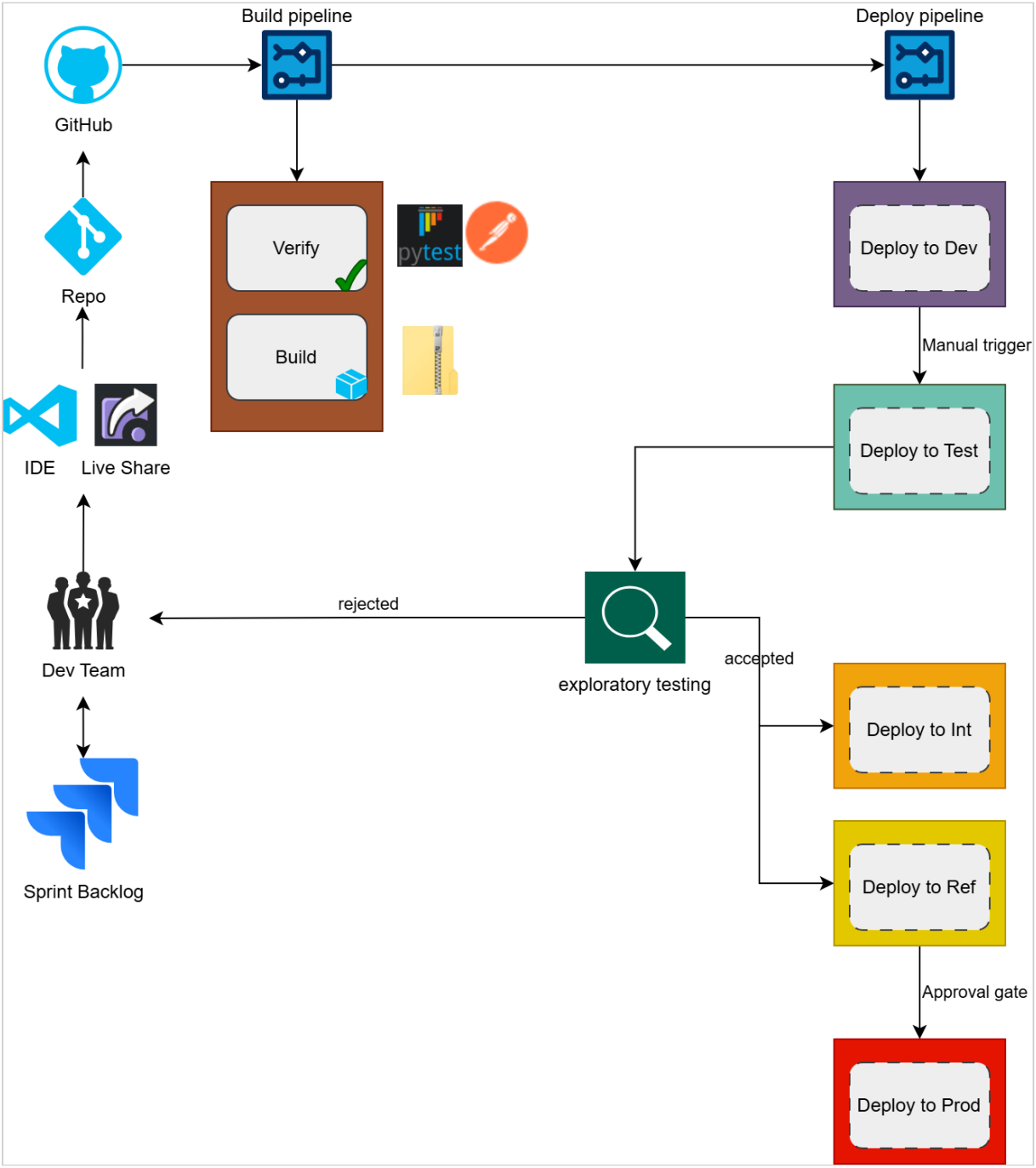
11. Tester validates the feature against story acceptance criteria.
12. If testing passes, move the story to **Done**.
13. If testing fails, log defects and return work to development.
14. Developer applies fixes and repeats review/testing until acceptance criteria are met.

5.1.4.7 Rules Summary

- Always branch from **develop**.
- Use branch naming standard: `feature/NPT-123-short-description`.
- Ensure unit and system tests pass before opening a pull request.
- Do not merge without peer review approval.
- Validate in the test environment after merge.
- Move to **Done** only after successful testing.
- Failed testing requires fixes and retesting.

5.1.5 Release process

The following process is for **building releasable artefacts** that can be **promoted through the non-production** environments and eventually into the **production** environment. The process is similar to that used for building a prerelease.



- When the developer updates the release branch this will trigger the build process.
  - Secret scanning will check for any passwords or API keys that have been accidentally included in the branch.
  - Unit tests will run and update the code coverage statistics
  - Version number updated
  - The application will be built
  - Artefact will be created

    - Artefact will be added to the repository.
    - Artefact will be pushed onto the development environment
- Once the code has been deployed the development environment
    - The developer can confirm the code is working as expected
- If the code is working as expected a tester will trigger a release to the test environment when they are ready
    - QA will be able to test the ticket against the acceptance criteria of the story
    - and do any exploratory testing
- If required a release can be triggered to the ref environment.
    - This can be used for load/performance testing

5.1.5.1 Rollback Procedures

5.1.5.1.1 Rollback a Content Deployment

Before being used in Production

- Delete the affected tables
- Deploy the fixed forward content

After being used in Production

- Instruct host system providers request the previous working version

5.1.5.1.2 Rollback a Deployment

If a released version has issues:

1. Identify the previous stable version from Git tags
2. Go to GitHub Actions → Deploy Release Pipeline[11]
3. Click **Run workflow**
4. Enter previous release tag (e.g., `v1.1.0`)
5. Select target environment: `test`, `int`, `ref`, or `prod`
6. Click **Run workflow**
7. Review Terraform plan
8. Manual approval is required before applying changes for policies, account wide and application deployments
9. Wait for completion
10. Verify:
    - Policies infrastructure deployed ✓
    - Account wide infrastructure deployed ✓
    - Application infrastructure deployed ✓

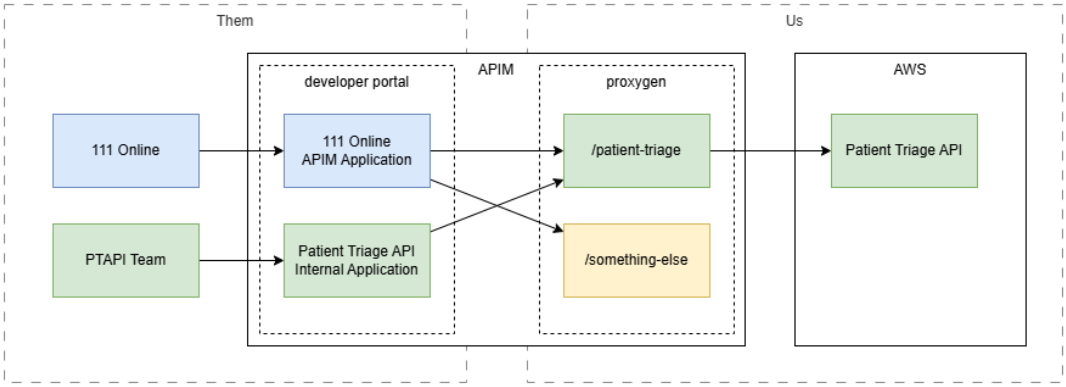
11. https://github.com/NHSDigital/ftrs-directory-of-services/actions/workflows/pipeline-deploy-release.yaml

- Services running ✓
- Automation tests pass ✓

5.2 JWT and OAuth2.0 with APIM

This page describes how the Patient Triage API uses JWT and OAuth2.0 with APIM for consumer authentication.

5.2.1 Overview



The diagram above shows how APIM and our Patient Triage API relate, and uses 111 Online as an example consumer.

Our responsibility is to deploy the API itself (in AWS), and tell proxygen how requests to APIM should be authenticated and then reach us.

Our consumers are responsible for creating their application in APIM via the Developer and Integration Hub[12]. An APIM "application" is an entry describing how their service wants to consume APIs that are hosted by APIM, such as ours. Requests from their service to `/patient-triage` should include authentication parameters (such as OAuth2.0 access tokens) which APIM will map to the corresponding application, verify that has been granted access to the API, and that the authentication matches what is required by it..

12. https://digital.nhs.uk/developer

5.2.2 Onboarding a New Client

5.2.2.1 Creating a Key Pair

https://digital.nhs.uk/developer/guides-and-documentation/security-and-authorisation/application-restricted-restful-apis-signed-jwt-authentication#step-2-generate-a-key-pair

First you need a Key ID. These must be unique across your application. A suggested format is `prod-ptapi-1`, where the `-1` suffix can be incremented with key rotations.

Then, to create your key pair:

```
KID=prod-ptapi-1
openssl genrsa -out $KID.pem 4096
openssl rsa -in $KID.pem -pubout -outform PEM -out $KID.pem.pub
```



Do **not** send us your private key. We do not need it, we do not want it, and if we get it we'll insist you create a new key pair.

Next you need the modulus of the private key:

```
MODULUS=$(
    openssl rsa -pubin -in $KID.pem.pub -noout -modulus `# Print modulus of public key` \
    | cut -d '=' -f2                                    `# Extract modulus value from output` \
    | xxd -r -p                                         `# Convert from string to bytes` \
    | openssl base64 -A                                 `# Base64 encode without wrapping lines` \
    | sed 's|+|-|g; s|/|_|g; s|=||g'                    `# URL encode as JWK standard requires`
)
```

and with it, build the JWKS with the format:

```
{
  "keys": [
    {
      "kty": "RSA",
      "n": "{{MODULUS}}",
      "e": "AQAB",
      "alg": "RS512",
      "kid": "{{KID}}",
      "use": "sig"
```

```
    }
  ]
}
```

5.2.2.2 Hosting the Public Key

https://digital.nhs.uk/developer/guides-and-documentation/security-and-authorisation/application-restricted-restful-apis-signed-jwt-authentication#step-2-generate-a-key-pair:~:text=For%20production%20applications%2C%20complete%20our%C2%A0onboarding%20enquiry%20form.

Use the onboarding form to tell APIM you want to register your generated JWKS with your application and that it should have access to the Patient Triage API.

5.2.3 Using our Service

5.2.3.1 Creating a JWT

https://digital.nhs.uk/developer/guides-and-documentation/security-and-authorisation/application-restricted-restful-apis-signed-jwt-authentication#step-4-generate-and-sign-a-jwt

There are 3 parts to a JWT: the header, the payload, and the signature.

The header:

```
{
  "alg": "RS512",
  "typ": "JWT",
  "kid": "{KID}"
}
```

The payload:

```
{
  "iss": "{API Key}",
  "sub": "{API Key}",
  "aud": "{audience}",
  "jti": "{randomUUID()}",
  "exp": "{now+5minutes}"
}
```

where `aud` is the corresponding APIM environment for `https://api.service.nhs.uk/oauth2/token` and `exp` is a unix timestamp in seconds.

The signature is made with the private key.

Example in JavaScript

```
const uuid = require('uuid');
const jwt = require('jsonwebtoken');

module.exports = function (secrets) {
  const { privateKey, endpoint, apiKey, keyId } = secrets;

  const claims = {};
  const config = {
    algorithm: 'RS512',
    expiresIn: '5m',
    audience: endpoint,
    issuer: apiKey,
    jwtid: uuid.v4(),
    subject: apiKey,
    keyid: keyId
  };

  const signedJwt = jwt.sign(claims, privateKey, config);
  return signedJwt;
};
```

5.2.3.2 Requesting an Access Token

The JWT is not for us: that goes to the APIM OAuth2.0 server. If valid, they will reply with a token (a short string) that you can use in your messages to our application in APIM.

To get an access token, send the following to `https://api.service.nhs.uk/oauth2/token` (or appropriate subdomain):

Verb: `POST`
Content-Type: `application/x-www-form-urlencoded`
Body:

```
grant_type=client_credentials\
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer\
&client_assertion={{Signed JWT}}
```

The response will be a JSON document like:

```
{
  "access_token": "pvSyidMABchdFDG6l2jNvabAm2Gi",
  "expires_in": "599",
  "token_type": "Bearer",
  "issued_at": "1790271008387"
```

```
}
```

`token_type` can be ignored

`access_token` is what you need to store and use when Querying our Service (see below)

`issued_at` is a unix timestamp in milliseconds

`expires_in` is the duration in seconds that the token will be valid for, from the `issued_at` instant

Note: Refresh tokens are not supported. When the token expires, just request a new one.

5.2.3.3 Querying our Service

To use our service with authorisation, send a request with the header:

```
Authorization: Bearer {{access token}}
```

To avoid errors, we recommend getting a new token ahead of time.

To avoid delays, we recommend getting a new token independently of your triage requests.

5.2.4 Preparing an Environment

5.2.4.1 Update OpenAPI Spec

`infrastructure/common/api-spec.yaml`

```
# ...
components:
  securitySchemes:
    # ...
    app-level3:
      $ref: https://proxygen.ptl.api.platform.nhs.uk/components/securitySchemes/app-level3
# ...
```

This is already in place.

Each: `infrastructure/environments/{env}/x-nhsd-apim.yaml`

```
x-nhsd-apim:
  access:
    - grants:
```

```
        app-level3: []
# ...
```

5.2.4.2 Onboard Ourselves

Follow the steps in the Onboarding section on this page, but also store the data in AWS.

- Open the AWS console for the appropriate environment
- Navigate to AWS Secrets Manager
- Open the appropriate `saet-{env}-account-wide-secret`
- Click 'Retrieve Secret Value'
- Click 'Edit'
- Add the following entries:
  - `apikey` (should already be there, but this is the API key from the NHS Digital developer account environment access portal)
  - `oauth_private_key` : the PEM-format private key, where new lines are encoded as literal `\n` sequences
  - `oauth_key_id` : the KID used when creating the key pair

5.3 Local environment

This page contains guidance on how technical colleagues can prepare to work on the IHWC (API) workstream.

5.3.1 Mac

5.3.1.1 Access

Most access will be sorted out without any actions by you, as per 1.1.1 Joiner Setup (before joining)[13]. You do not need to visit that page, but may want to refer to it to track your own onboarding progress. See 1.1.2 Onboarding Checklist (for new joiners)[14] for your actions regarding joining the wider Griffin / Pathways Transformation team.

- **Teams** - "NHS" account, "NHS Pathways Transformation" team
- **Github** - "NHSDigital" organization - "saet-triage-api" repository (need to add '@nhs.net[15]' email to your GitHub account to authorise access to the repo)
- **Confluence** - "NHS Pathways Transformation" space

13. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1222135921/1.1.1+Joiner+Setup+before+joining
14. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1222135931/1.1.2+Onboarding+Checklist+for+new+joiners
15. http://nhs.net

- **Jira** - "NPT" project, "Patient Triage API" board
- **Mural**
- **AWS** - "NHS England Safe and Effective Triage - Pathways" account (998245890657)
- **Pathways Web**
- ...

5.3.1.2 Software

- An IDE (Visual Studio Code[16] recommended)
- Postman[17]
- Docker[18] (or brew / Kainos Company Portal if applicable)
- Python[19] (version 3.13) (or brew?)
- Make (see repo instructions for details)
- JMeter[20]
  - Requires Java
- Localstack[21]

5.3.1.3 Techniques and Practices

Some ways-of-working to read up on and get familiar with at a theoretical level.

- hexagonal architecture
- (acceptance) test-driven development
- trunk-based development
- pair-programming / software teaming

5.3.1.4 Step by step

1. Open a terminal (zsh recommended)
2. Install git if not already available

```
command -v git >/dev/null 2>&1 || brew install git
```

16. https://code.visualstudio.com/
17. https://www.postman.com/downloads/
18. https://www.docker.com/products/docker-desktop/
19. https://www.python.org/downloads/release/python-3130/
20. https://jmeter.apache.org/download_jmeter.cgi
21. https://docs.localstack.cloud/aws/getting-started/installation/

3. Clone the repo to a destination of your choice - leave unspecified to create a new directory under the current one

```
git clone git@github.com:NHSDigital/saet-triage-api.git [target directory]
```

4. Change directory into the newly cloned repository, e.g.

```
cd saet-triage-api
```

5. Ensure you have the latest version of `make`, and other dependencies

```
brew install make jq asdf gnu-sed
```

6. Configure your local system (this installs `pre-commit`, `terraform` and `gitleaks`)

```
make config
```

7. Install and start docker
8. Install localstack[22]
9. Verify local environment

```
make clean-slate deploy-local test-integration-local
```

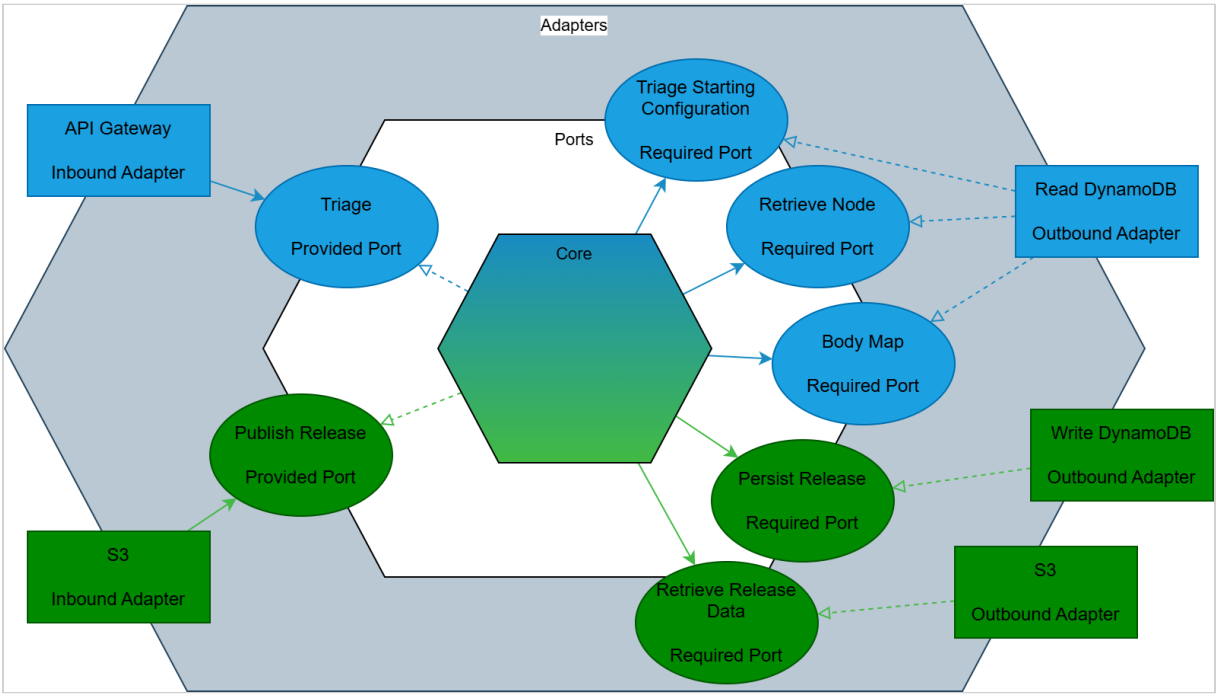
22. https://docs.localstack.cloud/aws/getting-started/installation/

10. Set up commit signing: https://docs.github.com/en/authentication/managing-commit-signature-verification/signing-commits

5.3.2 Windows (tbd later)

5.4 Triage Engine Architecture

The engine's architecture is hexagonal[23], with a core agnostic of our infrastructure choices. The nomenclature used in this page is "Incoming adapter" → "Provided port" → "core" → "Required port" → "Outbound adapter". (Spellings "adapter" and "adaptor" may be used interchangeably.)



(As of 16 Oct 2025)

5.4.1 Provided ports

It exposes two key pieces of functionality (the inbound ports): the ability to **Publish a Release**, and to perform a **Triage** against a release.

5.4.1.1 Publish Release

This port is responsible for ensuring a new release is available for triage, accepting a message consisting of a reference to the release, as a string. It is completely ignorant of the form of that string, only assuming that it should be understood by the outbound adaptor. In practice, at present, it is an S3 object name (filename).

23. https://en.wikipedia.org/wiki/Hexagonal_architecture_(software)

1. Receive message: `[release reference]`.
2. Ask required port ("retrieve release data") to provide the contents of the release corresponding to `release reference` in the structures defined by the core.
3. Ask required port ("persist release") to persist the release data.
4. Return success.

Currently consumed by an S3 event adaptor, triggered by putting a new file into a specific S3 bucket.

5.4.1.2 Triage

This port is responsible for deriving the next step needed by the operator, accepting a message consisting of the demographics of the patient, the skillset of the operator, and the answers to any already answered questions. Its response will contain any combination of a question, some care advice, and an outcome. The full assessment and summary content up to the current stage of triage will also be returned.

1. Receive message: `[demographics, skillset, answers]`.
2. Ask required port ("triage starting configuration") for the `coordinate` of the first node according to `demographics` and `skillset`, and other starting configuration details.
3. Ask required port ("retrieve node") for the `node` at `coordinate`.
4. Process `node`:
    a. evaluate the type of `node` to take different actions
    b. for **question** nodes, examine `answers` for if it has already been answered - take the answer's `target` if so and go to step 3 with `coordinate=target`
    c. for **body map** "nodes", examine `answers` for if a selection has already been made - take the selection if so and go to step 3 with `coordinate=selection.0`
    d. for **variable** setting nodes, store the value of the variable, take its `target` and go to step 3 with `coordinate=target`
    e. for **branch** nodes, try to read the value of the variable and match it against a set of rules; if the variable hasn't been written or doesn't match any rules, take the default path ("*")
    f. for **care advice** nodes, record the required care advice items and immediately proceed to the node's `transfer`
    g. for **disposition** nodes, record the provided disposition and immediately proceed to the node's `transfer`
5. When unable to progress further (because no answer provided to a question node, or no question to ask at all) return `{question?, care advice?, outcome?, report?, display?}`

Currently consumed by an APIG event adaptor, triggered by http requests to apigee, passed on to our AWS API Gateway.

5.4.2 Required ports

The things required from outside the engine are to: **retrieve release data**, **persist release**, fetch the **triage starting configuration**, **retrieve node** and fetch a **body map.**

5.4.2.1 Retrieve Release Data

This port is responsible for retrieving a pathways release and converting it into the core's domain objects. Currently implemented as an S3 adaptor, it accepts a reference to a release (the name of the file/object put into the S3 bucket) and requests its contents from the bucket. This object is assumed to be zipped, containing .csv files with particular names, as per the current standard release archives. It extracts the files in memory, converts them into domain objects and returns them to the core.

1. Receive message: `[release reference]`.
2. Request object `release reference` from the configured S3 bucket.
3. Extract file in-memory.
4. Analyse CSV files and map to domain objects.
5. Return domain objects.

5.4.2.2 Persist Release

This port is responsible for persisting the release data in a form more readily accessible to the triage engine than the original release file. Currently implemented as a DynamoDB adaptor.

1. Receive message: `[starting configurations, triage nodes, body maps]`
2. Convert these three collections into DynamoDB items and write them to their respective tables
3. Return success

5.4.2.3 Triage Starting Configuration

This port is responsible for determining how to start a pathways triage, determined by the skillset of the operator, and possibly the demographics of the patient. All those data must be supplied to keep the core ignorant of the details about when they are needed or not. This includes the coordinate of the first node to retrieve and the display type for the skillset (e.g. "Online").

1. Receive message: `[skillset, age, gender, party]`
2. Query the DynamoDB table for a matching entry
3. Return `[coordinate, display]`

5.4.2.4 Retrieve Node

This port is responsible for retrieving all of the information needed for a visit to a new node in domain objects. It must not make any decisions about what to do with the node - just convert it and present it back to the core.

1. Receive message: `[coordinate]`
2. Query the DynamoDB table for a matching entry
3. Return `[triage node]`

5.4.2.5 Body Map

This port is responsible for retrieving the possible body map selections for the current triage.

1. Receive message: `[skillset, age, gender, party]`
2. Query the DynamoDB table for a matching entry
3. Return `[body map]`

5.4.3 Data structures

5.4.3.1 DynamoDB

5.4.3.1.1 StartingConfigs

JSON Schema for the StartingConfigs table

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "Skillset": {
      "description": "The skillset of the operator. Case-insensitive.",
      "type": "string"
    },
    "GenderAgeParty": {
      "description": "A code representing the gender, age and party of the patient.
E.g., 'FA1' means 'F'emale, 'A'dult, '1'st person",
      "type": "string"
    },
    "Coordinate": {
      "description": "The coordinate of the node at which triage should start.",
      "type": "string"
    }
  },
  "required": [ "Skillset", "GenderAgeParty", "Coordinate" ]
}
```

5.4.3.1.2 TriageNodes

JSON Schema for the TriageNodes table

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "Coordinate": {
      "description": "The coordinate of this node",
      "type": "string"
    },
    "PathwayTitle": {
      "description": "The title of the pathway this node is found in",
      "type": "string"
    },
    "Node": {
      "description": "The contents of this node. Structure differs according to which
type of node it is.",
      "type": "object",
      "allOf": [
        { "$ref": "#/$defs/TriageNode" },
        {
          "oneOf": [
            { "$ref": "#/$defs/QuestionNode" },
            { "$ref": "#/$defs/VariableNode" },
            { "$ref": "#/$defs/BranchNode" },
            { "$ref": "#/$defs/CareAdviceNode" },
            { "$ref": "#/$defs/DispositionNode" }
          ]
        }
      ]
    }
  },
  "required": [ "Coordinate", "PathwayTitle", "Node" ],

  "$defs": {
    "Transfer": {
      "description": "Common elements for when a transfer needs to be described",
      "type": "object",
      "properties": {
        "Transfer": {
          "description": "Where to navigate to next if this answer is selected by the
operator",
          "type": "string"
        },
        "DispositionText": {
          "description": "The text associated with a disposition. Only needed when
'Transfer' is to a Dx code.",
          "type": "string"
        },
        "SdCode": {
          "description": "The code for the symptom discriminator associated with the
pathway of the transfer destination",
```

```
          "type": "string"
        },
        "SdDescription": {
          "description": "The description of the symptom discriminator associated with
the pathway of the transfer destination",
          "type": "string"
        }
      },
      "required": [ "Transfer" ]
    },
    "TriageNode": {
      "description": "Common elements for all nodes in this table",
      "type": "object",
      "properties": {
        "Type": {
          "description": "An distinguisher for the type of node being represented",
          "type": "string"
        },
        "SgCode": {
          "description": "The code for the symptom group associated with this node's
pathway",
          "type": "string"
        },
        "SgDescription": {
          "description": "The description of the symptom group associated with this
node's pathway",
          "type": "string"
        }
      },
      "required": [ "Type" ]
    },
    "QuestionNode": {
      "description": "A node holding a question to be put to the operator, along with
the possible answers",
      "type": "object",
      "properties": {
        "Type": {
          "description": "Distinguishes this node as holding a question",
          "const": "Question"
        },
        "QuestionId": {
          "description": "An identifier for the question held by this node",
          "type": "string"
        },
        "QuestionText": {
          "description": "The text of the question to be presented to the operator",
          "type": "string"
        },
        "Answers": {
          "type": "array",
          "items": {
            "allOf": [
              { "$ref": "#/$defs/Transfer" },
              {
                "type": "object",
                "properties": {
                  "Number": {
```

```
                    "description": "An identifier for this answer to the question",
                    "type": "string"
                  },
                  "Text": {
                    "description": "The text of the answer to be presented to the
operator",
                    "type": "string"
                  },
                  "PathwaysAssessment": {
                    "description": "The text to be added to the pathways assessment for
this triage",
                    "type": "string"
                  },
                  "ConsultantSummary": {
                    "description": "The text to be added to the consultant summary for
this triage",
                    "type": "string"
                  },
                  "Transfer": {
                    "description": "Where to navigate to next if this answer is
selected by the operator",
                    "type": "string"
                  },
                  "SdCode": {
                    "description": "The code for the symptom discriminator associated
with the pathway of the transfer destination",
                    "type": "string"
                  },
                  "SdDescription": {
                    "description": "The description of the symptom discriminator
associated with the pathway of the transfer destination",
                    "type": "string"
                  }
                },
                "required": [ "Number", "Text", "Transfer" ]
              }
            ]
          }
        }
      },
      "required": [ "QuestionId", "QuestionText", "Answers" ]
    },
    "VariableNode": {
      "description": "A node that defines a variable to be set",
      "allOf": [
        { "$ref": "#/$defs/Transfer" },
        {
          "type": "object",
          "properties": {
            "Type": {
              "description": "Distinguishes this node as holding a variable
assignment",
              "const": "Variable",
            },
            "Name": {
              "description": "The name of the variable to set or overwrite",
              "type": "string",
```

```
            },
            "Value": {
              "description": "The value to set or overwrite the variable to",
              "type": "string",
            }
          },
          "required": [ "Type", "Name", "Value" ]
        }
      ]
    },
    "BranchNode": {
      "description": "A node that branches to different places according to rules",
      "type": "object",
      "properties": {
        "Type": {
          "description": "Distinguishes this node as holding a branch point",
          "type": "string"
        },
        "VariableName": {
          "description": "The name of the variable to compare against the rules",
          "type": "string"
        },
        "Rules": {
          "description": "The rules against which to compare the variable",
          "type": "array",
          "items": {
            "allOf": [
              { "$ref": "#/$defs/Transfer" },
              {
                "type": "object",
                "properties": {
                  "Comparator": {
                    "description": "The operator to use to compare the variable with
the value. '*' is the catch-all for when none of the rules match or the variable is not
defined at all.",
                    "enum": [ "==", "<", ">", "<=", ">=", "*" ]
                  },
                  "Value": {
                    "description": "The value to compare against",
                    "value": "string"
                  }
                },
                "required": [ "Comparator" ]
              }
            ]
          }
        }
      },
      "required": [ "Type", "VariableName", "Rules" ]
    },
    "CareAdviceNode": {
      "description": "A node that provides care advice to be presented to the operator
before proceeding",
      "allOf": [
        { "$ref": "#/$defs/Transfer" },
        {
          "type": "object",
```

```
          "properties": {
            "Type": {
              "description": "Distinguishes this node as holding care advice",
              "const": "CareAdvice"
            },
            "CxId": {
              "description": "An identifier for this collection of care advice",
              "type": "string"
            },
            "CareAdviceItems": {
              "description": "The textual content of all of the care advice to present
to the operator",
              "type": "array",
              "items": {
                "type": "object",
                "properties": {
                  "Number": {
                    "description": "The ordinal of the care advice item",
                    "type": "string"
                  },
                  "Text": {
                    "description": "The textual content of the care advice to present
to the operator",
                    "type": "string"
                  }
                },
                "required": [ "Number", "Text" ]
              }
            }
          },
          "required": [ "Type", "CxId", "CareAdviceItems" ]
        }
      ]
    },
    "DispositionNode": {
      "description": "A node that establishes a disposition outcome",
      "allOf": [
        { "$ref": "#/$defs/Transfer" },
        {
          "type": "object",
          "properties": {
            "Type": {
              "description": "Distinguishes this node as holding a disposition",
              "const": "Disposition"
            },
            "DxId": {
              "description": "An identifier for this disposition",
              "type": "string"
            },
            "DispoCode": {
              "description": "The code for the disposition",
              "type": "string"
            },
            "DispoText": {
              "description": "The textual description of the disposition",
              "type": "string"
            }
```

```
          },
          "required": [ "Type", "DxId", "DispoCode", "DispoText" ]
        }
      ]
    }
  }
}
```

5.4.3.1.3 BodyMaps

JSON Schema for the BodyMaps table

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "id": {
      "description": "An identifier for this body map. Takes the form <age
group>.<gender>.<trauma>; e.g., 'Adult.Male.Trauma'.",
      "type": "string"
    },
    "groups": {
      "description": "Groups collect physically related body parts/areas",
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "group_id": {
            "description": "A human-readable name for the group",
            "type": "string"
          },
          "items": {
            "description": "Parts/areas of the body",
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "item_id": {
                  "description": "A human-readable name for the body part/area",
                  "type": "string"
                },
                "pathways": {
                  "description": "The actual selections the operator can make on the
body map display",
                  "type": "array",
                  "items": {
                    "type": "object",
                    "properties": {
                      "pw_title": {
                        "description": "The title of the pathway being presented",
                        "type": "string"
                      },
                      "pw_id": {
```

```
                        "description": "The internal ID of the pathway",
                        "type": "string"
                      },
                      "pw_guidance_use_text": {
                        "description": "Guidance to the operator about when this
pathway should be selected",
                        "type": "string"
                      },
                      "pw_guidance_do_not_use_text": {
                        "description": "Guidance to the operator about when this
pathway should NOT be selected",
                        "type": "string"
                      }
                    },
                    "required": [ "pw_title", "pw_id", "pw_guidance_use_text",
"pw_guidance_do_not_use_text" ]
                  }
                }
              },
              "required": [ "item_id", "pathways" ]
            }
          }
        },
        "required": [ "group_id" ]
      }
    }
  },
  "required": [ "id", "groups" ]
}
```

5.5 Working with dev workspaces

5.5.1 Log in

5.5.1.1 Prepare aws-cli

Write to ~/.aws/config:

```
[profile Default]
sso_session = Default
sso_account_id = 998245890657
sso_role_name = SAET-RW-Developer
region = eu-west-2
output = json

[sso-session Default]
sso_start_url = https://identitycenter.amazonaws.com/ssoins-75359606988cfdf9
sso_region = eu-west-2
sso_registration_scopes = sso:account:access
```

or use: `aws configure sso`

In a shell:

```
export AWS_PROFILE=Default
```

Consider adding this to your shell's rc file (e.g., ~/.zshrc)

5.5.1.2 Actually log in

To check if you are logged in:

```
aws sts get-caller-identity --no-cli-pager
```

To log in if not:

```
aws sso login
```

5.5.2 Get the endpoint URL

```
TICKET='NPT-1234' \
API_ID="$(aws apigateway get-rest-apis \
  --query 'items[?ends_with(name,to_string(`$TICKET`))].id' \
  --output text \
  --no-cli-pager)"
echo "https://$API_ID.execute-api.eu-west-2.amazonaws.com/v1/FHIR/R4/triage"
```

5.5.2.1 Brucey bonus

If you're in your project directory (specifically, with git), you can use the below to get the ticket number of your branch - assuming it's included!

```
git_current_branch | grep -o -e '[[:digit:]]\+'
```

5.5.3 Example using aws configure sso

```
➜  ~ aws configure sso
SSO session name (Recommended): Default
SSO start URL [None]: https://identitycenter.amazonaws.com/ssoins-75359606988cfdf9
```

```
SSO region [None]: eu-west-2
SSO registration scopes [sso:account:access]:
Attempting to automatically open the SSO authorization page in your default browser.
If the browser does not open, open the following URL:

https://oidc.eu-west-2.amazonaws.com/authorize
There are 3 AWS accounts available to you.
Using the account ID 998245890657
The only role available to you is: SAET-RW-Developer
Using the role name "SAET-RW-Developer"
Default client Region [us-east-1]: eu-west-2
CLI default output format (json if not specified) [json]:
Profile name [SAET-RW-Developer-998245890657]: Default
The AWS CLI is now configured to use the default profile.
Run the following command to verify your configuration:

aws sts get-caller-identity
➜  ~ aws sts get-caller-identity

An error occurred (InvalidClientTokenId) when calling the GetCallerIdentity operation:
The security token included in the request is invalid.
➜  ~ aws sso login

Missing the following required SSO configuration values: sso_start_url, sso_region. To
make sure this profile is properly configured to use SSO, please run: aws configure sso
➜  ~ export AWS_PROFILE=Default
➜  ~ aws sts get-caller-identity --no-cli-pager
{
    "UserId": "random:you@nhs.net",
    "Account": "998245890657",
    "Arn": "arn:aws:sts::998245890657:assumed-role/AWSReservedSSO_SAET-RW-
Developer_9498ccb693101224/you@nhs.net"
}
➜  ~
```

6 API Documentation

6.1 API Certificates

Page owner: Richard Beggs
Last reviewed: 20 Oct 2026
Review cadence: Six Monthly

Use this register to track certificates, expiry dates, owners, and renewal procedures. Add a link to the approved renewal instructions for each certificate.

Certificate / service	Environment	Purpose / endpoint	Owner	Expiry date	Renewal instructions link	Alert / renewal lead time	Status / last checked
APIM mtls secret	Prod	mTLS connection with PTAPI backend	Andrew Walker	26 Mar 2027	APIM mTLS Certificate Renewal Runbook[24]	30 Days	Active / 20 Oct 2026
	Ref		Andrew Walker	10 Apr 2029		30 Days	Active / 20 Oct 2026
	Int		Andrew Walker	20 Mar 2029		30 Days	Active / 20 Oct 2026
	Test		Andrew Walker	10 Apr 2029		30 Days	Active / 20 Oct 2026
	Dev		Andrew Walker	13 Mar 2029		30 Days	Active / 20 Oct 2026

24. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1345209264/APIM+mTLS+Certificate+Renewal+Runbook

Certificate / service	Environment	Purpose / endpoint	Owner	Expiry date	Renewal instructions link	Alert / renewal lead time	Status / last checked
truststore	Prod	mTLS connection with APIM	Andrew Walker	07 Apr 2042	TBC	30 Days	Active / 20 Oct 2026
	Non-prod		Andrew Walker	04 Aug 2032	TBC	30 Days	Active / 20 Oct 2026

6.1.1 Proxygen keys

Environment	Expiry date	Renewal instructions
Prod	02 Apr 2027	Ask to rotate the key at: https://nhsdigitallive.service-now.com/csm?id=sc_cat_item&sys_id=5e94fa6597f7a650dd80f2df9153af41&sysparm_category=63f4716697731e90dd80f2df9153affe
Ref	27 Jan 2027	
Int		
Test		
Dev		

6.1.2 Handling notes

- Include TLS, client-authentication, signing, and other certificates that could affect service availability.
- Link to certificate or secret locations in the approved vault. Do not store private keys, passwords, or secret material on this page.
- After renewal, record the new expiry date and verify the certificate is installed and trusted by its consumers.

6.2 Triage API documentation guidelines

This page is designed to provide guidance to team members who are updating the API specification, and the other published technical documentation relating to the use of the API.

Published documentation relating to the API can be found in the following locations:

Location	URL	Purpose
API specification	Triage API - NHS England Digital[25]	The primary source of information for potential consumers of the Triage API, and developers implementing a solution using the API. **Should**: • Provide an overview of the API, details of how to onboard with it and the service level available • Provide technical information relating specifically to the Triage API, such as the endpoints available, request and response schemas, environments available for testing and integration, and authentication methods • Introduce the triage concepts referred to in the API requests and response, such as dispositions, care advice **Should not:** • Describe the operation of other NHS APIs used in the implementation of a UEC solution. These should only be signposted in the Related APIs[26] section • Describe how an end-to-end solution using the Triage API should be implemented • Contain specific details relating to how/where the data returned by the API should be displayed in a user interface. The exception to this is where data should not be presented to the user, or relayed to the patient by a call handler • Need to justify technical design decisions (e.g. Rate Limit), or delivery details (e.g. Service Level) **Specific guidelines** 1. Enumerations should be detailed in the schema section of the API specification
Developer Hub	Documentation, guides and tutorials - NHS England Digital[27] Urgent & Emergency Care Interoperability & Technical Standards | uec-tech-standards[28]	Supporting documentation relating to building a solution using the Triage API **Should**: • Provide information related to building an end-to-end UEC solution that includes the Triage API **Should not:** • Duplicate the information in the API specification

25. https://digital.nhs.uk/developer/api-catalogue/triage-api
26. https://digital.nhs.uk/developer/api-catalogue/triage-api#overview--related-apis
27. https://digital.nhs.uk/developer/guides-and-documentation
28. https://developer.nhs.uk/apis/uec-tech-standards/

Location	URL	Purpose
Simplifier.net	NHS Digital FHIR Medicines[29]	

6.3 Triage API Specification

Published version is here Triage api - NHS England Digital[30].

The working version of the spec is in GitHub here → saet-triage-api/docs/user-guides/api-spec.yaml at main · NHSDigital/saet-triage-api[31].



Previous versions of the OAS created before the yaml file was added to GitHub can be found below and can be copied into https://editor.swagger.io/ for rendering

File	Modified
Triage API 0.2.oas.txt[32]	Oct 06, 2025 by Gareth Dempsey[33]
Triage API 0.3.oas.txt[34]	Oct 08, 2025 by Gareth Dempsey[35]
Triage API 0.4.oas.txt[36]	Oct 13, 2025 by Andrew Walker[37]
Triage API 0.1.oas.txt[38]	Oct 13, 2025 by Andrew Walker[39]
Triage API 0.5_first_published_version.oas.txt[40]	Nov 17, 2025 by Gareth Dempsey[41]

29. https://simplifier.net/guide/nhsdigital-medicines/Home?version=2.8.18
30. https://digital.nhs.uk/developer/api-catalogue/triage-api
31. https://github.com/NHSDigital/saet-triage-api/blob/main/docs/user-guides/api-spec.yaml
32. https://nhsd-confluence.digital.nhs.uk/download/attachments/1145310485/Triage%20API%200.2.oas.txt?api=v2
33. https://nhsd-confluence.digital.nhs.uk/display/~gareth.dempsey1@nhs.net
34. https://nhsd-confluence.digital.nhs.uk/download/attachments/1145310485/Triage%20API%200.3.oas.txt?api=v2
35. https://nhsd-confluence.digital.nhs.uk/display/~gareth.dempsey1@nhs.net
36. https://nhsd-confluence.digital.nhs.uk/download/attachments/1145310485/Triage%20API%200.4.oas.txt?api=v2
37. https://nhsd-confluence.digital.nhs.uk/display/~andrew.walker46@nhs.net
38. https://nhsd-confluence.digital.nhs.uk/download/attachments/1145310485/Triage%20API%200.1.oas.txt?api=v2
39. https://nhsd-confluence.digital.nhs.uk/display/~andrew.walker46@nhs.net
40. https://nhsd-confluence.digital.nhs.uk/download/attachments/1145310485/Triage%20API%200.5_first_published_version.oas.t
41. https://nhsd-confluence.digital.nhs.uk/display/~gareth.dempsey1@nhs.net

7 API Environments

7.1 Overview

This page provides a clear, organized view of our **Development (Dev)**, **Integration (Int)**, and **Production (Prod)** environments. Each environment serves a distinct purpose in our delivery pipeline—from early experimentation and feature testing to system-wide validation and finally live, customer-facing operations. Use this dashboard to quickly understand the status, purpose, and access points for each environment, helping ensure smooth development workflows and reliable deployments.

7.2 Environments

7.2.1 Core Environments

Name	Account	Internal DNS Name	API-M Environment	Authentication	Externally Available	Data source	Purpose
Integration (int)	aws.nhsepathways-saet-test@nhsdigital.nhs.uk[42]	**int.**patient-triage-api.nhs.uk	int **Apigee Env:** Prod **External:** Yes **Base URL:**	Private JWT	Yes		Used by internal and external parties with a need to integrate with triage api:

42. mailto:aws.nhsepathways-saet-test@nhsdigital.nhs.uk

Name	Account	Internal DNS Name	API-M Environment	Authentication	Externally Available	Data source	Purpose
Reference (ref)	aws.nhsepathways-saet-test@nhsdigital.nhs.uk[43]	**ref.**patient-triage-api.nhs.uk	ref **Apigee Env:** Prod **External:** No **Base URL:**	Private JWT	No		This environment should always mirror the current production infrastructure, except for a defined period prior to a release. Uses: • pre-production deployment • final regression/ smoke testing • volumetric and pre-release testing • performance/load/ stress testing
Production (prod)	aws.nhsepathways-saet-prod@nhsdigital.nhs.uk[44]	**prod.**patient-triage-api.nhs.uk	prod **Apigee Env:** Prod **External:** No **Base URL:**	Private JWT	Yes		The production infrastructure for Pathways API that delivers the functionality used for live clinical use cases.

43. mailto:aws.nhsepathways-saet-test@nhsdigital.nhs.uk
44. mailto:aws.nhsepathways-saet-prod@nhsdigital.nhs.uk

7.2.2 Development Environment

Name	Account	Internal DNS Name	API-M Environment	Authentication	Externally Available	Data source	Purpose
Development (dev)	aws.nhsepathways-saet-dev@nhsdigital.nhs.uk[45]	**dev.**patient-triage-api.nhs.uk	dev **Apigee Env:** Prod **External:** Yes **Base URL:**	Private JWT	Yes		Kept up to date with latest changes merged to the main branch of the project via a CD pipeline. Used for integration testing of changes to the main branch.
Test (test)	aws.nhsepathways-saet-test@nhsdigital.nhs.uk[46]	**test.**patient-triage-api.nhs.uk	test **Apigee Env:** Prod **External:** No **Base URL:**	Private JWT	No		Used to run end to end tests and an environment to allow testers to test in isolation.

7.3 Next steps

1. Create a new ticket for building each environment [(see page 82) Richard]

Int
NPT-777
Test
NPT-777
Ref
NPT-778[47] - Build the Ref environment for the API NOT REQUIRED

45. mailto:aws.nhsepathways-saet-dev@nhsdigital.nhs.uk
46. mailto:aws.nhsepathways-saet-test@nhsdigital.nhs.uk

Prod
NPT-779

7.4 Provisioning Environments

7.4.1 Develop

7.4.1.1 AWS Resources to be Provisioned

7.4.1.1.1 IAM

Create an execution role to be attached to both lambda functions

AWS managed policies to be added to the execution role:

- AWSLambdaBasicExecutionRole
- AWSXRayDaemonWriteAccess
- AWSLambdaDynamoDBExecutionRole

Custom policies to be added to the execution role:

- LambdaS3ReadAccess

Policy (JSON)

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "s3:GetObject",
            "Resource": "arn:aws:s3:::pathways-data-bucket/*"
        }
    ]
}
```

- DynamoDBLambdaAccess

Policy (JSON)

```
{
```

47. https://nhsd-jira.digital.nhs.uk/browse/NPT-778?src=confmacro

```
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:BatchGetItem",
        "dynamodb:BatchWriteItem",
        "dynamodb:Query",
        "dynamodb:Scan"
      ],
      "Resource": "arn:aws:dynamodb:eu-west-2:998245890657:table/*"
    }
  ]
}
```

7.4.1.1.2 DynamoDB

For each release version, the following tables will need to be provisioned

For all tables:

Parameter	Value/Setting
region	"eu-west-2"
billing-mode	PAY_PER_REQUEST
sse-specification	Enabled=true

PathwaysStartingNodes-{version}

AttributeName	AttributeType	KeyType
Skillset	S (String)	HASH
GenderAgeParty	S (String)	RANGE

PathwaysTriageNodes-{version}

AttributeName	AttributeType	KeyType
Coordinate	S (String)	HASH

PathwaysBodyMapNodes-{version}

AttributeName	AttributeType	KeyType
id	S (String)	HASH

7.4.1.2 Deploying Development Environment

7.4.1.2.1 Repo Location

The repo locations have been split up into multiple locations so our own private registry of Terraform is established. The reasons behind this approach means that we are creating safe repeatable patterns that can be versioned to have a specific Development environment version of the module. Having a private registry means that the source itself is controlled by the Pathways Team meaning that all aspects of the deployed code is controlled by the team.

7.4.1.2.2 Main Deployment Repo

https://github.com/NHSDigital/saet-aws-deploy-terraform

Modules

https://github.com/NHSDigital/saet-aws-restapi-terraform

https://github.com/NHSDigital/saet-aws-s3-terraform

https://github.com/NHSDigital/saet-aws-lambda-terraform

https://github.com/NHSDigital/saet-aws-trigger-terraform

https://github.com/NHSDigital/saet-aws-dynamodb-terraform

https://github.com/NHSDigital/saet-aws-iam-terraform

7.4.2 Int

7.4.3 Test

7.4.4 Ref

7.4.5 Production

8 API Logging

This page describes the intended approach for logging by the triage engine and release loader components.

⚠ = partially implemented

✅ = fully implemented

8.1 Events

8.1.1 Deployment

Event	Level	Details and log message
NewLambdaPublished		Publishing a new lambda version will not exercise the code itself.
NewPathwaysPublished	INFO	Publishing a new pathways release will trigger an `s3:ObjectCreated:*` event that will trigger the lambda. Technically, more than one event can be reported in a single lambda invocation, but for now we'll assume we receive one at a time.
		`Release file published: NHSPathways48.2.2ReleaseData.zip (1929455B)`
LoadPathwaysReleaseFileFailed	WARN	The lambda will then try to read this file into memory, which could fail for various reasons

		`Failed to load file NHSPathways48.2.2ReleaseData.zip because "error message"`
		`Stack Trace`
		`...`
		`...`

LoadPathwaysReleaseFileSucceeded	INFO	Successfully establish connection with S3 and read the file into a python object ready for processing
		`Loaded file NHSPathways48.2.2ReleaseData.zip`
ExtractPathwaysReleaseFileFailed	WARN	The lambda will then try to unzip the file, which could fail

Event	Level	Details and log message
		`Failed to extract files from NHSPathways48.2.2ReleaseData.zip because "error message" Stack Trace ... ...`
ExtractPathwaysReleaseFileSucceeded	INFO	Successfully extract the archived files, still just in memory as the lambda is serverless. Should be able to see filenames and sizes. (Log wrapped for readability here, but would be on one line.)
		`Extracted 26 files from NHSPathways48.2.2ReleaseData.zip: alternateText.csv (1030KB), BodyMap.csv (1KB), BodyMapOverrides.csv (1KB), CareAdvice.csv (276KB), CareAdviceExclude.csv (9KB), CareAdviceItems.csv (1009KB), CCSConditions.csv (63KB), CCSConditionSupportingInformation.csv (416KB), CCSJumps.csv (1KB), CCSTemplateConditions.csv (65KB), CCSTemplates.csv (17KB), checksum.xml (2KB), dispomap.csv (87KB), dispositionitems.csv (195KB), dispositions.csv (19KB), dxrank.csv (1KB), pathways.csv (258KB), Skillsets.csv (2KB), SymptomDiscriminators.csv (28KB), SymptomGroups.csv (9KB), systemflags.csv (1KB), SystemVariables.csv (2KB), triagelogic.csv (7890KB), TxAnswers.csv (1989KB), TxQuestions.csv (572KB), version.xml (1KB)`
IdentifyPathwaysVersionFailed	WARN	The lambda will then try to identify the version of the release by inspecting the version.xml file, which could fail for various reasons
		`Failed to identify release version from file NHSPathways48.2.2ReleaseData.zip because "error message" Stack Trace ... ...`
IdentifyPathwaysVersionSucceeded	INFO	Successfully read the version.xml file from the archive, parse the XML somehow and extract both the number and type of the release

Event	Level	Details and log message
		`Identified release version from file NHSPathways48.2.2ReleaseData.zip as 48.2.2-Production`
ParsePathwaysDataFailed	WARN	The lambda will then try to load the data it needs from the files and build an internal representation of it ready for publication to DynamoDB, which could fail for various reasons

		`Failed to load pathways 48.2.2-Production release data from file NHSPathways48.2.2ReleaseData.zip because "error message"`
		`Stack Trace`
		`...`
		`...`

ParsePathwaysDataSucceeded	INFO	Successfully read whichever files from the archive are needed into an internal model, ready for publication to DynamoDB
		`Loaded pathways 48.2.2-Production release data from file NHSPathways48.2.2ReleaseData.zip`
CreateReleaseTableFailed	WARN	The lambda will then try to create the required new DynamoDB tables, each of which could fail for various reasons

		`Failed to create DynamoDB table Pathways_48.2.2-Production_Nodes because "error message"`
		`Stack Trace`
		`...`
		`...`

CreateReleaseTableSucceeded	INFO	Successfully use the SDK to create a needed DynamoDB table
		`Created DynamoDB table Pathways_48.2.2-Production_Nodes`
PublishReleaseFailed	WARN	With the tables in place and the data ready to be persisted, the lambda will then try to populate the tables, which could fail for various reasons

		`Failed to populate DynamoDB table Pathways_48.2.2-Production_Nodes because "error message"`
		`Stack Trace`
		`...`
		`...`

Event	Level	Details and log message
PublishReleaseSucceeded	INFO	Successfully use the SDK to populate the dynamoDB tables with all of the required data from the zip
		`Populated Pathways_48.2.2-Production_Nodes successfully`

8.1.1.1 Example 1: Happy path

```
[2025-08-27T12:38:41.119Z] [INFO] Release file published:
NHSPathways48.2.2ReleaseData.zip (1929455B)
[2025-08-27T12:38:44.800Z] [INFO] Loaded file NHSPathways48.2.2ReleaseData.zip
[2025-08-27T12:38:46.929Z] [INFO] Extracted 26 files from
NHSPathways48.2.2ReleaseData.zip: alternateText.csv (1030KB), BodyMap.csv (1KB),
BodyMapOverrides.csv (1KB), CareAdvice.csv (276KB), CareAdviceExclude.csv (9KB),
CareAdviceItems.csv (1009KB), CCSConditions.csv (63KB),
CCSConditionSupportingInformation.csv (416KB), CCSJumps.csv (1KB),
CCSTemplateConditions.csv (65KB), CCSTemplates.csv (17KB), checksum.xml (2KB),
dispomap.csv (87KB), dispositionitems.csv (195KB), dispositions.csv (19KB), dxrank.csv
(1KB), pathways.csv (258KB), Skillsets.csv (2KB), SymptomDiscriminators.csv (28KB),
SymptomGroups.csv (9KB), systemflags.csv (1KB), SystemVariables.csv (2KB),
triagelogic.csv (7890KB), TxAnswers.csv (1989KB), TxQuestions.csv (572KB), version.xml
(1KB)
[2025-08-27T12:38:47.002Z] [INFO] Identified release version from file
NHSPathways48.2.2ReleaseData.zip as 48.2.2-Production
[2025-08-27T12:38:49.388Z] [INFO] Loaded pathways 48.2.2-Production release data from
file NHSPathways48.2.2ReleaseData.zip
[2025-08-27T12:38:51.508Z] [INFO] Created DynamoDB table Pathways_48.2.2-
Production_Nodes
[2025-08-27T12:38.52.544Z] [INFO] Created DynamoDB table Pathways_48.2.2-
Production_EntryPoints
[2025-08-27T12:39.01.489Z] [INFO] Populated Pathways_48.2.2-Production_Nodes
successfully
[2025-08-27T12:39.04.072Z] [INFO] Populated Pathways_48.2.2-Production_EntryPoints
successfully
```

8.1.2 Triage

Event	Level	Details and log message
TriageRequestReceived	INFO	The earliest possible point at which we can say "something has communicated with us"
		`Triage request received: [uuid]`
ParseTriageRequestFailed	WARN	If we cannot extract the required information to pass to the engine for any reason, the parse has failed
		`Failed to parse triage request [uuid]`
ParseTriageRequestSucceeded	INFO	A representation of the successful parse, showing what data was extracted in canonical form. (Log wrapped for readability here, but would be on one line.)
		`Triage request details for [uuid]: skillset = "111 Call Handler", gender = "F", ageType = "DOB", dob = "2000-01-31", party = "1", clinicalArea = "8", answers = ["Tx224076" = "3", "TX2990122" = "1"]`
TriageStarted ⚠️	INFO	A single triage operation has several stages, the first being to evaluate all of the data already provided. This log marks the time at which the business logic started.
		`Triage started for [uuid]`
FirstNodeIdentified ⚠️	INFO	The first step is always to identify where to start, retrieving the node coordinates (pathway ID and order number) and transfer code (T1..T4) to be used. If this fails, it indicates an incompatibility between the client and server.
		`Triage [uuid] starting at PW101.5 using transfers T2`
NodeRetrievalFailed	WARN	Most of the calls out made by the lambda will be to DynamoDB to retrieve details of a single node. We'll want to monitor this closely to decide if we need to add resilience with retries or caching etc.

		`Could not retrieve node PW101.5 for triage [uuid] because "error message"`
		`Stack Trace`
		`...`
		`...`

Event	Level	Details and log message
NodeRetrievalSucceeded ⚠	INFO	This tells us each hop made towards the next step or outcome. When the node is interpreted, a further event will be logged.
		`Triage [uuid] progressing to PW101.5`
NextQuestionRetrieved ⚠	INFO	When the retrieved node is identified as a question.
		`Triage [uuid] progressed to PW101.5, "Tx224076" which is the question "How much wood could a woodchuck chuck if a woodchuck could chuck wood?"`
CareAdviceRetrieved ⚠	INFO	When the retrieved node is identified as inline care advice.
		`Triage [uuid] progressed to PW101.5, "Cx123456" which is the care advice "Don't eat yellow snow."`
VariableSet ⚠	INFO	When the retrieved node is identified as a variable-setting node.
		`Triage [uuid] progressed to PW101.5, which sets variable "FOO" to "present"`
VariableRead ⚠	INFO	When the retrieved node is identified as a variable-reading / branching node.
		`Triage [uuid] progressed to PW101.5, matching condition "PATIENT_AGE<65"`
DispositionReached	INFO	When a Dx code is encountered. Not necessarily the end of triage, but when we will supply the SGSDDx codes
		`Triage [uuid] reached outcome: Symptom Group = SG2014, Symptom Discriminator = SD1193, Disposition = Dx110`
PathwayChanged	INFO	When we start looking at a new pathway for a triage
		`Triage [uuid] jumped from PW101 to PW2384`

9 APIM Proxygen Environment setup

Currently the following GitHub flow step is used to update the sandbox environment with APIM:

cicd-4-deploy-sandbox.yaml

```
- name: "Update APIM"
  run: |
    pip install proxygen-cli
    proxygen --version
    proxygen settings set api ${{ secrets.APIM_API_NAME }}
    mkdir -p ~/.proxygen
    touch ~/.proxygen/private.pem
    echo "${{ secrets.APIM_PRIVATE_KEY }}" > ~/.proxygen/private.pem
    touch ~/.proxygen/credentials.yaml
    echo "base_url: ${{ secrets.APIM_BASE_URL }}" >> ~/.proxygen/credentials.yaml
    echo "client_id: ${{ secrets.APIM_CLIENT_ID }}" >> ~/.proxygen/credentials.yaml
    echo "client_secret: ${{ secrets.APIM_CLIENT_SECRET }}" >> ~/.proxygen/credentials.yaml
    echo "key_id: ${{ secrets.APIM_KEY_ID }}" >> ~/.proxygen/credentials.yaml
    echo "password: ''" >> ~/.proxygen/credentials.yaml
    echo "private_key_path: private.pem" >> ~/.proxygen/credentials.yaml
    echo "username: ''" >> ~/.proxygen/credentials.yaml
    API_ID=$(aws apigateway get-rest-apis --query "items[?name=='triageAPI'].id" --output text)
    API_URL="https://${API_ID}.execute-api.${{ needs.metadata.outputs.aws_region }}.amazonaws.com/sandbox"
    # Remove everything from # x-nhsd-apim comment onwards (including any previous x-nhsd-apim content)
    sed -i '/^# x-nhsd-apim/,$d' ./docs/user-guides/api-spec.yaml
    # Re-add the comment marker
    echo "# x-nhsd-apim" >> ./docs/user-guides/api-spec.yaml
    cat <<EOT >> ./docs/user-guides/api-spec.yaml
    x-nhsd-apim:
      target:
          ype: external
        url: ${API_URL}
        healthcheck: /_status
        security:
          type: apikey
          header: x-api-key
          secret: example-api-key
      access:
        - title: 'Triage API'
          grants:
            app-level0: []
      ratelimiting:
        app-default:
          limit: 10000
          timeunit: "second"
    EOT
```

```
    echo y | proxygen instance deploy sandbox triage-api ./docs/user-guides/
api-spec.yaml
    echo y | proxygen spec publish ./docs/user-guides/api-spec.yaml
```

This flow will get the secrets from the GitHub repo used to set up APIM for the Patient Triage API, store them locally then deploy the Sandbox API with the API Spec.

The 'x-nhsd-apim' yaml is appended to the api spec, which configures it for deploying to APIM

If this flow is run for the first time for a different environment e.g. Prod, the api key needs to be configured to the APIM environment for the deployment to APIM to work.

To do this, the following script must be added and run before the 'proxygen instance deploy'

Put secret into APIM environment

```
# Put the API key into a file to be used for proxygen and set on sandbox
environment
touch ~/.proxygen/triage-api-key.txt
echo "${{ secrets.APIM_API_KEY }}" > ~/.proxygen/triage-api-key.txt
proxygen secret put sandbox example-api-key --apikey --secret-file
~/.proxygen/triage-api-key.txt
```

This will store the api key secret, already on the repo, locally in a text file then run the command to set up the environment with the api key.

Once run once successfully, this can be removed. The original GitHub flow step shown will run and deploy the API to APIM successfully once the environment is set up correctly.

9.1 APIM Onboarding Questions

- **How do we know when APIM is down. Should we have a health check we monitor etc, what is that, what do other teams do?**
  - There is a subscription page you can subscribe for any service issues/MIs. However, APIM is not on there at the moment which is something we have raised and getting added to the list. The link below requires HSCN network or can register when you're in the office.
    https://nww.digital.nhs.uk/servicemanagement/status/sub/[48]

48. https://gbr01.safelinks.protection.outlook.com/?url=https%3A%2F%2Fnww.digital.nhs.uk%2Fservicemanagement%2Fstatus%2Fsub%2F&data=05%7C02%7Crichard.beggs2%40nhs.net%7Cdf31cf7a16b14c4000ae08de529961d8%7C37c354b285b047f5b22207b48d774ee3%7C0%7C0%7C639039015882235814%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlY

Generally, if Apigee is down, everything on it would be down too with the agreement from Google that they will spring up a new instance from when it went down almost immediately.

- **Is there any more config needed to secure the traffic from APIM > API. Do we have certificates in place or need to put them in place? (internal question for the devs). If you have any documentation covering this that would help.**
  - This page guides you to the API Key or the mTLS options and how to set them up in Proxygen

    Configuring your connection to your API backend with Proxy Generator - API Platform - CDT Confluence[49]
- **If we move from API Key to JWT, is that purely APIM config or something else we need to do at our side. Can we just trust that APIm is managing that filtering process on our behalf and trust the traffic. Any conf pages explaining this?**
  - JWT would include providing APIM with a key and KID. APIM would use to authorise and generate an access token. More information is here[50]
- **How do we manage accounts in APIM, will we get an Apigee login? Can we get a login?**
  - We can provide you with Apigee login to view your API. This will use your hscic login and can be requested through Service now[51].
- **Team members need access to APIM splunk for API monitoring.**

  andrew.walker46@nhs.net[52],
  richard.beggs2@nhs.net[53],

---

iOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=z0k9zfVajnjZmJRnSWJNmbV%2BrME7xgrdb3dR%2B5MFV%2Bk%3D&reserved=0

49. https://gbr01.safelinks.protection.outlook.com/?url=https%3A%2F%2Fnhsd-confluence.digital.nhs.uk%2Fspaces%2FAPM%2Fpages%2F678899046%2FConfiguring%2Byour%2Bconnection%2Bto%2Byour%2BAPI%2Bbackend%2Bwith%2BProxy%2BGenerator&data=05%7C02%7Crichard.beggs2%40nhs.net%7Cdf31cf7a16b14c4000ae08de529961d8%7C37c354b285b047f5b22207b48d774ee3%7C0%7C0%7C639039015882245556%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=uJ31SgE2T1Ah%2BR0xXuj8T%2FzLbPaiPBWouPgzjBN%2BPUA%3D&reserved=0
50. https://gbr01.safelinks.protection.outlook.com/?url=https%3A%2F%2Fdigital.nhs.uk%2Fdeveloper%2Fguides-and-documentation%2Fsecurity-and-authorisation%2Fapplication-restricted-restful-apis-signed-jwt-authentication%23step-1-register-your-application-on-the-api-platform&data=05%7C02%7Crichard.beggs2%40nhs.net%7Cdf31cf7a16b14c4000ae08de529961d8%7C37c354b285b047f5b22207b48d774ee3%7C0%7C0%7C639039015882253800%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=0NYTW0gZO8cPU58Pulobr%2BxB50YbJpU63Eufs2M%2B2B4%3D&reserved=0
51. https://gbr01.safelinks.protection.outlook.com/?url=https%3A%2F%2Fnhsdigitallive.service-now.com%2Fcsm%3Fid%3Dsc_category%26sys_id%3D63f4716697731e90dd80f2df9153affe%26catalog_id%3D65bcd377c3011200b12d9f2974d3aea0&data=05%7C02%7Crichard.beggs2%40nhs.net%7Cdf31cf7a16b14c4000ae08de529961d8%7C37c354b285b047f5b22207b48d774ee3%7C0%7C0%7C639039015882262736%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=uDHWaUHXhqUJ4xBpZUGD6AxHB0BqRy8bjdQ0vVhF76E%3D&reserved=0
52. mailto:andrew.walker46@nhs.net

gareth.nelson1@nhs.net[54],
afzal.mufti1@nhs.net[55],
tharack.ahmed2@nhs.net[56]

- Splunk access can be requested through https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/117652647/Get+access+to+Azure+AD+groups+and+Splunk#GetaccesstoAzureADgroupsandSplunk-RequestingSplunkaccess[57] If you send me/Vikas an email with the shortcodes, emails etc then we can approve and get it set up.

53. mailto:richard.beggs2@nhs.net
54. mailto:gareth.nelson1@nhs.net
55. mailto:afzal.mufti1@nhs.net
56. mailto:tharack.ahmed2@nhs.net
57. https://gbr01.safelinks.protection.outlook.com/?url=https%3A%2F%2Fnhsd-confluence.digital.nhs.uk%2Fspaces%2FAPM%2Fpages%2F117652647%2FGet%2Baccess%2Bto%2BAzure%2BAD%2Bgroups%2Band%2BSplunk%23GetaccesstoAzureADgroupsandSplunk-RequestingSplunkaccess&data=05%7C02%7Crichard.beggs2%40nhs.net%7Cdf31cf7a16b14c4000ae08de529961d8%7C37c354b285b047f5b22207b48d774ee3%7C0%7C0%7C639039015882270989%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=AgrC%2BVLlpM06UrRVOazsvlsc6G5rQULisHp8T%2FTbpTI%3D&reserved=0

10 API Pipeline

We use github actions[58] to guarantee releasability of the triage api.

10.1 Commit stage

The commit stage will run after every update to the `main` branch. It performs the following tasks:

1. Run fast tests
2. Build the application / Construct an artefact
3. Push the artefact to a repository

The commit should only be built the once.

10.2 Acceptance stage

This stage is to ensure the artefact satisfies the acceptance criteria, as encapsulated by automated acceptance tests. The "guarantee" of releasability is closely tied to the quality of the acceptance tests. It will be run manually by developers.

1. Run smoke tests
2. Run acceptance tests

10.3 QA stage

This stage is for QA engineers to run manual and exploratory tests. It will be run manually by QA engineers.

1. Deploy to QA environment

A second signoff stage will also be provided to mark the release candidate as successful or failed.

10.4 Production stage

1. Deploy to prod environment

58. https://github.com/features/actions

11 API Release

12 API Tech stack

Which tools and technologies we use for the Safe and Effective Triage API

This tech stack has been produced from:

- Engineering Tech Radar[59]
- Engineering Red Lines[60]
- Non-Functional Requirements[61]
- Team skills preferences and existing knowledge
- Specific requirements for certain tooling abilities

Areas:

Actions needed:

Action	Why	Who

12.1 Build

Technology	Use Cases	Decision basis	Notes
GitHub	For code store	Mandated: Tech Radar & Red Lines	Must use Enterprise NHS organisation Must use repository-template Must have all repos as public
GitHub Actions	For pipelines (quick)	Mandated: Engineering Red Lines	
GitHub CoPilot	For AI coding assistance		Team (including SoW) should NOT use their own personal license as these will not honour the sensitivity agreements that we have in place with MS/AWS.

59. https://radar.engineering.england.nhs.uk/
60. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-lines.aspx
61. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/512755603/Non+Functional+Requirements+-+Triage+API

Technology	Use Cases	Decision basis	Notes
Github repository-template	As quickstart for new repos	Mandated: Engineering Red Lines	We will try to keep up to date as the repo changes, but acknowledged that this is difficult as projects diverge.
Terraform	For Infrastructure as Code	Preferred: Tech Radar	

12.2 Code analysis

Technology	Use Cases	Decision basis	Notes
Checkov, Trivy	For IaC scanning	Mandated: Tech Radar & Red Lines https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Tech-Radar-iac-scanning.aspx	Must use Enterprise NHS organisation Must use repository-template Must have all repos as public
Dependabot	For dependency scanning	Mandated: Tech Radar & Red Lines https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Vulnerability-Management---Application---Dependency-Scanning.aspx	See Managing Dependencies with Dependabot (see page 107)
SonarCloud	For static code analysis	Mandated: Engineering Red Lines https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Tech-Radar---Static-Analysis-Tooling.aspx	License free if repo public
grype	For CVE scanning	Mandated: part of the NHS template-repo https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Vulnerability-Management---Application---Dependency-Scanning.aspx	

Technology	Use Cases	Decision basis	Notes
syft	For SBOM collection	Mandated: part of the NHS template-repo https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Vulnerability-Management---Application---Dependency-Scanning.aspx	
gitleaks	For secret scanning	Mandated: part of the NHS template-repo https://github.com/gitleaks/gitleaks	

12.3 Testing

Technology	Use Cases	Decision basis	Notes
JMeter	For performance testing		Tech Radar has JMeter / Locust
Postman	For API Testing		To be confirmed

12.4 Infrastructure

Technology	Use Cases	Decision basis	Notes
AWS	For Cloud hosting		Following "you build it, you run it", teams should look to host their own infrastructure, and not rely on separate hosting teams. This encourages ownership, and limits handoffs and responsibility gaps.

Technology	Use Cases	Decision basis	Notes
AWS Lambda	For architecture approach, where possible	• Engineering Red Lines cost scaling 40% • Not running clusters / Maintenance of running containers • Lower total cost of ownership Alternatives considered: • EC2 - much more complicated, having to manage the server itself • ECS - less complicated than EC2 but still more so than lambdas. Might be reconsidered if DynamoDB integration proves problematic and we decide to switch to an in-memory approach	Use of containers to be avoided
AWS Dynamo DB	Architecture approach, DB store	• Engineering Red Lines cost scaling 40% • Managed serverless service • Core AWS service, suited for low latency high performance Alternatives considered: • MemoryDB - Overkill for our performance requirements. These can be met with Dynamo. This assumption to be tested as part of alpha PoC	
Terraform	For Infrastructure as Code	Preferred: Tech Radar	
AWS Backup	For backups / DR	Mandated: Engineering blueprint	Immutable backups guidance from SEQF[62]

Technology	Use Cases	Decision basis	Notes
AWS WAF	For Firewall rules		Must use CCOE's set of WAF rules[63]

12.5 Application Layer

Technology	Use Cases	Decision basis	Notes
NHS API-M	For API management layer	Mandated: API guidance	We shouldn't use any of our own API management (Apigee, documentation hosting, auth, etc.). It should all be handled through API-M. This solution also goes hand-in-hand with use of Digital Onboarding Service, CPM, and CIS2.
FHIR UK Core	For API message structure and interops.	Mandated: Interops policy	To be used for any new externally facing APIs (https://digital.nhs.uk/developer/guides-and-documentation/our-api-technologies#fhir) Any writing of new FHIR messages must involve the data standards team
Python	For language-choice: Lambdas	Decision based on: • Expected guidance from Engineering Technical Authority proposaing a move to consolidating around development languages. Link to ADR[64].	

62. https://github.com/NHSDigital/core-engineering-standards-documentation/blob/main/guidelines/immutable_backups.md
63. https://nhsd-confluence.digital.nhs.uk/spaces/CCEP/pages/423306252/WAF+Web+Application+Firewall
64. https://nhsd-confluence.digital.nhs.uk/x/hxd3R

12.6 Frontend

N/A

12.7 Integration

N/A

12.8 Support / Observability

Technology	Use Cases	Decision basis	Notes
AWS X-ray	For tracking events	We should default to cloud-native tools, but needs a PoC to explore whether these work for us	Assumption: we don't think we need Instana / DataDog / etc. Needs proving!
AWS CloudWatch	For system-audit For log-file analytics For operational dashboards	""	Favour automated alerts (emails, sms) over dashboards
AWS CloudTrail	For user-audit	""	
AWS CloudWatch EUM	For real user browser monitoring	""	
AWS SecurityHub	For security monitoring and quality assessments		SecurityHub should be woven into our security testing
AWS Budgets	For real-time cost monitoring		
NHS CloudHealth	For long-term cost monitoring	Mandated: NHS central solution	
Splunk	For forwarding logs to CSOC	We're aiming to ONLY use Splunk for CSOC logs. Our own log files analytics should be through CloudWatch.	

12.9 Analytics

Technology	Use Cases	Decision basis	Notes
Decision needed	For analytics / commissioning insights		Warrants an ADR / business case / who knows! We need a whole solution around this, probably involving an analytics platform (FDP, CDP, etc).

12.10 Managing Dependencies with Dependabot

When one of our dependencies has an update released, dependabot will create a PR automatically, amending where the version of that dependency is defined.

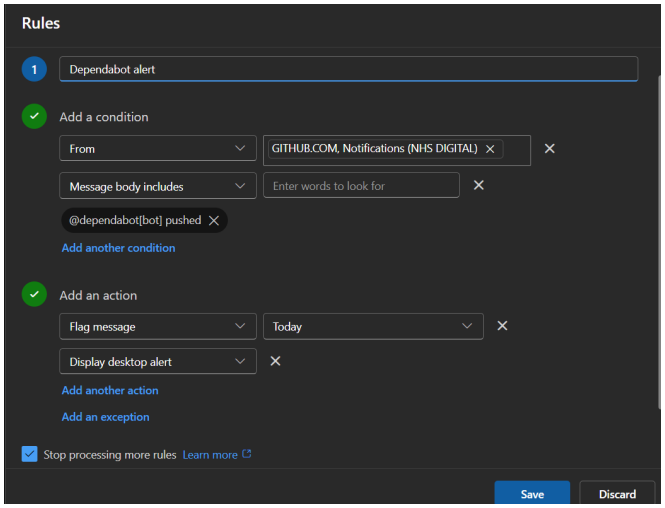
For minor releases, we can assume backwards compatibility, approve it and get it merged.

Major releases should be checked more thoroughly before approving and merging, by switching to the PR's source branch locally and doing what we can to verify nothing has broken.

When one of dependabot's PRs becomes unmergable because of new updates on the target branch, add a comment `@dependabot recreate` which will force it to reapply the change on the latest.

The very latest releases might not always be viable. There may be a delay between a release by the authors and when it is compatible with the rest of our technological ecosystem. (For example, python 3.14 was released October 2025, but AWS only supported it from November that year.)

To help ensure that dependabot PRs aren't missed, Outlook can be configured to create prominent alerts on receipt of a notification email about them.



13 API Testing

13.1 Unit testing

3 scopes: inbound adapters, core, outbound adapters

Tools: pytest

All unit tests are sociable[65].

We prefer test-driven development[66] to create our unit tests, writing the expected behaviour of each scope before implementing, in small steps.

Coverage tests will fail if LoC% ever drops below 80%. The core triage engine separately requires 95% coverage.

```
make test-unit test-coverage
```

13.1.1 Core

Tests of the business logic exposed by provided ports, with required ports replaced with test doubles. With the core isolated from any infrastructure concerns, these tests are written purely in terms of business logic, using domain objects. For example:

```
def test_retrieves_starting_config_from_persistence(dsl) -> None:
    dsl.given_starting_config(dsl.question_node(coord="PW123.10"))
    dsl.when_triage(age_group="Adult", gender="Female", party="3")
    dsl.then_requested_triage_start_config(
        age_group="Adult", gender="Female", party="3"
    )
```

and

```
def test_release_service_calls_adapter_with_correct_id(
    mock_release_adapter, mock_triage_node_adapter
) -> None:
    release_service = ReleaseService(mock_release_adapter, mock_triage_node_adapter)
    release_service.publish_release_data("releaseId")
    mock_release_adapter.retrieve_release_data.assert_called_once_with("releaseId")
```

65. https://martinfowler.com/bliki/UnitTest.html
66. https://tidyfirst.substack.com/p/canon-tdd

13.1.2 Inbound adapters

Tests of the translation of inbound lambda events to core use cases, and the response back across that inbound channel, where applicable. For example:

```
def test_question_from_engine_is_serialised_with_alt_text(dsl) -> None:
    dsl.given_result(
        display_type="ONLINE",
        question=dsl.question(
            id="Q123",
            text="What is your symptom?",
            alt_text={"ONLINE": "How are your symptoms?"},
        ),
    )
    dsl.when_event_received()
    dsl.then_response().contains_question(id="Q123", text="How are your symptoms?")
```

13.1.3 Outbound adapters

Tests of the translation of core requirements to infrastructure, and the response back from that outbound channel to the core with domain objects. For example:

```
def test_control_type_1_deserialised_to_question_node(dsl) -> None:
    dsl.given_release_data(
        {
            "triagelogic.csv": """\
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T1","T3"
"PFA1","5","Tx220074","1","-1","PW1809.0","PW1809.0"
""",
            "TxQuestions.csv": """\
"QuID","QuestionText","ControlType","Qu_Rationale"
"Tx220074","What is the reason for the contact?","1",""
""",
        }
    )
    dsl.when_release_requested()
    dsl.then_result().contains_question(coord="PFA1.5")
```

13.2 Acceptance testing

Out of date, pending on NPT-1341 and NPT-1342

Scope: HTTP message to API Gateway, through triage engine lambda to dynamodb

Tool: Postman

We use acceptance test driven development, writing acceptance tests that will pass only when desired functionality has been fully implemented. The tests are written as postman collections stored in <project root>/tests/integration[67]. As a stateless and idempotent service, set up is relatively simple.

Locally, we use localstack to deploy a simulation of the target infrastructure and automatically create a postman environment with the correct endpoint defined, using newman to execute the tests.

```
make clean-slate deploy-local test-integration
```

13.2.1 Integration Test Scenarios

This page describes the **planned** automated integration test suite of the current functionality of the Patient Triage API.

This page is not maintained. Integration test scenarios are now managed at https://github.com/NHSDigital/patient-triage-api/tree/develop/tests/integration/capabilities

PLANNED

NPT-1287[68] - [SPIKE] Update integration tests to be functionality-focused DONE

The goal is for the automated test suite to cover all of the Pathways algorithm that has been implemented. When the whole algorithm has been implemented then while the suite is passing we have confidence that we can support any release conforming to the algorithm.

They are coupled to the structure of the releases from (V)PAT, and the FHIR format of our API communications.

The scenarios and examples listed do not include everything needed to make a valid release bundle, but focus on the key elements needed to express the targeted behaviour. For example, they could describe a node of a particular form, but not include the prerequisites for the algorithm to reach that node in its analysis; the implementations of these scenarios will have to handle that.

Each scenario has up to 4 sections:

1. **Release**: Describes the contents of certain CSV files in the release bundle. Not all necessary columns or rows will be included - just those that are required to describe the scenario. (More or less mapping to "Given".)
2. **Request**: The contents of the payload from the client. Typically the operator skillset, patient demographics, and/or any question answers. (More or less mapping to "When".)

67. https://github.com/NHSDigital/saet-triage-api/tree/main/tests/integration
68. https://nhsd-jira.digital.nhs.uk/browse/NPT-1287?src=confmacro

3. **Engine**: Assertions of what our solution should do. Some of these may be described in ways that don't necessarily manifest, but the implementations of these scenarios can easily be made to verify them. (More or less mapping to "Then", along with **Response**.)
4. **Response**: Assertions about the contents returned in the response. (More or less mapping to "Then", along with **Engine**.)

Scenarios will be flagged as `TO DO` until their tests have been written and are passing.

Scenarios will be flagged as `INCOMPLETE` if the implementation doesn't fully conform to the algorithm.

13.2.1.1 Contents

13.2.1.2 Triage Start

13.2.1.2.1 Starts at the node given in Skillsets.csv when not "AgeGenderParty"

Given an ingested release with **Skillsets.csv**:

SkillsetDescription	EntryPathwayId	EntryOrderNumber
S	P	N

And **P** is not "AgeGenderParty" or "PaCCS"
When starting a triage as skillset **S**
Then triage progresses to node **P.N**

Example

Release:

Skillsets.csv

```
"SkillsetDescription","EntryPathwayId","EntryOrderNumber"
"999 Call Handler","PW999","0"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"999 Call Handler"}
```

Engine:

Navigates to node PW999.0

13.2.1.2.2 Starts at the pathway matching patient demographics when "AgeGenderParty"

Given an ingested release with **Skillsets.csv**:

SkillsetDescription	EntryPathwayId	EntryOrderNumber
S	"AgeGenderParty"	**N**

And **pathways.csv**:

PW_ID	Party	PW_Age	PW_Gender
P	**R**	**E**	**G**

When starting a triage as skillset **S**
And the patient is of gender **G**, age group **E**, and party **R**
Then triage progresses to node **P.N**

Example

Release:

Skillsets.csv

```
"SkillsetDescription","EntryPathwayId","EntryOrderNumber"
"Nurse","AgeGenderParty","5"
```

pathways.csv

```
"PW_ID","Party","PW_Age","PW_Gender"
"PN1","1","Adult","Female"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"Nurse"},
{"type":{"text":"gender"},"valueString":"Female"},
{"type":{"text":"ageGroup"},"valueString":"Adult"},
{"type":{"text":"party"},"valueString":"1"}
```


Engine:

Navigates to node PN1.5

13.2.1.2.3 Missing scenarios

None identified

13.2.1.3 Transfers

13.2.1.3.1 Transfers within the same pathway for non-negative JumpID

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	AnswerNo	JumpID
P	N	A	J ≥ 0

And the triage is currently at node **X.Y**
When processing answer **A** from coordinate **P.N**
Then the triage progresses to node **P.J**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","AnswerNo","JumpID"
"P123","0","1","5"
```

Engine:

(Processes node P123.0 with answer number 1)

Navigates to node P123.5

13.2.1.3.2 Transfers with -1 JumpID (go to)

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	AnswerNo	JumpID	T*n*
P	N	A	"-1"	T

And **Skillsets.csv**:

SkillSetDescription	Tbranch
S	n

When processing answer **A** from coordinate **P.N** as skillset **S**
Then the triage progresses to **T**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","AnswerNo","JumpID","T2"
"P123","0","1","-1","P456.100"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T2"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Engine:

(Processes node P123.0 with answer number 1)

Navigates to node P456.100

13.2.1.3.3 Transfers with -6 JumpID (1st person safety pathway)

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	AnswerNo	JumpID
P	N	A	"-6"

When processing answer **A** from coordinate **P.N**
Then triage progresses to node SP1.0

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","AnswerNo","JumpID"
"P123","0","1","-6"
```

Engine:

(Processes node P123.0 with answer number 1)

Navigates to node SP1.0

13.2.1.3.4 Transfers with -4 JumpID after a -6 (safety pathway return)

Given an ingested release with **triagelogic.csv**:

PW_ID	**OrderNo**	**AnswerNo**	**JumpID**	**T*n***
P	**N1**	**A1**	"-6"	**T**
"SP1"	**N2**	**A2**	"-4"	

And **Skillsets.csv**:

SkillSetDescription	**Tbranch**
S	**n**

And having previously processed answer $\mathbf{A_1}$ from coordinate $\mathbf{P.N_1}$ as skillset **S**
When processing answer $\mathbf{A_2}$ from coordinate SP1.$\mathbf{N_2}$ as skillset **S**
Then triage progresses to node **T**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","AnswerNo","JumpID","T2"
"P123","0","1","-6","P456.100"
"SP1","5","3","-4",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"Dental"}
```

Engine:

(Processes node P123.0 with answer number 1 as skillset Dental)

Navigates to node P456.100

13.2.1.3.5 Transfers with -9 JumpID (end triage)

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	AnswerNo	JumpID
P	N	A	"-9"

When processing answer **A** from coordinate **P.N**
Then triage concludes

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","AnswerNo","JumpID"
"P123","0","1","-9"
```

Engine:

Concludes the triage, returning appropriate final content.

13.2.1.3.6 Transfers with -2 JumpID (body map)

TO DO

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	AnswerNo	JumpID
P	N	A	"-2"

When processing answer **A** from coordinate **P.N**
Then triage progresses to a **body map**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","AnswerNo","JumpID"
"P123","0","1","-2"
```

Engine:

(Processes node P123.0 with answer number 1)

Progresses to the body map

13.2.1.3.7 Transfers with -3 JumpID (3rd person safety pathway)

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	AnswerNo	JumpID
P	N	A	"-3"

When processing answer **A** from coordinate **P.N**
Then triage progresses to node SP3.0

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","AnswerNo","JumpID"
"P123","0","1","-3"
```

Engine:

(Processes node P123.0 with answer number 1)

Navigates to node SP3.0

13.2.1.3.8 Missing scenarios

- What is "appropriate final content"?

13.2.1.4 Question Nodes

13.2.1.4.1 Original question text

Given an ingested release with **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**T**	"1"

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	<blank>

When returning question **Q** as skillset **S**
Then the response uses **T** for the question text

Example

Release:

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx123456","Why are we here?","1"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"111 Call Handler",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456",
  "text": "Why are we here?",
}
```

13.2.1.4.2 Original answer text

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType
Q	"1"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerText
Q	**N**	**A**

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	<blank>

When returning question **Q** as skillset **S**
Then the response uses **A** for the text for answer **N**

Example

Release:

TxQuestions.csv
"QuID","ControlType"

```
"Tx123456","1"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerText"
"Tx123456","2","Not sure"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"111 Call Handler",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456.2.Answer",
  "type": "choice",
  "answerOption": [
    {
      "valueCoding": {
        "code": "2",
        "display": "Not sure"
      }
    }
  ]
}
```

13.2.1.4.3 Original question rationale

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType	Qu_Rationale
Q	"1"	**R**

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	<blank>

When returning question **Q** as skillset **S**
Then the responses uses **R** for the question rationale

Example

Release:

TxQuestions.csv

```
"QuID","ControlType","Qu_Rationale"
"Tx123456","1","To find out if the individual is responsive"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"111 Call Handler",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Answers does not include Tx123456

Response:

```
{
  "linkId": "Tx123456.Rationale",
  "type": "display",
  "text": "To find out if the individual is responsive"
```

```
}
```

13.2.1.4.4 Original answer rationale

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType
Q	"1"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerRationale
Q	**N**	**A**

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	<blank>

When returning question **Q** as skillset **S**
Then the response uses **A** for the text for answer **N**'s rationale

Example

Release:

TxQuestions.csv

```
"QuID","ControlType"
"Tx123456","1"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerRationale"
"Tx123456","3","This means blood coming from one or both nostrils."
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"111 Call Handler",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456.3.Rationale",
  "type": "display",
  "text": "This means blood coming from one or both nostrils"
}
```

13.2.1.4.5 Alternate question text

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType
Q	"1"

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	**A**

And **alternateText.csv**:

DisplayType	ItemType	ItemId	ItemText
A	"TxText"	**Q**	**T**

When returning question **Q** as skillset **S**
Then the response uses **T** for the question text

Example

Release:

TxQuestions.csv

```
"QuID","ControlType"
"Tx123456","1"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"999 Clinician","Clinical"
```

alternateText.csv

```
"DisplayType","ItemType","ItemId","ItemText"
"Clinician","TxText","Tx123456","Have you travelled to Africa in the past 4
weeks?"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"999 Clinician"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456",
  "text": "Have you travelled to Africa in the past 4 weeks?",
}
```

13.2.1.4.6 Alternate answer text

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType
Q	"1"

And **TxAnswers.csv**:

QuID	AnswerNo
Q	**N**

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	**A**

And **alternateText.csv**:

DisplayType	ItemType	ItemId	ItemNumber	ItemText
A	"TxAnsText"	**Q**	**N**	**T**

When returning question **Q** as skillset **S**
Then the response uses **T** for the text of answer **N**

Example

Release:

TxQuestions.csv

```
"QuID","ControlType"
"Tx123456","1"
```

TxAnswers.csv

```
"QuID","AnswerNo"
"Tx123456","2"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"111 Online","Online"
```

alternateText.csv

```
"DisplayType","ItemType","ItemId","ItemNumber","ItemText"
"Online","TxAnsText","Tx123456","2","Blurred vision"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Online"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456.2.Answer",
  "type": "choice",
  "answerOption": [
    {
      "valueCoding": {
        "code": "2",
```

```
        "display": "Blurred vision"
      }
    }
  ]
}
```

13.2.1.4.7 Alternate question rationale

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType
Q	"1"

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	**A**

And **alternateText.csv**:

DisplayType	ItemType	ItemId	ItemText
A	"TxRationale"	**Q**	**T**

When returning question **Q** as skillset **S**
Then the response uses **T** for the question rationale

Example

Release:

TxQuestions.csv

```
"QuID","ControlType"
"Tx123456","1"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"999 Call Handler","Call Handler"
```

alternateText.csv

```
"DisplayType","ItemType","ItemId","ItemText"
"Call Handler","TxRationale","Tx123456","This means a problem with your blood sugar"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"999 Call Handler"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456.Rationale",
  "type": "display",
  "text": "This means a problem with your blood sugar"
}
```

13.2.1.4.8 Alternate answer rationale

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType
Q	"1"

And **TxAnswers.csv**:

QuID	AnswerNo
Q	**N**

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	**A**

And **alternateText.csv**:

DisplayType	ItemType	ItemId	ItemNumber	ItemText
A	"TxAnsRationale"	**Q**	**N**	**T**

When returning question **Q** as skillset **S**
Then the response uses **T** for the text of answer **N**'s rationale

Example

Release:

TxQuestions.csv

```
"QuID","ControlType"
"Tx123456","1"
```

TxAnswers.csv

```
"QuID","AnswerNo"
"Tx123456","6"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"ED Streaming and Redirection","EDS"
```

alternateText.csv

```
"DisplayType","ItemType","ItemId","ItemNumber","ItemText"
"EDS","TxAnsRationale","Tx123456","6","Ongoing pain or discomfort."
```

Request:

```
{"type":{"text":"skillset"},"valueString":"ED Streaming and Redirection"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456.6.Rationale",
  "type": "display",
  "text": "Ongoing pain or discomfort."
}
```

Returns "Ongoing pain or discomfort" as the rationale to answer number 6

13.2.1.4.9 Question text substitution

INCOMPLETE

Given an ingested release with **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**T**	"1"

And **T** includes any substitution markers

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	<blank>

When returning question **Q** as skillset **S**
And the patient is of gender **G** and party **P**
Then the response uses **T** for the question text
And the substitution markers are replaced with content appropriate for gender **G** and party **P**

Example

Release:

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx123456","xxx got palpitations?","1"
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"111 Call Handler",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"},
{"type":{"text":"gender"},"valueString":"Female"},
{"type":{"text":"party"},"valueString":"3"}
```

(Answers does not include Tx123456)

Response:

```
{
  "linkId": "Tx123456",
  "text": "Has she got palpitations?",
}
```

13.2.1.4.10 <Temp> Include node coordinate

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID
P	N	Q

And **TxQuestions.csv**:

QuID	ControlType
Q	"1"

When returning question **Q**
Then the response includes the coordinate **P.N**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID"
"PW111","250","TX288288"
```

TxQuestions.csv

```
"QuID","ControlType"
"TX288288","1"
```

Request:

(Answers does not include TX288288)

Response:

```
"identifier": [
  {
    "use": "old",
    "type": {
      "text": "PathwayID.OrderNo"
    },
    "value": "PW111.250"
  }
```

```
],
```

13.2.1.4.11 <Temp> Include answer positive flag

Given an ingested release with **TxQuestions.csv**:

QuID	ControlType
Q	"1"

And **TxAnswers.csv**:

QuID	AnswerNo	Positive
Q	**N**	**P**

When returning question **Q** as skillset **S**
Then the response includes the positive value **P** for answer **N**

Example

Release:

TxQuestions.csv

```
"QuID","ControlType"
"Tx111222","1"
```

TxAnswers.csv

```
"QuID","AnswerNo","Positive"
"Tx111222","3","1"
```

Request:

(Answers does not include Tx111222)

Response:


```
{
  "linkId": "Tx111222.3.Positive",
  "type": "integer",
  "code": [
    {
      "code": "1"
    }
  ]
}
```

13.2.1.4.12 Missing scenarios

- Prioritised alternate display text (e.g. "EDS|Online")

13.2.1.5 Variables and Rules

13.2.1.5.1 Pass over variable nodes

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	"1"	**<transfer>**	

And **TxQuestions.csv**:

QuID	ControlType
Q	"3"

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	n

When processing node **P.N** as skillset **S**
Then triage progresses according to <**transfer**>

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T2"
"PMA3","150","Tx123123","1","-1","PW5.0"
```

TxQuestions.csv

```
"QuID","ControlType"
"Tx123123","3"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"OOH Clinician","T2"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"OOH Clinician"}
```

Engine:

Navigates to node PW5.0

13.2.1.5.2 Match an equality branch rule

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	**<transfer>**	

And **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**V**	"4"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerText
Q	**A**	"=="**T**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	n

When processing node **P.N** as skillset **S**
And the value of **V** is equal to **T**
Then triage progresses according to **<transfer>**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T3"
"PFC3","1000","Tx5253","2","-1","PW4.500"
```

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx5253","AbdominalPain","4"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerText"
"Tx5253","2","==""present"""
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T3"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Engine:

(Has the variable "AbdominalPain" with value "present")

(Navigates to node PFC3.1000)

Navigates to node PW4.500

13.2.1.5.3 Match a less-than branch rule

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	**<transfer>**	

And **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**V**	"4"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerText
Q	**A**	"<"**T**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	n

When processing node **P.N** as skillset **S**
And the value of **V** is less than **T**
Then triage progresses according to **<transfer>**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T3"
"PFC3","1000","Tx5253","2","-1","PW4.500"
```

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx5253","PATIENT_AGE","4"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerText"
"Tx5253","2","<55"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T3"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Engine:

(Has the variable "PATIENT_AGE" with value "30")

(Navigates to node PFC3.1000)

Navigates to node PW4.500

13.2.1.5.4 Match a greater-than branch rule

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	**<transfer>**	

And **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**V**	"4"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerText
Q	**A**	">"**T**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	n

When processing node **P.N** as skillset **S**
And the value of **V** is greater than **T**
Then triage progresses according to **<transfer>**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T3"
"PFC3","1000","Tx5253","2","-1","PW4.500"
```

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx5253","PATIENT_AGEinDAYS","4"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerText"
"Tx5253","2",">183"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T3"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Engine:

(Has the variable "PATIENT_AGEinDAYS" with value "365")

(Navigates to node PFC3.1000)

Navigates to node PW4.500

13.2.1.5.5 Match a less-than-or-equal-to branch rule

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	**<transfer>**	

And **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**V**	"4"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerText
Q	**A**	"<="**T**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	n

When processing node **P.N** as skillset **S**
And the value of **V** is less than or equal to **T**
Then triage progresses according to **<transfer>**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T3"
"PFC3","1000","Tx5253","2","-1","PW4.500"
```

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx5253","PATIENT_AGE","4"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerText"
"Tx5253","2","<=10"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T3"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Engine:

(Has the variable "PATIENT_AGE" with value "8")

(Navigates to node PFC3.1000)

Navigates to node PW4.500

13.2.1.5.6 Match a greater-than-or-equal-to branch rule

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	**<transfer>**	

And **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**V**	"4"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerText
Q	**A**	**">="T**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	n

When processing node **P.N** as skillset **S**
And the value of **V** is greater than or equal to **T**
Then triage progresses according to **<transfer>**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T3"
"PFC3","1000","Tx5253","2","-1","PW4.500"
```

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx5253","PATIENT_AGE","4"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerText"
"Tx5253","2",">=11"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T3"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Engine:

(Has the variable "PATIENT_AGE" with value "16")

(Navigates to node PFC3.1000)

Navigates to node PW4.500

13.2.1.5.7 Fallback to default transfer when no branch rule applies

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	**<transfer>**	

And **TxQuestions.csv**:

QuID	QuestionText	ControlType
Q	**V**	"4"

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerText
Q	**A**	"*"

QuID	AnswerNo	AnswerText
Q	B<>A	T

And **Skillsets.csv**:

SkillSetDescription	Tbranch
S	n

When processing node **P.N** as skillset **S**
And the value of **V** does not match any rule **T**
Then triage progresses according to **<transfer>**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T3"
"PFC3","1000","Tx5253","1","-1","PW4.100"
"PFC3","1000","Tx5253","2","-1","PW4.200"
"PFC3","1000","Tx5253","3","-1","PW4.300"
```

TxQuestions.csv

```
"QuID","QuestionText","ControlType"
"Tx5253","PATIENT_AGE","4"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerText"
"Tx5253","1","<25"
"Tx5253","2","<=49"
"Tx5253","3","*"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T3"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"}
```

Engine:

(Has the variable "PATIENT_AGE" with value "62")

(Navigates to node PFC3.1000)

Navigates to node PW4.300

13.2.1.5.8 Missing scenarios

None identified

13.2.1.6 Reports

13.2.1.6.1 Compiles an assessment

TO DO

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo
P	**N**	**Q**	**A**

And **TxQuestions.csv**:

QuID	ControlType
Q	1

And **TxAnswers.csv**:

QuID	AnswerNo	AnswerReportText
Q	**A**	**R**

When processing node **P**.**N**
And finishes and returns a response
Then an assessment is returned that includes **R**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo"
"PMA1","25","Tx500011","5"
```

TxQuestions.csv

```
"QuID","ControlType"
"Tx500011","1"
```

TxAnswers.csv

```
"QuID","AnswerNo","AnswerReportText"
```

```
"Tx500011","5","An object was visible in the mouth."
```

Request:

```
{"type":{"coding":[{"code":"Tx226548"}]},"valueCoding":"5"}
```

Engine:

(Navigates to node PMA1.25)

Response:

```
{
  "type": {
    "text": "Pathways Assessment"
  },
  "valueString": "An object was visible in the mouth."
}
```

13.2.1.6.2 Compiles a summary

TO DO

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo
P	**N**	**Q**	**A**

And **TxQuestions.csv**:

QuID	ControlType
Q	1

And **TxAnswers.csv**:

QuID	AnswerNo	DispoDisplayText
Q	**A**	**D**

When processing node **P**.**N**
And finishes and returns a response
Then a summary is returned that includes **D**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo"
"PMI","3000","Tx330011","2"
```

TxQuestions.csv

```
"QuID","ControlType"
"Tx330011","1"
```

TxAnswers.csv

```
"QuID","AnswerNo","DispoDisplayText"
"Tx330011","2","Thermal burn"
```

Request:

```
{"type":{"coding":[{"code":"Tx330011"}]},"valueCoding":"2"}
```

Engine:

(Navigates to node PMI.3000)

Response:

```
{
  "type": {
```

```
    "text": "Consultation Summary"
  },
  "valueString": "Thermal burn"
}
```

13.2.1.6.3 Compiles a care advice report with original text

TO DO INCOMPLETE

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID
P	N	Q

And **Q** starts with "Cx"
And **CareAdviceItems.csv**:

CxID	CxText
Q	T

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	<blank>

When the triage engine finishes and returns a response
Then a report of the care advice given is returned that includes **T**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID"
```

```
"P100","250","Cx110000"
```

CareAdviceItems.csv

```
"CxID","CxText"
"Cx110000","Check the opening times of your chosen pharmacy."
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"Dental",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"Dental"}
```

Engine:

(Processes node P100.250)

Response:

```
{
  "type": {
    "text": "Advice Given"
  },
  "valueString": "Check the opening times of your chosen pharmacy."
}
```

13.2.1.6.4 Compiles a care advice report with alternative text

TO DO INCOMPLETE

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID
P	**N**	**Q**

And **Q** starts with "Cx"
And **CareAdviceItems.csv**:

CxID	CxNo	CxText
Q	**C**	**T_0**

And **alternateText.csv**:

DisplayType	ItemType	ItemId	ItemNumber	ItemText
A	"CxItemText"	**Q**	**C**	**T_1**

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	**A**

When processing node **P.N** as skillset **S**
And finishes and returns a response
Then a report of the care advice given is returned that includes **T_1**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID"
"P100","250","Cx110000"
```

CareAdviceItems.csv

```
"CxID","CxNo","CxText"
```

```
"Cx110000","2","Check the opening times of your chosen pharmacy."
```

alternateText.csv

```
"DisplayType","ItemType","ItemId","ItemNumber","ItemText"
"Online","CxItemText","Cx110000","2","Check your pharmacy for their opening
times."
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
"Nurse","Online"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"Nurse"}
```

Engine:

(Navigates to node P100.250)

Response:

```
{
  "type": {
    "text": "Advice Given"
  },
  "valueString": "Check your pharmacy for their opening times."
}
```

13.2.1.7 Care Advice

13.2.1.7.1 Returns the original most recent inline care advice

TO DO INCOMPLETE

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID
P	N	Q

And **Q** starts with "Cx"
And **CareAdviceItems.csv**:

CxID	CxText
Q	T

And **Skillsets.csv**:

SkillsetDescription	AlternateDisplayText
S	<blank>

When processing node **P.N** as skillset **S**
And no further question nodes are traversed
Then the response includes care advice **T**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID"
"PMC3","800","Cx210011"
```

CareAdviceItems.csv

```
"CxID","CxText"
"Cx210011","Encourage the individual to cough."
```

Skillsets.csv

```
"SkillsetDescription","AlternateDisplayText"
```

```
"Dental",""
```

Request:

```
{"type":{"text":"skillset"},"valueString":"Dental"}
```

Engine:

(Navigates to PMC3.800)

Response:

```
{
  "resource": {
    "resourceType": "Communication",
    "status": "completed",
    "payload": [
      {"contentString": "Encourage the individual to cough."}
    ]
  }
}
```

13.2.1.7.2 Does not return old inline care advice

TO DO

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo
P_1	N_1	Q_1	A_1
P_2	N_2	Q_2	A_2

And Q_1 starts with "Cx"
And Q_2 starts with "Tx"
And **CareAdviceItems.csv**:

CxID	CxText
Q_1	T

And **TxQuestions.csv**:

QuID	ControlType
Q_2	1

And having previously processed node $P_1.N_1$
When applying a supplied answer A_2 to node $P_2.N_2$
Then the response does not include care advice **T**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo"
"P1","0","Cx100001","1"
"P2","100","Tx277233","4"
```

CareAdviceItems.csv

```
"CxID","CxText"
"Cx100001","Tell them to get the defibrillator."
```

TxQuestions.csv

```
"QuID","ControlType"
"Tx277233","1"
```

Request:

```
{"type":{"coding":[{"code":"Tx277233"}]},"valueCoding":"4"}
```

Engine:

(Processes node P1.0)

(Processes node P2.100)

Response:

Does not include care advice "Tell them to get the defibrillator."

13.2.1.8 Outcomes

13.2.1.8.1 Symptom Groups

TO DO

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	-1	"Dx#"

And **pathways.csv**:

PW_ID	Symptom_Group
P	**G**

And **SymptomGroups.csv**:

SgId	SgDesc
G	**D**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	**n**

When processing node **P.N** as skillset **S**
Then the response includes symptom group "SG"+**G** and description **D**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T1"
"PFI3","125","Tx810034","3","-1","Dx110034"
```

pathways.csv

```
"PW_ID","Symptom_Group"
"PFI3","2398"
```

SymptomGroups.csv

```
"SgId","SgDesc"
"2398","Fever"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"OOH Call Handler","T1"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"OOH Call Handler"},
{"type":{"coding":[{"code":"Tx810034"}]},"valueCoding":"3"}
```

Response:

```
{
  "coding": [
    {
```

```
      "code": "SG2398",
      "display": "symptomGroup"
    }
  ],
  "text": "Fever"
}
```

13.2.1.8.2 Symptom Discriminators

TO DO

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P_1	**N_1**	**Q_1**	**A**	<anything>	
P_2	**N_2**	**Q_2**	<anything>	-1	"Dx#"

And **Q_1** starts with "Tx"
And **TxAnswers.csv**:

QuID	AnswerNo	SymptomDiscriminator
Q_1	**A**	**M**

And **SymptomDiscriminators.csv**:

SdId	SdDesc
M	**D**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	**n**

And having previously processed answer **A** from coordinate $\mathbf{P_1.N_1}$ as skillset **S**
(And not having more recently processed any other answer with an attached symptom discriminator)
When processing node $\mathbf{P_2.N_2}$ as skillset **S**
Then the response includes symptom discriminator "SD"+**M** and description **D**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T4"
"PMA1","2500","Tx540034","3","3000",""
"PW188","100","Tx30789","2","-1","Dx190"
```

TxAnswers.csv

```
"QuID","AnswerNo","SymptomDiscriminator"
"Tx540034","3","4099"
```

SymptomDiscriminators.csv

```
"SdId","SdDesc"
"4099","PC post-traumatic stress"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"111 Call Handler","T4"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"111 Call Handler"},
{"type":{"coding":[{"code":"Tx540034"}]},"valueCoding":"3"}
```

Engine:

(Processes node PMA1.2500 with answer 3 as skillset 111 Call Handler)

(Processes node PW188.100 with answer 2 as skillset 111 Call Handler

Response:


```
{
  "coding": [
    {
      "code": "SG4099",
      "display": "symptomDiscriminator"
    }
  ],
  "text": "PC post-traumatic stress"
}
```

13.2.1.8.3 Dispositions

TO DO

Given an ingested release with **triagelogic.csv**:

PW_ID	OrderNo	Qu_ID	AnswerNo	JumpID	T*n*
P	**N**	**Q**	**A**	-1	**D**

And **D** starts with "Dx"

And **dispositions.csv**:

DispoCode	DispoText
D	**T**

And **Skillsets.csv**:

SkillsetDescription	Tbranch
S	**n**

When processing answer **A** from coordinate **P.N** as skillset **S**
Then the response includes disposition **D** with text **T**

Example

Release:

triagelogic.csv

```
"PW_ID","OrderNo","Qu_ID","AnswerNo","JumpID","T1"
"PW221","750","Tx303311","1","-1","Dx326"
```

dispositions.csv

```
"DispoCode","DispoText"
"Dx326","Contact Pharmacist next working day"
```

Skillsets.csv

```
"SkillsetDescription","Tbranch"
"HCP Callback","T1"
```

Request:

```
{"type":{"text":"skillset"},"valueString":"HCP Callback"},
{"type":{"coding":[{"code":"Tx303311"}]},"valueCoding":"1"}
```

Engine:

(Processes node PW221.750 with answer 1 as skillset HCP Callback)

Response:

```
{
  "coding": [
    {
      "code": "Dx326",
      "display": "disposition"
    }
  ],
  "text": "Contact Pharmacist next working day"
}
```

13.2.1.8.4 Missing scenarios

- Returns alternative most recent care advice (`DELIVERED`)

13.2.1.9 <TBD> Body map

13.2.1.9.1 Tailored to age, gender and trauma

TO DO

Given an ingested release with **BodyMap.csv**:

BMGroup	BMItem	Ordering
G	I	O

13.2.1.9.2 Missing scenarios

- everything

13.2.1.10 Cross-cutting concerns

13.2.1.10.1 <TBD> Case-insensitivity

TO DO

13.3 Conformance testing

Scope: FHIR messages, in and out

Could be manual after each update to the inbound adapter using https://validator.fhir.org/, or possibly automated somehow

13.4 Testing Strategy Gap Analysis

13.4.1 1. Purpose

This document provides an assurance-focused review of the documented testing strategy for the Patient Triage API. Its purpose is to assess whether the current approach provides sufficient confidence in the quality, reliability, and maintainability of the service, and to identify areas where the strategy may require strengthening.

The analysis is structured to identify:

- **Gaps**: testing activities not evidenced in the current documentation but likely to be required
- **Excesses**: areas where testing effort may be broader or heavier than necessary
- **Errors**: assumptions or areas of focus that may not be sufficiently supported
- **Improvements**: areas where the current approach is sound but could be enhanced

13.4.2 2. Evidence Reviewed

13.4.2.1 2.1 Unit testing

Reference: Unit testing (see page 108)

- Three unit test scopes are defined:
  - inbound adapters
  - core
  - outbound adapters
- Tooling: `pytest`
- All unit tests are described as **sociable**
- Unit tests are created using **test-driven development**
- Coverage checks fail if line coverage falls below **80%**

13.4.2.2 2.2 Acceptance testing

Reference: Acceptance testing (see page 109)

- Scope: HTTP message to API Gateway, through triage engine Lambda to DynamoDB
- Tooling:
  - Postman
  - Newman
  - Localstack
- Acceptance tests are created using **acceptance test driven development**
- The service is described as **stateless and idempotent**, with relatively simple set-up

13.4.2.3 2.3 Performance testing

Reference: Patient Triage API - Performance Test Results [30/10/2025 - 28/01/2026] (see page 214)

- Primary focus is on **latency rather than transactions per second**
- It is assumed that Lambda addresses TPS concerns
- API Gateway and DynamoDB are recognised as potential bottlenecks, though considered unlikely
- Tooling: AWS Distributed Load Testing on AWS

13.4.2.4 2.4 Conformance testing

Reference: Conformance testing (see page 168)

- Scope: FHIR messages, inbound and outbound
- Current view is that this could be completed manually after each inbound adapter update using the FHIR validator
- Automation is noted as a possibility but is not yet defined

13.4.3 3. Headline Assessment

The documented strategy demonstrates a clear intent to apply testing across multiple levels of the service, with the strongest definition and maturity evident in:

- unit testing
- acceptance testing

However, the current documentation also indicates that some parts of the strategy remain less mature or less well defined, particularly:

- conformance testing
- intermediate integration testing between unit and full end-to-end tests
- non-functional and risk-based testing types not explicitly described

13.4.4 4. Detailed Analysis by Test Type

13.4.4.1 4.1 Unit Testing

13.4.4.1.1 Current documented approach

- Unit testing is structured into three scopes:
  - Core

    Tests of business logic exposed by provided ports, with required ports replaced by test doubles. These tests are written in terms of business behaviour and domain objects, with the core isolated from infrastructure concerns.

- Inbound adapters

  Tests of the translation of inbound Lambda events into core use cases, together with translation of the resulting response back across the inbound channel.

- Outbound adapters

  Tests of the translation of core requirements into infrastructure interactions, and translation of infrastructure responses back into domain objects.

- All unit tests are described as sociable. Unit tests are written using test-driven development, and line coverage is enforced with a minimum threshold of 80%.

13.4.4.1.2 Assurance view

This is the strongest and most clearly articulated part of the testing strategy. It demonstrates good alignment with the service architecture and provides confidence that the documented approach is intended to verify:

- business logic in the core domain
- translation behaviour at service boundaries
- the interaction between layers through defined ports and adapters

13.4.4.1.3 Gaps

The documentation does not explicitly state whether unit testing includes systematic coverage of:

- error handling
- invalid inputs
- negative paths
- edge conditions or boundary cases

13.4.4.1.4 Excesses

- The use of a mandatory line coverage threshold may create an incentive to optimise for coverage percentage rather than test value. This is not necessarily a defect, but it may encourage coverage of lower-value code paths where assurance gain is limited.
- There is also a possibility that adapter-level tests could become overly detailed if they assert low-risk translation behaviour that does not materially affect service confidence.

13.4.4.1.5 Errors

- No direct defect in the documented unit testing approach is evident. However, exclusive reliance on sociable tests may reduce diagnostic precision when failures occur, particularly in the core domain where isolated tests can sometimes provide clearer fault localisation.

13.4.4.1.6 Improvements

- The strategy would be strengthened by making explicit whether unit testing is expected to cover:
  - invalid or malformed inputs
  - failure scenarios
  - business rule boundaries
  - exceptional flows
- It may also be beneficial to consider a limited use of more isolated tests for particularly complex business rules where diagnostic clarity is important.

13.4.4.2 4.2 Narrow Integration Testing

13.4.4.2.1 Current documented approach

- No distinct narrow integration test layer is described in the material provided.
- There is no documented test scope that verifies a single real technical boundary in isolation, for example:
  - Lambda to DynamoDB integration
  - API Gateway to Lambda mapping
  - infrastructure-specific serialization or deserialization behaviour

13.4.4.2.2 Assurance view

Based on the current documentation, there is a significant step between:

- unit tests using test doubles
- broad acceptance tests spanning the full service path

This suggests that integration confidence is currently expected to come either from unit tests or from full end-to-end testing, with no explicitly documented middle layer.

13.4.4.2.3 Gaps

- No evidence of a focused integration test layer
- No documented validation of individual infrastructure boundaries
- No intermediate test scope between the isolated domain and the full HTTP-to-database path

13.4.4.2.4 Excesses

None identified.

13.4.4.2.5 Errors

- The absence of a middle test layer increases the likelihood that integration defects will surface later, when diagnosis and remediation are more costly.

13.4.4.2.6 Improvements

- A narrow integration layer would strengthen assurance by validating key technical boundaries directly, without requiring full end-to-end execution. This would help distribute confidence more evenly across the test strategy.

13.4.4.3 4.3 Acceptance Testing

13.4.4.3.1 Current documented approach

- Acceptance testing covers the path from HTTP request to API Gateway, through the triage engine Lambda, to DynamoDB.
- Tests are written as Postman collections and executed using Newman. Localstack is used to simulate target infrastructure locally.
- Acceptance tests are created using acceptance test driven development.
- The documentation notes that, because the service is stateless and idempotent, set-up is relatively simple.

13.4.4.3.2 Assurance view

This is a clearly defined end-to-end test layer that provides confidence that the principal components of the service can operate together across the intended request path.

13.4.4.3.3 Gaps

The documentation does not state:

- which scenarios are prioritised
- whether error paths are explicitly covered
- whether acceptance tests are focused on a small number of critical behaviours or a wider regression suite
- how negative paths are treated at this level

13.4.4.3.4 Excesses

Because the scope is broad, there is a risk that acceptance tests are validating behaviours that are already covered more efficiently in unit tests. If this is the case, the acceptance layer may be doing more work than is necessary for its intended assurance purpose.

13.4.4.3.5 Errors

- The statement that set-up is relatively simple because the service is stateless and idempotent may be operationally reasonable, but it does not in itself reduce the maintenance burden or representational limitations of end-to-end tests.
- The documentation does not address whether Localstack-based execution is considered fully representative of the real runtime environment.

13.4.4.3.6 Improvements

The strategy would benefit from clarifying the role of acceptance testing, for example whether it is intended primarily for:

- critical path validation
- release confidence
- smoke testing
- broader regression assurance

This would make it easier to ensure that acceptance testing remains proportionate and targeted.

13.4.4.4 4.4 Performance Testing

13.4.4.4.1 Current documented approach

- The current performance approach is focused on observing latency rather than transactions per second. The rationale given is that Lambda addresses TPS concerns, while API Gateway and DynamoDB remain possible, though unlikely, bottlenecks.
- The stated objective is to observe latency degradation over time so that any significant worsening can be associated with changes introduced into the solution.
- The documented tool is AWS Distributed Load Testing on AWS.

13.4.4.4.2 Assurance view

- This represents a pragmatic starting point for performance assessment, particularly where the main intent is to monitor trends rather than to demonstrate full capacity assurance.
- The documentation clearly shows awareness that infrastructure components outside Lambda may still influence performance.

13.4.4.4.3 Gaps

The documentation does not define:

- specific latency targets
- acceptable degradation thresholds
- expected load profiles
- concurrency assumptions
- performance scenarios such as spike, stress, or sustained load

There is also no explicit reference to cold starts or service behaviour under changing operational conditions.

13.4.4.4.4 Excesses

None idenfified.

13.4.4.4.5 Errors

- The statement that Lambda addresses TPS concerns is currently presented as an assumption rather than as test evidence. Where API Gateway and DynamoDB remain identified as potential bottlenecks, this assumption may need to be supported by defined test activity or operational evidence.

13.4.4.4.6 Improvements

- The strategy would be strengthened by distinguishing between:
  - baseline latency monitoring
  - regression detection
  - capacity-related testing
  - infrastructure bottleneck validation
- It would also be beneficial to define measurable expectations so that performance assessment can be evaluated consistently over time.

13.4.4.5 4.5 Conformance Testing

13.4.4.5.1 Current documented approach

- Conformance testing is identified as applying to FHIR messages both inbound and outbound.
- The current position is that validation could be performed manually after each update to the inbound adapter using the FHIR validator. Automation is noted as a possibility, but no automated approach is currently defined.

13.4.4.5.2 Assurance view

This area is recognised as important, but the documentation indicates that the approach is not yet established to the same degree as the other test types.

13.4.4.5.3 Gaps

- No formal conformance testing process is defined
- No automated conformance checks are described
- No criteria are stated for:
  - when checks must be run
  - what constitutes a pass or fail outcome
  - whether inbound and outbound conformance are treated consistently

13.4.4.5.4 Excesses

None identified.

Errors

- If conformance testing remains manual only, there is a risk that standards assurance becomes inconsistent or dependent on individual judgement.
- There is a potential limitation in linking validation primarily to inbound adapter updates, as outbound conformance may also be affected by changes elsewhere in the solution.

13.4.4.5.5 Improvements

- Include conformance testing into the core testing strategy rather than a possible or optional activity.

13.4.4.6 4.6 Exploratory Testing

13.4.4.6.1 Current documented approach

Exploratory testing approach is not defined.

13.4.4.6.2 Assurance view

Exploratory testing is not a part of the current strategy.

13.4.4.6.3 Gaps

- No documented exploratory testing activity
- No defined time-boxed investigation of unusual, malformed, or unexpected requests
- No evidence of structured learning activity outside scripted tests

13.4.4.6.4 Excesses

None identified.

13.4.4.6.5 Errors

- The absence of exploratory testing means that unexpected behaviours may only be discovered incidentally rather than through intentional investigation.

13.4.4.6.6 Improvements

- If exploratory testing is already taking place informally, this would benefit from being reflected in the documented strategy.
- If it is not taking place, consideration could be given to introducing a lightweight exploratory practice focused on risk-based API behaviour.

13.4.4.7 4.7 Manual Testing

13.4.4.7.1 Current documented approach

- Manual testing is referenced only in relation to possible FHIR conformance checks following updates to the inbound adapter.
- No broader manual testing approach is described.

13.4.4.7.2 Assurance view

At present, manual testing appears to be a limited or provisional element of the documented strategy rather than a defined testing layer.

13.4.4.7.3 Gaps

- No clear statement of when manual testing is expected to be used
- No indication of whether manual testing is limited to standards validation or also used for exploratory or operational checks

13.4.4.7.4 Excesses

None identified.

13.4.4.7.5 Errors

None identified.

13.4.4.7.6 Improvements

The strategy would benefit from clarifying whether manual testing is:

- a temporary control
- a fallback where automation is not yet available
- an intended part of exploratory or conformance activity

13.4.4.8 4.8 Other Testing Types Not Evidenced in the Documentation

The following testing types are not described in the material provided:

- security testing
- resilience or failure-path testing
- compatibility or backward compatibility testing
- provider or consumer contract testing
- broader data integrity testing beyond the stated scopes

13.4.4.8.1 Improvements

The next revision of the strategy should confirm whether these test types are:

- in scope and active
- not currently in use
- deferred intentionally
- covered elsewhere outside this document

13.4.5 5. Cross-Cutting Findings

13.4.5.1 5.1 Areas of relative strength

The strongest aspects of the documented approach are:

- clear unit test structure aligned to system architecture
- consistent use of test-driven development
- presence of an end-to-end acceptance testing layer
- explicit recognition of conformance and performance as relevant concerns

13.4.5.2 5.2 Main structural gap

The most significant structural gap in the documented strategy is the absence of a clearly defined middle layer between:

- unit tests with test doubles
- full end-to-end acceptance tests

13.4.5.3 5.3 Main maturity gap

Some areas are clearly operationalised and repeatable, while others remain provisional or under-defined. The clearest examples of this are:

- conformance testing
- exploratory testing
- the role of manual testing
- other testing types not currently evidenced

13.4.5.4 5.4 Main assumptions requiring evidence or clarification

Several points in the documentation are currently framed as assumptions rather than explicit test outcomes. These include:

- Lambda sufficiently addressing TPS concerns
- API Gateway and DynamoDB being unlikely bottlenecks

- statelessness and idempotency making acceptance set-up simple
- manual FHIR validation being sufficient following inbound adapter updates

These assumptions may be reasonable, but they would benefit from either supporting evidence or clearer treatment within the strategy.

13.4.6 6. Action points

13.4.6.1 Unit Testing

Important

- Explicitly define expectations for unit test coverage of negative paths, invalid inputs, and boundary conditions.
- Clarify that business-critical rules in the core must have clear behavioural assertions, not just structural coverage.

Nice to have

- Introduce limited isolated (non-sociable) unit tests for complex or high-risk business logic to improve diagnostic clarity.
- Supplement line coverage with simple behavioural or scenario-based coverage indicators.

13.4.6.2 Narrow Integration Testing

Important

- Introduce a narrow integration test layer to validate key technical boundaries (e.g. Lambda–DynamoDB, API Gateway–Lambda).
- Define the scope and responsibility of this layer to sit between unit tests and full acceptance tests.

Nice to have

- Prioritise narrow integration tests based on the most change-prone or failure-prone boundaries.

13.4.6.3 Acceptance Testing

Important

- Clearly define the purpose of acceptance testing (e.g. critical path validation, release confidence).
- Identify and document a small, explicit set of critical acceptance scenarios.

Nice to have

- Reduce duplication by moving lower-value behavioural permutations from acceptance tests into unit tests where appropriate.

- Document known limitations of Localstack-based execution versus real AWS environments.

13.4.6.4 Performance Testing

Important

- Define explicit latency expectations or acceptable degradation thresholds.
- Clearly document which performance characteristics are tested versus assumed.

Nice to have

- Expand performance scenarios to include spike or concurrency-focused tests where feasible.
- Include explicit consideration of cold-start behaviour if operationally relevant.

13.4.6.5 Conformance Testing (FHIR)

Important

- Formalise conformance testing as a mandatory part of the testing strategy rather than a discretionary activity.
- Define when conformance checks must be run and what constitutes pass/fail.

Nice to have

- Automate FHIR validation to ensure consistent and repeatable execution.
- Ensure conformance checks apply equally to inbound and outbound messages.

13.4.6.6 Exploratory Testing

Important

- Introduce a lightweight, documented exploratory testing practice focused on risk and unexpected API behaviour.

Nice to have

- Time-box exploratory sessions and feed outcomes into automated tests where appropriate.

13.4.6.7 Manual Testing

Important

- Clearly define the intended role of manual testing within the overall strategy.

Nice to have

- Restrict manual testing explicitly to exploratory or standards-based activities to avoid duplication with automation.

13.4.6.8 Other Test Types Not Currently Evidenced

Important

- Confirm whether security, resilience, compatibility, and contract testing are in scope and active, or explicitly out of scope.

Nice to have

- Reference these test types in the strategy even if they are owned or delivered elsewhere, to present a complete assurance picture.

13.4.7 7. Summary

This review found that the Patient Triage API has a **strong testing foundation**, particularly in how individual components are tested and how the service is validated end to end. These practices provide good confidence that the system behaves as intended when changes are made.

However, the review also identified that **confidence is uneven across the testing approach**. Some important areas are well established, while others are either lightly defined or still based on assumptions rather than clear evidence.

13.4.7.1 Key points:

- **What is working well**
  - Core functionality is thoroughly tested at a technical level.
  - There is automated testing that checks the service works end to end.
  - Performance and standards compliance (FHIR) are recognised as important and are at least partially addressed.
- **Where confidence is weaker**
  - There is limited testing focused specifically on how system components integrate with each other.
  - Standards compliance (FHIR) is recognised but not yet fully formalised or consistently automated.
  - Some important quality risks (such as unusual inputs, failures, or security-related concerns) are not clearly covered in the documented approach.
- **Why this matters**
  - Without clearer coverage in these areas, issues are more likely to be detected later, when they are harder and more expensive to diagnose and fix.
  - Assurance currently relies on a small number of strong test types, rather than being evenly spread across all key risk areas.
- **Overall position**
  - There is no indication of poor practice or missing testing altogether.

- The main opportunity is to **strengthen and balance** the existing strategy, turning assumptions and provisional practices into clear, repeatable controls.

13.5 Performance testing

13.5.1 Patient Triage API - Performance Test Results [05/05/2026]

13.5.1.1 Summary

This page captures the execution outcomes of the **Patient Triage API Performance Testing Strategy** and confirms whether the service meets defined non-functional requirements (NFRs) for performance, scalability, reliability, and availability.

- Test window: 05 May 2026 to 13 May 2026
- Environment: Dev to Ref
- Build/version under test: 0.2.0
- Test owner: Andrew Walker
- Report version: 1.0
- Status: Approved

13.5.1.2 Document Control

Item	Details
Author	Performance Engineering Team
Results page owner	Andrew Walker
Date created	05 May 2026
Last updated	15 May 2026
Linked strategy	Patient Triage Api - Performance Testing Strategy
Next review date	TBC

13.5.1.3 1. Executive Outcome

13.5.1.3.1 Overall Result

Overall verdict: PASS

13.5.1.3.2 Headline Findings

- P95 latency target (< 300 ms): MET
- P99 latency target (< 500 ms): MET
- Peak error rate target (< 0.1%): MET
- 5xx target under normal load (< 0.01%): MET
- Availability target (99.90%): MET
- Peak tested throughput: 110TPs
- Autoscaling behaviour: Stable

13.5.1.3.3 Go/No-Go Recommendation

- Recommendation: GO
- Rationale: Some of the P99 results were slightly higher than SLA of 500ms There has been a story created to invest this issue further. However the rest of the results were in line with what were expected and met all SLA targets.
  ☑ NPT-1360[69] - Improve P99 Performance BACKLOG

13.5.1.4 2. KPI Results Against Targets

KPI	Target	Status	Notes
P50 response time (Triage POST)	< 150 ms	PASS	N/A
P95 response time (Triage POST)	< 300 ms	PASS	N/A
P99 response time (Triage POST)	< 500 ms	PASS	N/A
Peak load error rate	< 0.1%	PASS	N/A
5xx rate at normal load	< 0.01%	PASS	N/A

69. https://nhsd-jira.digital.nhs.uk/browse/NPT-1360?src=confmacro

KPI	Target	Status	Notes
Service availability	99.90%	PASS	N/A
Max sustained throughput	120 TPS target	PASS	N/A

13.5.1.5 3. Test Coverage and Execution Evidence

13.5.1.5.1 Scope Confirmation

- In-scope scenarios executed: Yes
- Out-of-scope items excluded: Yes
- Deviations from strategy: <None / List below>

13.5.1.5.2 Evidence Links

- Test scripts repo/path: <link>
- JMeter/Gatling test plan versions: <link>
- JMeter latency logs:
  - Baseline: s3://performancetesting-dlttestrunnerstoragedltscenario-ltexeja5pom4/results/cf1U1xgypx/
  - Peak: s3://performancetesting-dlttestrunnerstoragedltscenario-ltexeja5pom4/results/xOwacu5k9X/
  - Stress: s3://performancetesting-dlttestrunnerstoragedltscenario-ltexeja5pom4/results/RDXR9wUUee/
  - Soak: s3://performancetesting-dlttestrunnerstoragedltscenario-ltexeja5pom4/results/H3TteEtewc/
  - Spike: s3://performancetesting-dlttestrunnerstoragedltscenario-ltexeja5pom4/results/UCCbKfeIo0/
  - Capacity: s3://performancetesting-dlttestrunnerstoragedltscenario-ltexeja5pom4/results/HlJjnUxwc9/

13.5.1.6 4. Scenario-by-Scenario Results

Scenario	Planned Profile	Actual Profile	Key Results	Status	Defects / Incidents
1. Baseline	2 TPS, 30 min	1.88 TPS, 30 min	P95: 222 ms, Error: 0.0295 %	PASS	N/A
2. Current Peak	6 TPS, 60 min	5.2 TPS, 60 min	P95: 190 ms, Error: 0.004 %	PASS	N/A
3. Stress (Future State)	Ramp 6 to 120 TPS, hold 60 min	Ramp 6 to 36 TPS, hold 60 min	P95: 195ms , Error: 0.02%, Throttle: 0	PASS	N/A
4. Endurance (Soak)	120 TPS, 4 hours	110 TPS, 2 hours	Drift in latency: +0.65%, Error trend: 41 errors in first third (381,009 requests), 29 in last third (381,009 requests)	PASS	N/A
5. Spike	2 to 50 to 2 TPS	~60-70 TPS throughput, 20 min	Scale-up time: N/A, SLA breaches: P99 724ms	CONDITIONAL	P99 slightly higher than SLA
6. 2x Traffic Buffer	13 TPS, 30 min	14.29 TPS, 30 min	P95: 177ms, Error: 0.00755%	PASS	N/A

13.5.1.6.1 Execution Detail Per Test (Steps + Result Capture)

13.5.1.6.1.1 Common Pre-Checks (Run Before Every Scenario)

Check	Owner	Status	Evidence
Test environment healthy and isolated	Andrew Walker	Done	N/A
Monitoring dashboards active (Grafana)	Andrew Walker	Done	N/A
Test data loaded and validated	Andrew Walker	Done	N/A
Script version and config signed off	Andrew Walker	Done	N/A

Check	Owner	Status	Evidence
Incident channel and rollback plan ready	Andrew Walker	Done	N/A

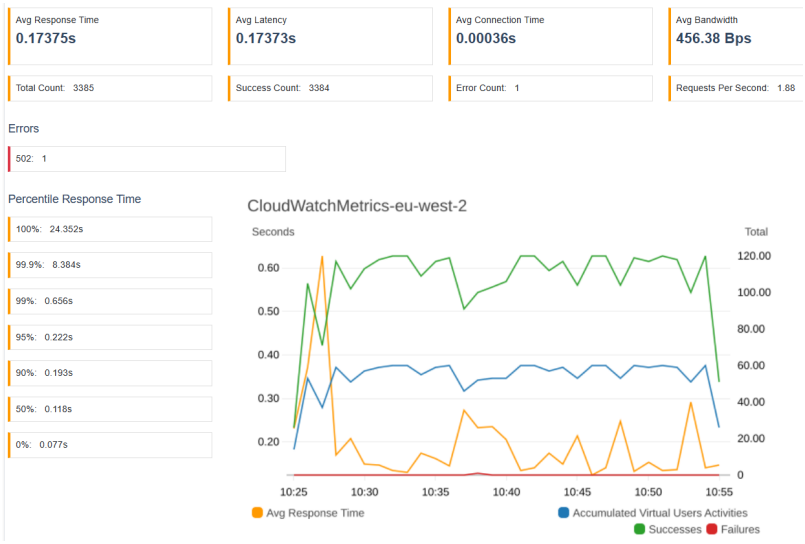
13.5.1.6.1.2 Scenario 1: Baseline Performance Test

Objective: Establish baseline under normal operating load.
Planned profile: 2 TPS sustained for 30 minutes.

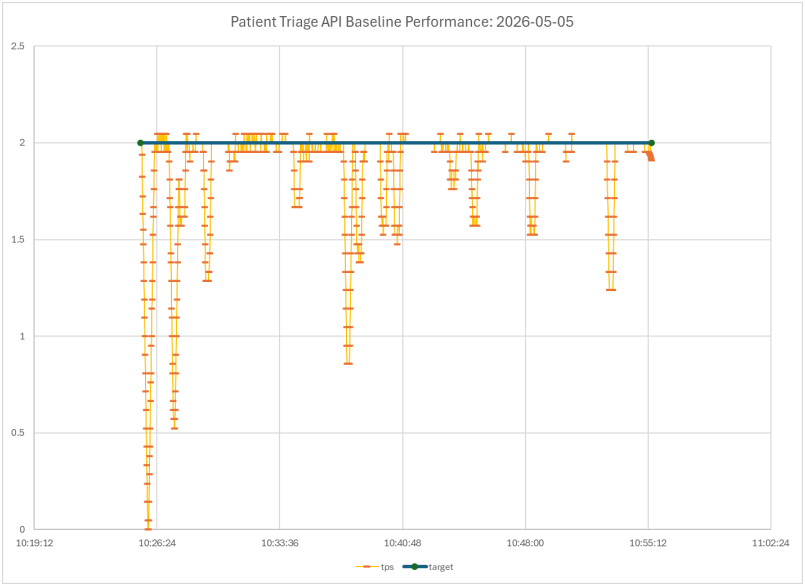
Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Confirm no background load and environment readiness	Stable baseline conditions confirmed	Complete	Done	See below
2	Start baseline script at 2 TPS	Steady 2 TPS reached without throttling	No throttling	Done	See below
3	Run for full 30 minutes	No interruptions or major variance	No interruptions	Done	See below
4	Capture latency and error metrics	P50/P95/P99 and errors available	118 / 222 / 656	Done	See below
5	Validate against baseline acceptance	P95 < 300 ms and error rate < 0.01%	222ms 0.03% error	PASS	See below

Result field	Value
Run ID	2026-05-05 (cf1U1xgypx)
Start / End time	2026-05-05 10:24:33 to 2026-05-05 10:55:46
P50 / P95 / P99 (ms)	118 / 222 / 656
Error rate (%)	0.03
Overall scenario result	PASS
Notes	N/A

JMeter results



Smoothed results vs target



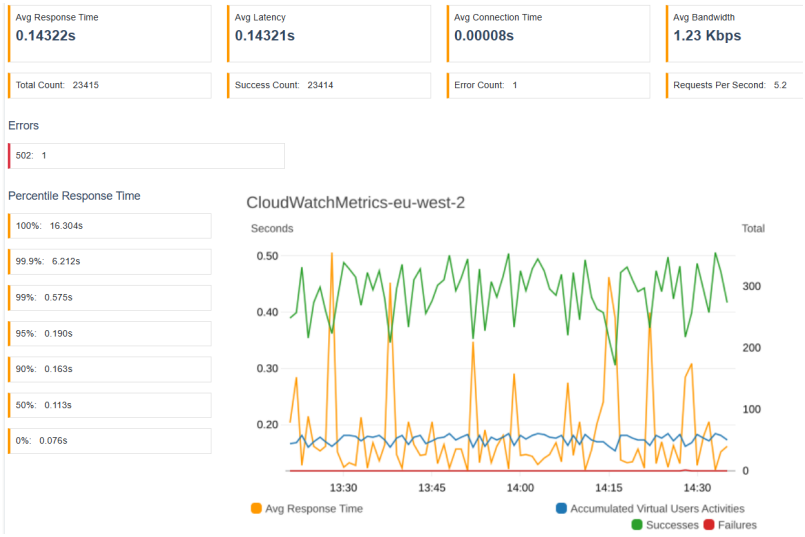
13.5.1.6.1.3 Scenario 2: Current Peak Demand Load Test

Objective: Validate performance at currently observed peak.
Planned profile: 6.56 TPS sustained for 60 minutes, 10-minute ramp.

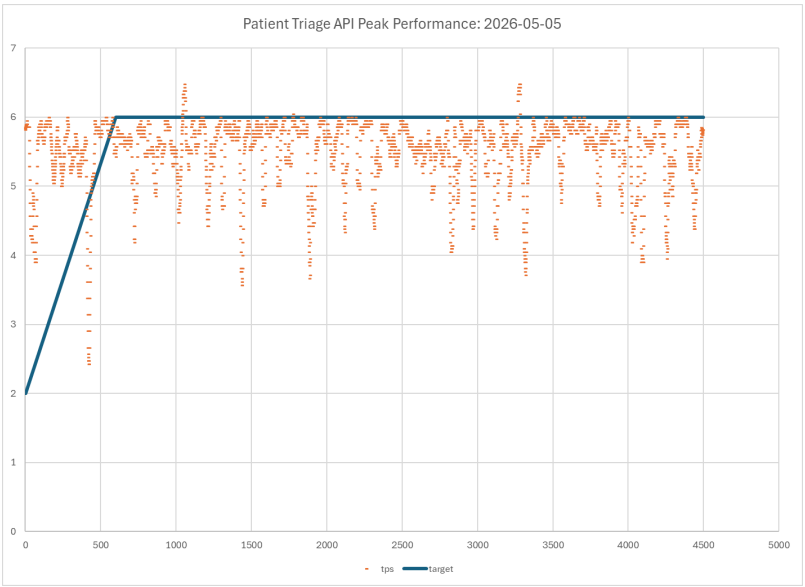
Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Ramp from baseline to 6 TPS over 10 minutes	Smooth ramp, no immediate SLA breach	No SLA breach	Done	See below
2	Hold 6 TPS for 60 minutes	Stable throughput and response profile	Stable	Done	See below
3	Monitor Lambda/API Gateway/ DynamoDB behaviour	No capacity alarms or sustained throttling	No Alarms	Done	See below
4	Validate SLA and error limits	P95 < 300 ms, error rate < 0.1%	190ms 0.004% error rate	PASS	See below

Result field	Value
Run ID	2026-05-05 peak (xOwacu5k9X)
Start / End time	2026-05-05 13:20:16 to 2026-05-05 14:36:27
P95 / P99 (ms)	190 / 575
Error rate (%)	0.004
Time to steady-state	10 minutes
Overall scenario result	PASS

JMeter results



Smoothed results vs target



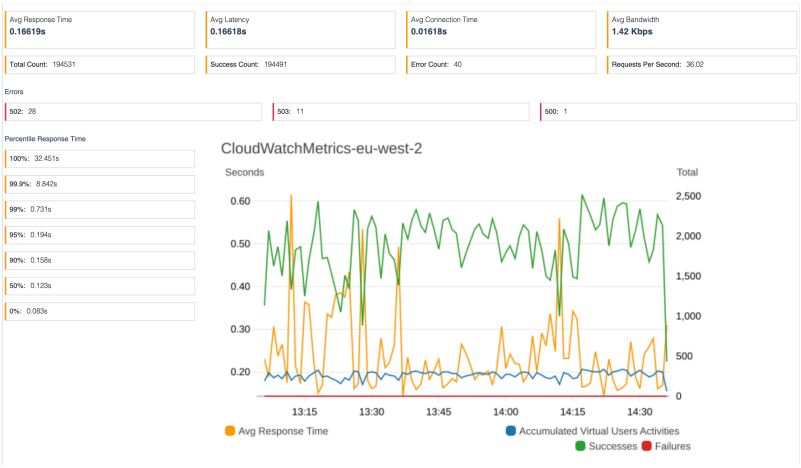
13.5.1.6.1.4 Scenario 3: Stress Test (Future State Capacity)

Objective: Validate future growth capacity at 120 TPS.
Planned profile: Ramp 6.56 to 120 TPS over 30 minutes, hold 120 TPS for 60 minutes.

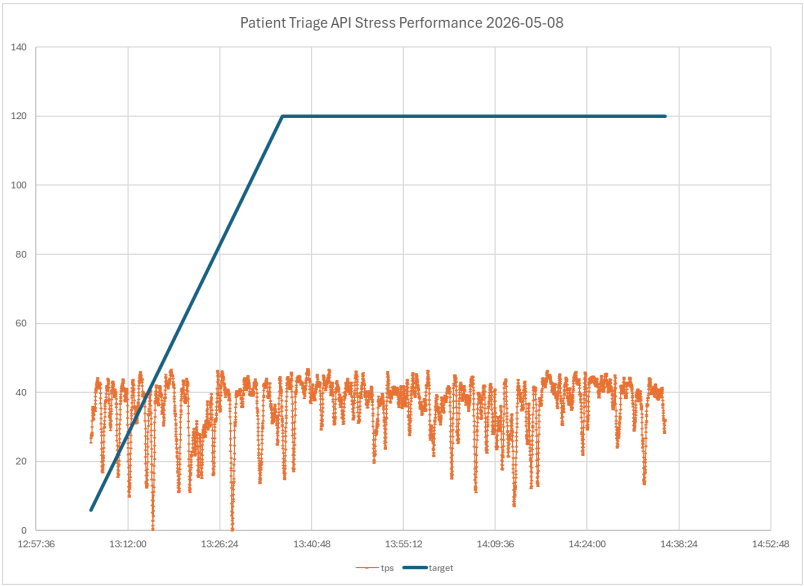
Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Warm-up at 6 TPS	Stable pre-stress baseline	Stable	Done	See Below
2	Ramp to 120 TPS over 30 minutes	Controlled scaling and stable error profile	Stable	Done	See Below
3	Hold 120 TPS for 60 minutes	Sustained load without service degradation	No degradation	Done	See Below
4	Track autoscaling, cold starts, throttles	Scaling events recorded and acceptable	N/A	N/A	See Below
5	Validate acceptance criteria	P95 < 300 ms, error < 0.1%, no harmful throttling	195ms 0.02% error rate	PASS	See Below

Result field	Value
Run ID	2026-05-08 Stress Test (RDXR9wUUee)
Peak TPS achieved	36.02
P95 / P99 at peak (ms)	195ms / 735ms
Error rate at peak (%)	0.019%
Autoscaling response time	N/A
Overall scenario result	PASS

JMeter results



Smoothed results vs target



13.5.1.6.1.5 Scenario 4: Endurance (Soak) Test

Objective: Validate long-duration stability and degradation behaviour.
Planned profile: 120 TPS for 4 hours.

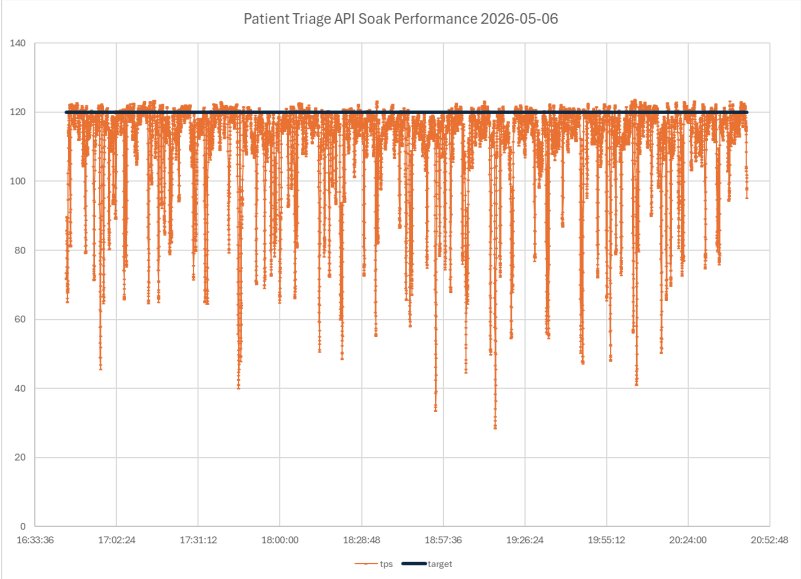
Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Start sustained 120 TPS load	Stable entry into steady state	Stable	Done	See Below
2	Execute continuous run for 4 hours	No crashes, no progressive degradation	No degradation	Done	See Below
3	Capture periodic snapshots (hourly)	Trend data available for latency/ errors/resources	See below	Done	See Below
4	Compare hour 1 vs hour 4 metrics	Minimal drift and acceptable consistency	+0.65%	Done	See Below
5	Validate soak success criteria	P95 stable and error rate consistent	154ms 0.0088 error rate	PASS	See Below

Result field	Value
Run ID	2026-05-06 Soak (H3TteEtewc)
Duration achieved	4 hours
Latency drift (%)	+0.65%
Error trend	41 errors in first third (381,009 requests), 29 in last third (381,009 requests)
Resource saturation observed	No
Overall scenario result	PASS

JMeter results



Smoothed results vs target



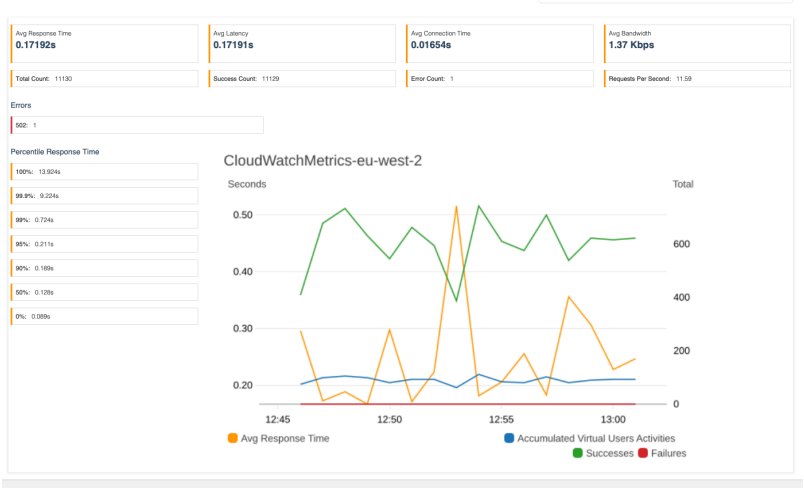
13.5.1.6.1.6 Scenario 5: Spike Test

Objective: Validate scale behaviour for sudden traffic spikes.
Planned profile: 2 TPS for 5 min, jump to 50 TPS for 10 min, return to 2 TPS.

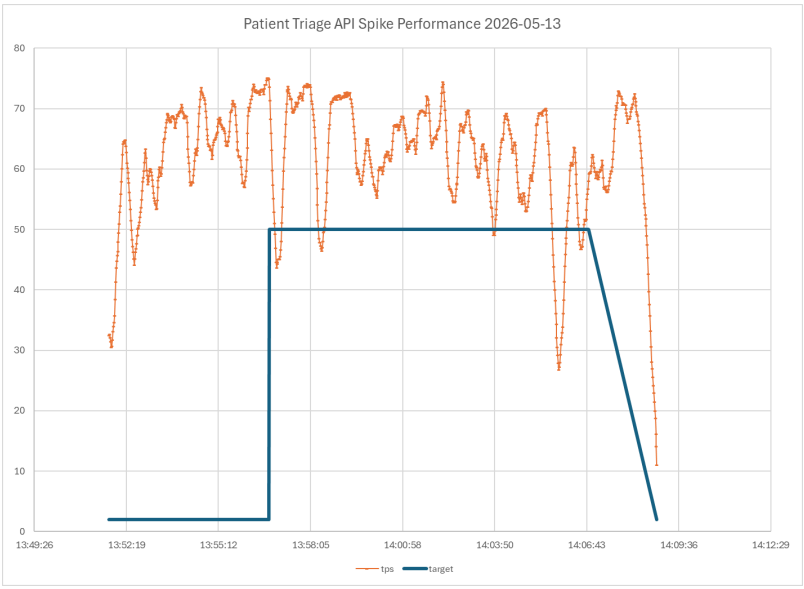
Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Hold baseline at 2 TPS for 5 minutes	Stable baseline metrics	Stable	Done	See below
2	Execute immediate spike to 50 TPS	Rapid scale-up with controlled latency/error impact	controlled latency/error	Done	See below
3	Hold spike for 10 minutes	Service remains available and responsive	Successful	Done	See below
4	Return to 2 TPS	Controlled scale-down, no residual instability	no residual instability	Done	See below
5	Validate spike criteria	No critical SLA breach during transition	No SLA breach	PASS	See below

Result field	Value
Run ID	Spike Test (jif76fcdwl)
Scale-up time	N/A
Scale-down time	N/A
Peak P95 (ms)	212 ms
Transition error rate (%)	0.0089
Overall scenario result	PASS

JMeter results



Smoothed results vs target



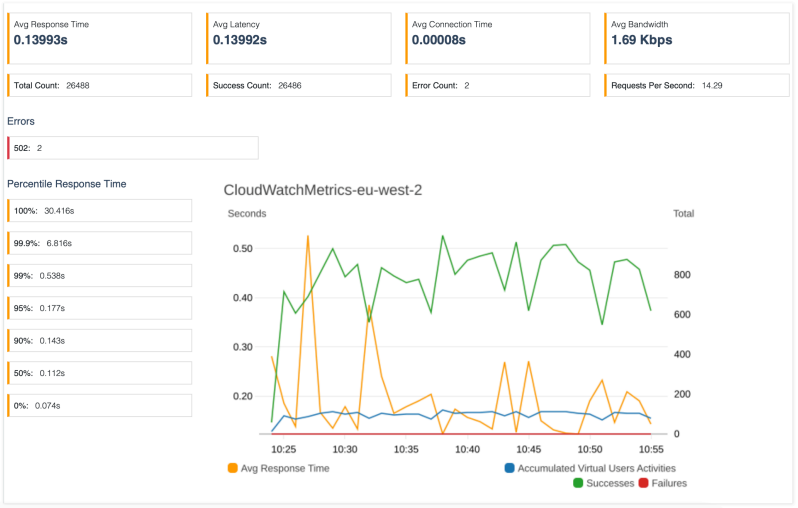
13.5.1.6.1.7 Scenario 6: Capacity Test (2x Normal Traffic)

Objective: Demonstrate resilience at 2x current peak.
Planned profile: 13.12 TPS sustained for 30 minutes.

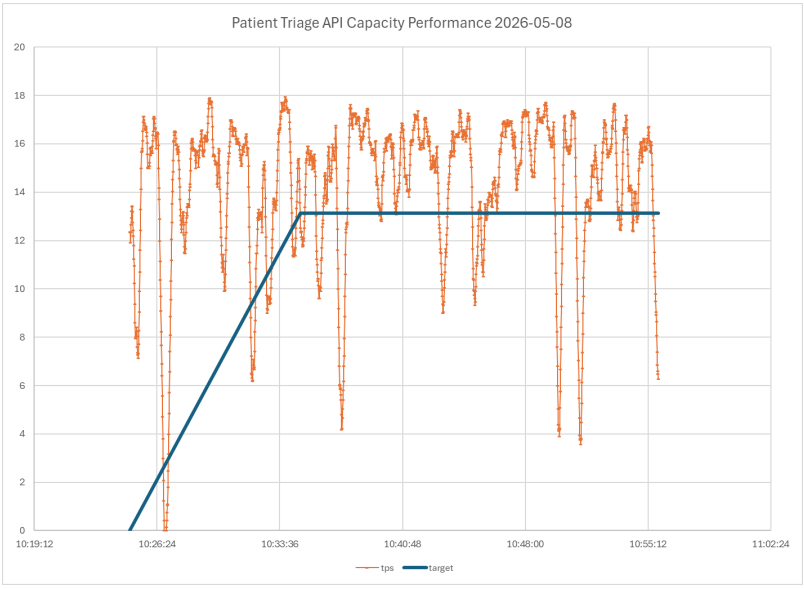
Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Ramp to 13 TPS	Throughput target reached smoothly	reached	Done	See below
2	Hold for 30 minutes	Stable service behaviour	stable	Done	See below
3	Monitor quality thresholds	P95 < 500 ms, error < 1% minimum criteria	117ms	Done	See below
4	Record capacity ceiling signs	Any bottlenecks documented	No bottlenecks	Done	See below

Result field	Value
Run ID	2026-05-08 Capacity Test (HlJjnUxwc9)
TPS sustained	14.29
P95 (ms)	0.177s
Error rate (%)	0.00755%
Constraints observed	none
Overall scenario result	PASS

JMeter results



Smoothed results vs target



13.5.1.7 7. Defects, Risks, and Mitigations

13.5.1.7.1 Defects Logged

ID	Title	Severity	Scenario	Status	Link
DEF-001	<title>	Low	P99 figure were slightly over SLA of 500ms	Open	NPT-1360[70] - Improve P99 Performance BACKLOG

13.5.1.8 8. Optimization Recommendations

13.5.1.8.1 Immediate (Pre-Go-Live)

- <Recommendation 1>
- <Recommendation 2>
- <Recommendation 3>

13.5.1.8.2 Near-Term (Post-Go-Live)

- <Recommendation 1>
- <Recommendation 2>

13.5.1.8.3 Capacity Planning Updates

- Confirm revised safe operating limit: 120TPS is the future state
- Confirm alert threshold updates: Alerts are aligned with the SLA P95 300ms and P99 500ms
- Confirm runbook updates completed: No updates required

13.5.1.9 9. Acceptance Decision

13.5.1.9.1 Critical Pass Criteria Check

Criterion	Required	Result	Status
P95 < 300 ms up to 13 TPS	Must meet	See above	PASS
Peak error rate < 0.1%	Must meet	See above	PASS

70. https://nhsd-jira.digital.nhs.uk/browse/NPT-1360?src=confmacro

Criterion	Required	Result	Status
5xx under normal load < 0.01%	Must meet	See above	PASS
99.90% availability	Must meet	See above	PASS
No data corruption/loss	Must meet	See above	PASS
Autoscaling maintains SLA in spike tests	Must meet	No monitored in this test	N/A

13.5.1.9.2 Conditional Criteria Check

Criterion	Expected	Result	Status
P99 < 500 ms across scenarios	Should meet	Most of the P99 results were > 500ms but still sub second	FAIL
Error rate at 2x traffic < 0.5%	Should meet	See above	PASS

13.5.1.9.3 Final Decision

- Final status: PASS
- Conditions (if any): <list>
- Approval required before production: <Yes/No>

13.5.2 Patient Triage API - Performance Test Results [28/10/2026]

13.5.2.1 Summary

This page captures the execution outcomes of the **Patient Triage API Performance Testing Strategy** and confirms whether the service meets defined non-functional requirements (NFRs) for performance, scalability, reliability, and availability.

- Test window: <DD MM YYYY> to <DD MM YYYY>
- Environment: <DEV / TEST / pre-PROD>
- Build/version under test: <version>
- Test owner: <name>
- Report version: 1.0
- Status: <Draft / In Review / Approved>

13.5.2.2 Document Control

Item	Details
Author	Performance Engineering Team
Results page owner	<Name>
Date created	<DD MMM YYYY>
Last updated	<DD MMM YYYY>
Linked strategy	Patient Triage Api - Performance Testing Strategy
Next review date	<DD MMM YYYY>

13.5.2.3 1. Executive Outcome

13.5.2.3.1 Overall Result

Overall verdict: <PASS / CONDITIONAL PASS / FAIL>

13.5.2.3.2 Headline Findings

- P95 latency target (< 300 ms): <Met / Not met>
- P99 latency target (< 500 ms): <Met / Not met>
- Peak error rate target (< 0.1%): <Met / Not met>
- 5xx target under normal load (< 0.01%): <Met / Not met>
- Availability target (99.90%): <Met / Not met>
- Peak tested throughput: <X TPS>
- Autoscaling behavior: <Stable / Partial / Unstable>

13.5.2.3.3 Go/No-Go Recommendation

- Recommendation: <GO / GO WITH CONDITIONS / NO-GO>
- Rationale: <short justification>

13.5.2.4 2. KPI Results Against Targets

KPI	Target	Actual	Status	Notes
P50 response time (Triage POST)	< 150 ms	<ms>	<Pass/Fail>	<notes>
P95 response time (Triage POST)	< 300 ms	<ms>	<Pass/Fail>	<notes>
P99 response time (Triage POST)	< 500 ms	<ms>	<Pass/Fail>	<notes>
Peak load error rate	< 0.1%	<%>	<Pass/Fail>	<notes>
5xx rate at normal load	< 0.01%	<%>	<Pass/Fail>	<notes>
Service availability	99.90%	<%>	<Pass/Fail>	<notes>
Max sustained throughput	120 TPS target	<TPS>	<Pass/Fail>	<notes>

13.5.2.5 3. Test Coverage and Execution Evidence

13.5.2.5.1 Scope Confirmation

- In-scope scenarios executed: <Yes/No>
- Out-of-scope items excluded: <Yes/No>
- Deviations from strategy: <None / List below>

13.5.2.5.2 Evidence Links

- Test scripts repo/path: <link>
- JMeter/Gatling test plan versions: <link>
- CloudWatch dashboard: <link>
- X-Ray trace group: <link>
- ODIN/Grafana dashboard: <link>
- Raw run logs/artifacts: <link>

13.5.2.6 4. Scenario-by-Scenario Results

Scenario	Planned Profile	Actual Profile	Key Results	Status	Defects / Incidents
1. Baseline	2 TPS, 30 min	<actual profile>	P95: <ms>, Error: <%>	<Pass/ Fail>	<refs>
2. Current Peak	6 TPS, 60 min	<actual profile>	P95: <ms>, Error: <%>	<Pass/ Fail>	<refs>
3. Stress (Future State)	Ramp 6 to 120 TPS, hold 60 min	<actual profile>	P95: <ms>, Error: <%>, Throttle: <value>	<Pass/ Fail>	<refs>
4. Endurance (Soak)	120 TPS, 4 hours	<actual profile>	Drift in latency: <value>, Error trend: <value>	<Pass/ Fail>	<refs>
5. Spike	2 to 50 to 2 TPS	<actual profile>	Scale-up time: <value>, SLA breaches: <value>	<Pass/ Fail>	<refs>
6. 2x Traffic Buffer	13 TPS, 30 min	<actual profile>	P95: <ms>, Error: <%>	<Pass/ Fail>	<refs>
7. Data Layer	DynamoDB/Cache validation	<actual profile>	Read/Write latency: <value>, Throttle: <value>	<Pass/ Fail>	<refs>

13.5.2.6.1 Execution Detail Per Test (Steps + Result Capture)

13.5.2.6.1.1 Common Pre-Checks (Run Before Every Scenario)

Check	Owner	Status	Evidence
Test environment healthy and isolated	<owner>	<Done/Not done>	<link>
Monitoring dashboards active (Grafana)	<owner>	<Done/Not done>	<link>
Test data loaded and validated	<owner>	<Done/Not done>	<link>
Script version and config signed off	<owner>	<Done/Not done>	<link>
Incident channel and rollback plan ready	<owner>	<Done/Not done>	<link>

13.5.2.6.1.2 Scenario 1: Baseline Performance Test

Objective: Establish baseline under normal operating load.
Planned profile: 2 TPS sustained for 30 minutes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Confirm no background load and environment readiness	Stable baseline conditions confirmed	<actual>	<Done/ Failed>	<link>
2	Start baseline script at 2 TPS	Steady 2 TPS reached without throttling	<actual>	<Done/ Failed>	<link>
3	Run for full 30 minutes	No interruptions or major variance	<actual>	<Done/ Failed>	<link>
4	Capture latency and error metrics	P50/P95/P99 and errors available	<actual>	<Done/ Failed>	<link>
5	Validate against baseline acceptance	P95 < 300 ms and error rate < 0.01%	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-BASE-001>
Start / End time	<date time - date time>
P50 / P95 / P99 (ms)	<value / value / value>
Error rate (%)	<value>
Overall scenario result	<Pass/Fail>
Notes	<notes>

13.5.2.6.1.3 Scenario 2: Current Peak Demand Load Test

Objective: Validate performance at currently observed peak.
Planned profile: 6.56 TPS sustained for 60 minutes, 10-minute ramp.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Ramp from baseline to 6 TPS over 10 minutes	Smooth ramp, no immediate SLA breach	<actual>	<Done/ Failed>	<link>
2	Hold 6 TPS for 60 minutes	Stable throughput and response profile	<actual>	<Done/ Failed>	<link>
3	Monitor Lambda/API Gateway/ DynamoDB behavior	No capacity alarms or sustained throttling	<actual>	<Done/ Failed>	<link>
4	Validate SLA and error limits	P95 < 300 ms, error rate < 0.1%	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-PEAK-001>
Start / End time	<date time - date time>
P95 / P99 (ms)	<value / value>
Error rate (%)	<value>
Time to steady-state	<value>
Overall scenario result	<Pass/Fail>

13.5.2.6.1.4 Scenario 3: Stress Test (Future State Capacity)

Objective: Validate future growth capacity at 120 TPS.
Planned profile: Ramp 6.56 to 120 TPS over 30 minutes, hold 120 TPS for 60 minutes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Warm-up at 6 TPS	Stable pre-stress baseline	<actual>	<Done/ Failed>	<link>
2	Ramp to 120 TPS over 30 minutes	Controlled scaling and stable error profile	<actual>	<Done/ Failed>	<link>

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
3	Hold 120 TPS for 60 minutes	Sustained load without service degradation	<actual>	<Done/ Failed>	<link>
4	Track autoscaling, cold starts, throttles	Scaling events recorded and acceptable	<actual>	<Done/ Failed>	<link>
5	Validate acceptance criteria	P95 < 300 ms, error < 0.1%, no harmful throttling	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-STRESS-001>
Peak TPS achieved	<value>
P95 / P99 at peak (ms)	<value / value>
Error rate at peak (%)	<value>
Autoscaling response time	<value>
Overall scenario result	<Pass/Fail>

13.5.2.6.1.5 Scenario 4: Endurance (Soak) Test

Objective: Validate long-duration stability and degradation behavior.
Planned profile: 120 TPS for 4 hours.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Start sustained 120 TPS load	Stable entry into steady state	<actual>	<Done/ Failed>	<link>
2	Execute continuous run for 4 hours	No crashes, no progressive degradation	<actual>	<Done/ Failed>	<link>
3	Capture periodic snapshots (hourly)	Trend data available for latency/errors/resources	<actual>	<Done/ Failed>	<link>

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
4	Compare hour 1 vs hour 4 metrics	Minimal drift and acceptable consistency	<actual>	<Done/ Failed>	<link>
5	Validate soak success criteria	P95 stable and error rate consistent	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-SOAK-001>
Duration achieved	<value>
Latency drift (%)	<value>
Error trend	<value>
Resource saturation observed	<Yes/No + notes>
Overall scenario result	<Pass/Fail>

13.5.2.6.1.6 Scenario 5: Spike Test

Objective: Validate scale behavior for sudden traffic spikes.
Planned profile: 2 TPS for 5 min, jump to 50 TPS for 10 min, return to 2 TPS.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Hold baseline at 2 TPS for 5 minutes	Stable baseline metrics	<actual>	<Done/ Failed>	<link>
2	Execute immediate spike to 50 TPS	Rapid scale-up with controlled latency/error impact	<actual>	<Done/ Failed>	<link>
3	Hold spike for 10 minutes	Service remains available and responsive	<actual>	<Done/ Failed>	<link>
4	Return to 2 TPS	Controlled scale-down, no residual instability	<actual>	<Done/ Failed>	<link>

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
5	Validate spike criteria	No critical SLA breach during transition	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-SPIKE-001>
Scale-up time	<value>
Scale-down time	<value>
Peak P95 (ms)	<value>
Transition error rate (%)	<value>
Overall scenario result	<Pass/Fail>

13.5.2.6.1.7 Scenario 6: Capacity Test (2x Normal Traffic)

Objective: Demonstrate resilience at 2x current peak.
Planned profile: 13.12 TPS sustained for 30 minutes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Ramp to 13 TPS	Throughput target reached smoothly	<actual>	<Done/ Failed>	<link>
2	Hold for 30 minutes	Stable service behavior	<actual>	<Done/ Failed>	<link>
3	Monitor quality thresholds	P95 < 500 ms, error < 1% minimum criteria	<actual>	<Done/ Failed>	<link>
4	Record capacity ceiling signs	Any bottlenecks documented	<actual>	<Done/ Failed>	<link>

Result field	Value
Run ID	<RUN-2X-001>
TPS sustained	<value>
P95 (ms)	<value>
Error rate (%)	<value>
Constraints observed	<notes>
Overall scenario result	<Pass/Fail>

13.5.2.6.1.8 Scenario 7: Error Scenario Testing

Objective: Validate resilience and recovery behavior under fault conditions.
Planned profile: Dependency failures, injected errors, partial failures, resource exhaustion.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Inject dependency timeout/failure	Graceful handling, no cascade failure	<actual>	<Done/ Failed>	<link>
2	Inject controlled error responses	Recovery within expected timeframe	<actual>	<Done/ Failed>	<link>
3	Simulate partial infrastructure failure	Service remains resilient with degraded mode	<actual>	<Done/ Failed>	<link>
4	Push resource limits (safe boundary)	System protection mechanisms engage correctly	<actual>	<Done/ Failed>	<link>
5	Verify state consistency and recovery	No data corruption/loss after recovery	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-ERR-001>
Recovery time (MTTR style)	<value>

Result field	Value
Cascade failure observed	<Yes/No>
Data integrity outcome	<Pass/Fail>
Open defects raised	<count + refs>
Overall scenario result	<Pass/Fail>

13.5.2.6.1.9 Scenario 8: Data Layer Performance Test

Objective: Validate DynamoDB and cache behavior under load.
Planned profile: Read/write throughput and latency validation at realistic and stress volumes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Execute write-heavy profile	Writes complete within acceptable latency	<actual>	<Done/ Failed>	<link>
2	Execute read-heavy profile	Reads meet target latency profile	<actual>	<Done/ Failed>	<link>
3	Monitor consumed capacity and throttling	No persistent throttling under expected limits	<actual>	<Done/ Failed>	<link>
4	Validate cache behavior (if applicable)	Hit ratio and eviction trend acceptable	<actual>	<Done/ Failed>	<link>
5	Compare cached vs uncached latency	Clear performance benefit documented	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-DATA-001>
Read P95 / Write P95 (ms)	<value / value>
DynamoDB throttles	<value>
Cache hit ratio / eviction rate	<value / value>

Result field	Value
Data layer bottleneck summary	<notes>
Overall scenario result	<Pass/Fail>

13.5.2.7 5. Autoscaling and Resilience Results

13.5.2.7.1 Autoscaling

Metric	Target	Actual	Outcome
Time to scale up	<TBC>	<value>	<Pass/Fail>
Time to scale down	<TBC>	<value>	<Pass/Fail>
Lambda cold start impact	Minimize SLA impact	<value>	<Pass/Fail>
Throttling events (Lambda/API Gateway/DynamoDB)	0 in normal operation	<value>	<Pass/Fail>

13.5.2.7.2 Resilience and Recovery

- Dependency failure handling: <Observed behavior>
- Error injection recovery: <Observed behavior>
- AZ/partial failure behavior: <Observed behavior>
- Data integrity checks: <Pass/Fail + notes>

13.5.2.8 6. Bottlenecks and Root Cause Analysis

13.5.2.8.1 Top Bottlenecks Identified

Rank	Component	Symptom	Trigger Condition	Impact	Root Cause Hypothesis
1	<Component>	<Symptom>	<Condition>	<Impact>	<Cause>
2	<Component>	<Symptom>	<Condition>	<Impact>	<Cause>

Rank	Component	Symptom	Trigger Condition	Impact	Root Cause Hypothesis
3	<Component>	<Symptom>	<Condition>	<Impact>	<Cause>

13.5.2.8.2 Supporting Observations

- <Observation 1>
- <Observation 2>
- <Observation 3>

13.5.2.9 7. Defects, Risks, and Mitigations

13.5.2.9.1 Defects Logged

ID	Title	Severity	Scenario	Status	Link
<DEF-001>	<title>	<Low/Med/High/Critical>	<scenario>	<Open/Closed>	<link>

13.5.2.9.2 Residual Risks

Risk	Current Exposure	Mitigation	Owner	Due Date
<risk>	<exposure>	<mitigation>	<owner>	<DD MMM YYYY>

13.5.2.10 8. Optimization Recommendations

13.5.2.10.1 Immediate (Pre-Go-Live)

- <Recommendation 1>
- <Recommendation 2>
- <Recommendation 3>

13.5.2.10.2 Near-Term (Post-Go-Live)

- <Recommendation 1>
- <Recommendation 2>

13.5.2.10.3 Capacity Planning Updates

- Confirm revised safe operating limit: <X TPS>
- Confirm alert threshold updates: <details>
- Confirm runbook updates completed: <Yes/No>

13.5.2.11 9. Acceptance Decision

13.5.2.11.1 Critical Pass Criteria Check

Criterion	Required	Result	Status
P95 < 300 ms up to 13 TPS	Must meet	<result>	<Pass/Fail>
Peak error rate < 0.1%	Must meet	<result>	<Pass/Fail>
5xx under normal load < 0.01%	Must meet	<result>	<Pass/Fail>
99.90% availability	Must meet	<result>	<Pass/Fail>
No data corruption/loss	Must meet	<result>	<Pass/Fail>
Autoscaling maintains SLA in spike tests	Must meet	<result>	<Pass/Fail>

13.5.2.11.2 Conditional Criteria Check

Criterion	Expected	Result	Status
P99 < 500 ms across scenarios	Should meet	<result>	<Pass/Fail>
Error rate at 2x traffic < 0.5%	Should meet	<result>	<Pass/Fail>
Autoscaling response acceptable	Should meet	<result>	<Pass/Fail>
Cache efficiency target (if applicable)	Should meet	<result>	<Pass/Fail>
DynamoDB utilization optimized	Should meet	<result>	<Pass/Fail>

13.5.2.11.3 Final Decision

- Final status: <PASS / CONDITIONAL PASS / FAIL>
- Conditions (if any): <list>
- Approval required before production: <Yes/No>

13.5.2.12 10. Sign-Off

Role	Name	Decision	Date	Signature
Performance Engineering Lead	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Architecture Lead	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Test Manager	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Operations Lead	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Project Manager	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>

13.5.2.13 11. Appendices

13.5.2.13.1 Appendix A: Run Inventory

Run ID	Date/Time	Scenario	Build	Duration	Status
<RUN-001>	<date/time>	<scenario>	<build>	<duration>	<Pass/Fail>

13.5.2.13.2 Appendix B: Metric Snapshots

- Include representative charts for:
  - Throughput vs latency
  - P95/P99 over time
  - Error rate trend
  - Lambda concurrency/cold starts
  - DynamoDB consumed capacity/throttles

13.5.2.13.3 Appendix C: Environment and Configuration

- Lambda memory and timeout settings: <value>
- Reserved/provisioned concurrency: <value>
- API Gateway throttling limits: <value>
- DynamoDB mode and capacity: <value>
- Cache topology and sizing (if used): <value>

13.5.2.14 How To Use This Page

1. Paste this content into a new Confluence page using wiki markup.
2. Replace all placeholder values with actual run data.
3. Add chart images/links from CloudWatch, Grafana, and X-Ray.
4. Complete the acceptance tables before requesting sign-off.
5. Set status to **Approved** only after all critical criteria are met.

13.5.3 Patient Triage API - Performance Test Results [30/10/2025 - 28/01/2026]

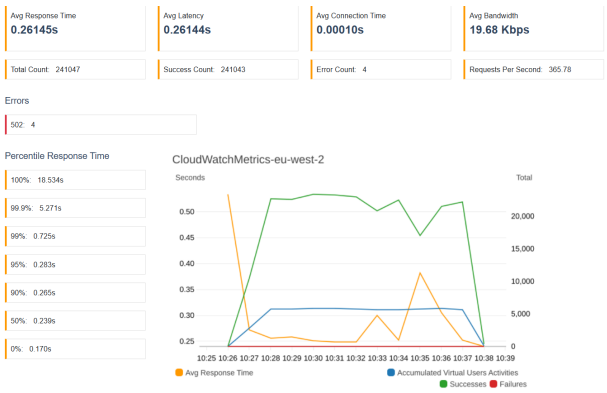
Scope: latency more than TPS. lambda solves TPS, but we still have a single APIGW and dynamoDB that might prove to be bottlenecks (unlikely).

Goal for performance testing is mostly to observe latency degradation as the solution is built out. If a change results in significantly worse latency, we know the change is likely responsible.

Tools: https://aws.amazon.com/solutions/implementations/distributed-load-testing-on-aws/

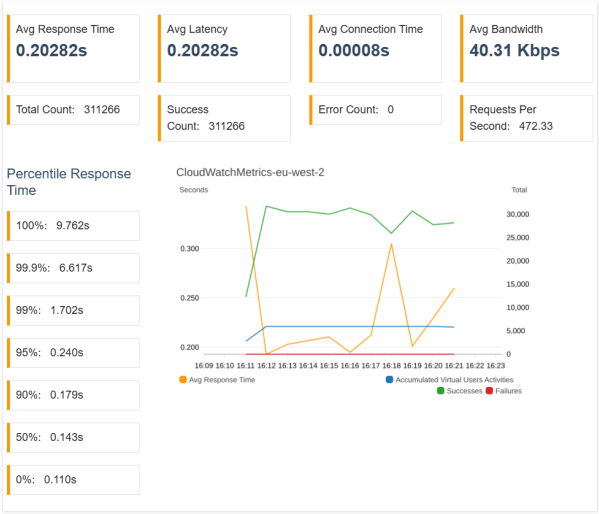
13.5.3.1 Measurements

13.5.3.1.1 2026-01-28



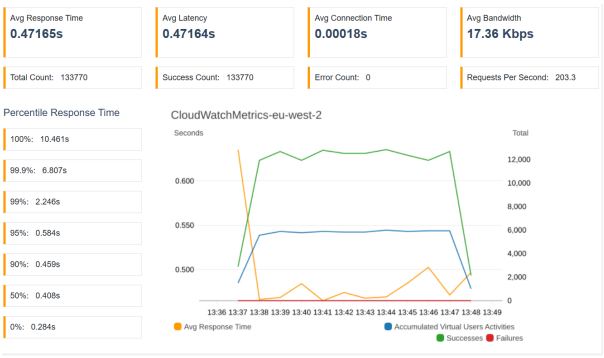
Inputs		Outputs	
Tasks:	5	**Average response time:**	0.26145s
Concurrency:	20	**Success:**	99.998%
Ramp up:	1 minute		
Hold:	10 minutes		

13.5.3.1.2 2025-12-01 (Alpha + DAX)



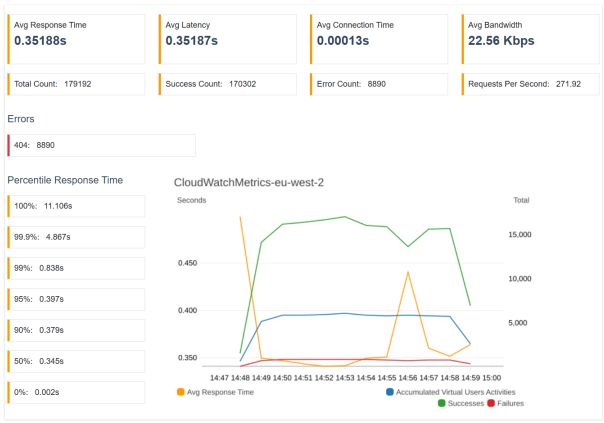
Inputs		Outputs	
Cluster:	dax.r5.2xlarge, 5 nodes	**Average response time:**	0.20282s
Tasks:	5	**Success:**	100.00%
Concurrency:	20		
Ramp up:	1 minute		
Hold:	10 minutes		

13.5.3.1.3 2025-11-27 (Alpha)



Inputs		Outputs	
Tasks:	5	**Average response time:**	0.47164s
Concurrency:	20	**Success:**	100.00%
Ramp up:	1 minute		
Hold:	10 minutes		

13.5.3.1.4 2025-11-11 (WIP)



Inputs	Outputs

Tasks:	5	**Average response time:**	0.35188s
Concurrency:	20	**Success:**	95.04%
Ramp up:	1 minute		
Hold:	10 minutes		

13.5.3.1.5 2025-10-30 (WIP)



Inputs		Outputs	
Tasks:	1	**Average response time:**	0.05842s
Concurrency:	1	**Success:**	100.00%
Ramp up:	1 minute		
Hold:	1 minute		

13.5.4 Patient Triage API - Performance Test Results [DD/MM/YYYY]

13.5.4.1 Summary

This page captures the execution outcomes of the **Patient Triage API Performance Testing Strategy** and confirms whether the service meets defined non-functional requirements (NFRs) for performance, scalability, reliability, and availability.

- Test window: <DD MM YYYY> to <DD MM YYYY>
- Environment: <DEV / TEST / pre-PROD>
- Build/version under test: <version>
- Test owner: <name>
- Report version: 1.0
- Status: <Draft / In Review / Approved>

13.5.4.2 Document Control

Item	Details
Author	Performance Engineering Team
Results page owner	<Name>
Date created	<DD MMM YYYY>
Last updated	<DD MMM YYYY>
Linked strategy	Patient Triage Api - Performance Testing Strategy
Next review date	<DD MMM YYYY>

13.5.4.3 1. Executive Outcome

13.5.4.3.1 Overall Result

Overall verdict: <PASS / CONDITIONAL PASS / FAIL>

13.5.4.3.2 Headline Findings

- P95 latency target (< 300 ms): <Met / Not met>
- P99 latency target (< 500 ms): <Met / Not met>
- Peak error rate target (< 0.1%): <Met / Not met>
- 5xx target under normal load (< 0.01%): <Met / Not met>
- Availability target (99.90%): <Met / Not met>
- Peak tested throughput: <X TPS>
- Autoscaling behavior: <Stable / Partial / Unstable>

13.5.4.3.3 Go/No-Go Recommendation

- Recommendation: <GO / GO WITH CONDITIONS / NO-GO>
- Rationale: <short justification>

13.5.4.4 2. KPI Results Against Targets

KPI	Target	Actual	Status	Notes
P50 response time (Triage POST)	< 150 ms	<ms>	<Pass/Fail>	<notes>
P95 response time (Triage POST)	< 300 ms	<ms>	<Pass/Fail>	<notes>
P99 response time (Triage POST)	< 500 ms	<ms>	<Pass/Fail>	<notes>
Peak load error rate	< 0.1%	<%>	<Pass/Fail>	<notes>
5xx rate at normal load	< 0.01%	<%>	<Pass/Fail>	<notes>
Service availability	99.90%	<%>	<Pass/Fail>	<notes>
Max sustained throughput	120 TPS target	<TPS>	<Pass/Fail>	<notes>

13.5.4.5 3. Test Coverage and Execution Evidence

13.5.4.5.1 Scope Confirmation

- In-scope scenarios executed: <Yes/No>

- Out-of-scope items excluded: <Yes/No>
- Deviations from strategy: <None / List below>

13.5.4.5.2 Evidence Links

- Test scripts repo/path: <link>
- JMeter/Gatling test plan versions: <link>
- CloudWatch dashboard: <link>
- X-Ray trace group: <link>
- ODIN/Grafana dashboard: <link>
- Raw run logs/artifacts: <link>

13.5.4.6 4. Scenario-by-Scenario Results

Scenario	Planned Profile	Actual Profile	Key Results	Status	Defects / Incidents
1. Baseline	2 TPS, 30 min	<actual profile>	P95: <ms>, Error: <%>	<Pass/ Fail>	<refs>
2. Current Peak	6 TPS, 60 min	<actual profile>	P95: <ms>, Error: <%>	<Pass/ Fail>	<refs>
3. Stress (Future State)	Ramp 6 to 120 TPS, hold 60 min	<actual profile>	P95: <ms>, Error: <%>, Throttle: <value>	<Pass/ Fail>	<refs>
4. Endurance (Soak)	120 TPS, 4 hours	<actual profile>	Drift in latency: <value>, Error trend: <value>	<Pass/ Fail>	<refs>
5. Spike	2 to 50 to 2 TPS	<actual profile>	Scale-up time: <value>, SLA breaches: <value>	<Pass/ Fail>	<refs>
6. 2x Traffic Buffer	13 TPS, 30 min	<actual profile>	P95: <ms>, Error: <%>	<Pass/ Fail>	<refs>
7. Data Layer	DynamoDB/Cache validation	<actual profile>	Read/Write latency: <value>, Throttle: <value>	<Pass/ Fail>	<refs>

13.5.4.6.1 Execution Detail Per Test (Steps + Result Capture)

13.5.4.6.1.1 Common Pre-Checks (Run Before Every Scenario)

Check	Owner	Status	Evidence
Test environment healthy and isolated	<owner>	<Done/Not done>	<link>
Monitoring dashboards active (Grafana)	<owner>	<Done/Not done>	<link>
Test data loaded and validated	<owner>	<Done/Not done>	<link>
Script version and config signed off	<owner>	<Done/Not done>	<link>
Incident channel and rollback plan ready	<owner>	<Done/Not done>	<link>

13.5.4.6.1.2 Scenario 1: Baseline Performance Test

Objective: Establish baseline under normal operating load.
Planned profile: 2 TPS sustained for 30 minutes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Confirm no background load and environment readiness	Stable baseline conditions confirmed	<actual>	<Done/Failed>	<link>
2	Start baseline script at 2 TPS	Steady 2 TPS reached without throttling	<actual>	<Done/Failed>	<link>
3	Run for full 30 minutes	No interruptions or major variance	<actual>	<Done/Failed>	<link>
4	Capture latency and error metrics	P50/P95/P99 and errors available	<actual>	<Done/Failed>	<link>
5	Validate against baseline acceptance	P95 < 300 ms and error rate < 0.01%	<actual>	<Pass/Fail>	<link>

Result field	Value
Run ID	<RUN-BASE-001>

Result field	Value
Start / End time	<date time - date time>
P50 / P95 / P99 (ms)	<value / value / value>
Error rate (%)	<value>
Overall scenario result	<Pass/Fail>
Notes	<notes>

13.5.4.6.1.3 Scenario 2: Current Peak Demand Load Test

Objective: Validate performance at currently observed peak.
Planned profile: 6.56 TPS sustained for 60 minutes, 10-minute ramp.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Ramp from baseline to 6 TPS over 10 minutes	Smooth ramp, no immediate SLA breach	<actual>	<Done/ Failed>	<link>
2	Hold 6 TPS for 60 minutes	Stable throughput and response profile	<actual>	<Done/ Failed>	<link>
3	Monitor Lambda/API Gateway/ DynamoDB behavior	No capacity alarms or sustained throttling	<actual>	<Done/ Failed>	<link>
4	Validate SLA and error limits	P95 < 300 ms, error rate < 0.1%	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-PEAK-001>
Start / End time	<date time - date time>
P95 / P99 (ms)	<value / value>
Error rate (%)	<value>

Result field	Value
Time to steady-state	<value>
Overall scenario result	<Pass/Fail>

13.5.4.6.1.4 Scenario 3: Stress Test (Future State Capacity)

Objective: Validate future growth capacity at 120 TPS.
Planned profile: Ramp 6.56 to 120 TPS over 30 minutes, hold 120 TPS for 60 minutes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Warm-up at 6 TPS	Stable pre-stress baseline	<actual>	<Done/ Failed>	<link>
2	Ramp to 120 TPS over 30 minutes	Controlled scaling and stable error profile	<actual>	<Done/ Failed>	<link>
3	Hold 120 TPS for 60 minutes	Sustained load without service degradation	<actual>	<Done/ Failed>	<link>
4	Track autoscaling, cold starts, throttles	Scaling events recorded and acceptable	<actual>	<Done/ Failed>	<link>
5	Validate acceptance criteria	P95 < 300 ms, error < 0.1%, no harmful throttling	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-STRESS-001>
Peak TPS achieved	<value>
P95 / P99 at peak (ms)	<value / value>
Error rate at peak (%)	<value>
Autoscaling response time	<value>
Overall scenario result	<Pass/Fail>

13.5.4.6.1.5 Scenario 4: Endurance (Soak) Test

Objective: Validate long-duration stability and degradation behavior.
Planned profile: 120 TPS for 4 hours.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Start sustained 120 TPS load	Stable entry into steady state	<actual>	<Done/ Failed>	<link>
2	Execute continuous run for 4 hours	No crashes, no progressive degradation	<actual>	<Done/ Failed>	<link>
3	Capture periodic snapshots (hourly)	Trend data available for latency/errors/resources	<actual>	<Done/ Failed>	<link>
4	Compare hour 1 vs hour 4 metrics	Minimal drift and acceptable consistency	<actual>	<Done/ Failed>	<link>
5	Validate soak success criteria	P95 stable and error rate consistent	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-SOAK-001>
Duration achieved	<value>
Latency drift (%)	<value>
Error trend	<value>
Resource saturation observed	<Yes/No + notes>
Overall scenario result	<Pass/Fail>

13.5.4.6.1.6 Scenario 5: Spike Test

Objective: Validate scale behavior for sudden traffic spikes.
Planned profile: 2 TPS for 5 min, jump to 50 TPS for 10 min, return to 2 TPS.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Hold baseline at 2 TPS for 5 minutes	Stable baseline metrics	<actual>	<Done/ Failed>	<link>
2	Execute immediate spike to 50 TPS	Rapid scale-up with controlled latency/error impact	<actual>	<Done/ Failed>	<link>
3	Hold spike for 10 minutes	Service remains available and responsive	<actual>	<Done/ Failed>	<link>
4	Return to 2 TPS	Controlled scale-down, no residual instability	<actual>	<Done/ Failed>	<link>
5	Validate spike criteria	No critical SLA breach during transition	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-SPIKE-001>
Scale-up time	<value>
Scale-down time	<value>
Peak P95 (ms)	<value>
Transition error rate (%)	<value>
Overall scenario result	<Pass/Fail>

13.5.4.6.1.7 Scenario 6: Capacity Test (2x Normal Traffic)

Objective: Demonstrate resilience at 2x current peak.
Planned profile: 13.12 TPS sustained for 30 minutes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Ramp to 13 TPS	Throughput target reached smoothly	<actual>	<Done/ Failed>	<link>

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
2	Hold for 30 minutes	Stable service behavior	<actual>	<Done/ Failed>	<link>
3	Monitor quality thresholds	P95 < 500 ms, error < 1% minimum criteria	<actual>	<Done/ Failed>	<link>
4	Record capacity ceiling signs	Any bottlenecks documented	<actual>	<Done/ Failed>	<link>

Result field	Value
Run ID	<RUN-2X-001>
TPS sustained	<value>
P95 (ms)	<value>
Error rate (%)	<value>
Constraints observed	<notes>
Overall scenario result	<Pass/Fail>

13.5.4.6.1.8 Scenario 7: Error Scenario Testing

Objective: Validate resilience and recovery behavior under fault conditions.
Planned profile: Dependency failures, injected errors, partial failures, resource exhaustion.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Inject dependency timeout/failure	Graceful handling, no cascade failure	<actual>	<Done/ Failed>	<link>
2	Inject controlled error responses	Recovery within expected timeframe	<actual>	<Done/ Failed>	<link>
3	Simulate partial infrastructure failure	Service remains resilient with degraded mode	<actual>	<Done/ Failed>	<link>

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
4	Push resource limits (safe boundary)	System protection mechanisms engage correctly	<actual>	<Done/ Failed>	<link>
5	Verify state consistency and recovery	No data corruption/loss after recovery	<actual>	<Pass/ Fail>	<link>

Result field	Value
Run ID	<RUN-ERR-001>
Recovery time (MTTR style)	<value>
Cascade failure observed	<Yes/No>
Data integrity outcome	<Pass/Fail>
Open defects raised	<count + refs>
Overall scenario result	<Pass/Fail>

13.5.4.6.1.9 Scenario 8: Data Layer Performance Test

Objective: Validate DynamoDB and cache behavior under load.
Planned profile: Read/write throughput and latency validation at realistic and stress volumes.

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
1	Execute write-heavy profile	Writes complete within acceptable latency	<actual>	<Done/ Failed>	<link>
2	Execute read-heavy profile	Reads meet target latency profile	<actual>	<Done/ Failed>	<link>
3	Monitor consumed capacity and throttling	No persistent throttling under expected limits	<actual>	<Done/ Failed>	<link>
4	Validate cache behavior (if applicable)	Hit ratio and eviction trend acceptable	<actual>	<Done/ Failed>	<link>

Step	Required action	Expected outcome	Actual outcome	Status	Evidence
5	Compare cached vs uncached latency	Clear performance benefit documented	<actual>	<Pass/Fail>	<link>

Result field	Value
Run ID	<RUN-DATA-001>
Read P95 / Write P95 (ms)	<value / value>
DynamoDB throttles	<value>
Cache hit ratio / eviction rate	<value / value>
Data layer bottleneck summary	<notes>
Overall scenario result	<Pass/Fail>

13.5.4.7 5. Autoscaling and Resilience Results

13.5.4.7.1 Autoscaling

Metric	Target	Actual	Outcome
Time to scale up	<TBC>	<value>	<Pass/Fail>
Time to scale down	<TBC>	<value>	<Pass/Fail>
Lambda cold start impact	Minimize SLA impact	<value>	<Pass/Fail>
Throttling events (Lambda/API Gateway/DynamoDB)	0 in normal operation	<value>	<Pass/Fail>

13.5.4.7.2 Resilience and Recovery

- Dependency failure handling: <Observed behavior>
- Error injection recovery: <Observed behavior>
- AZ/partial failure behavior: <Observed behavior>

- Data integrity checks: <Pass/Fail + notes>

13.5.4.8 6. Bottlenecks and Root Cause Analysis

13.5.4.8.1 Top Bottlenecks Identified

Rank	Component	Symptom	Trigger Condition	Impact	Root Cause Hypothesis
1	<Component>	<Symptom>	<Condition>	<Impact>	<Cause>
2	<Component>	<Symptom>	<Condition>	<Impact>	<Cause>
3	<Component>	<Symptom>	<Condition>	<Impact>	<Cause>

13.5.4.8.2 Supporting Observations

- <Observation 1>
- <Observation 2>
- <Observation 3>

13.5.4.9 7. Defects, Risks, and Mitigations

13.5.4.9.1 Defects Logged

ID	Title	Severity	Scenario	Status	Link
<DEF-001>	<title>	<Low/Med/High/Critical>	<scenario>	<Open/Closed>	<link>

13.5.4.9.2 Residual Risks

Risk	Current Exposure	Mitigation	Owner	Due Date
<risk>	<exposure>	<mitigation>	<owner>	<DD MMM YYYY>

13.5.4.10 8. Optimization Recommendations

13.5.4.10.1 Immediate (Pre-Go-Live)

- <Recommendation 1>
- <Recommendation 2>
- <Recommendation 3>

13.5.4.10.2 Near-Term (Post-Go-Live)

- <Recommendation 1>
- <Recommendation 2>

13.5.4.10.3 Capacity Planning Updates

- Confirm revised safe operating limit: <X TPS>
- Confirm alert threshold updates: <details>
- Confirm runbook updates completed: <Yes/No>

13.5.4.11 9. Acceptance Decision

13.5.4.11.1 Critical Pass Criteria Check

Criterion	Required	Result	Status
P95 < 300 ms up to 13 TPS	Must meet	<result>	<Pass/Fail>
Peak error rate < 0.1%	Must meet	<result>	<Pass/Fail>
5xx under normal load < 0.01%	Must meet	<result>	<Pass/Fail>
99.90% availability	Must meet	<result>	<Pass/Fail>
No data corruption/loss	Must meet	<result>	<Pass/Fail>
Autoscaling maintains SLA in spike tests	Must meet	<result>	<Pass/Fail>

13.5.4.11.2 Conditional Criteria Check

Criterion	Expected	Result	Status
P99 < 500 ms across scenarios	Should meet	<result>	<Pass/Fail>
Error rate at 2x traffic < 0.5%	Should meet	<result>	<Pass/Fail>
Autoscaling response acceptable	Should meet	<result>	<Pass/Fail>
Cache efficiency target (if applicable)	Should meet	<result>	<Pass/Fail>
DynamoDB utilization optimized	Should meet	<result>	<Pass/Fail>

13.5.4.11.3 Final Decision

- Final status: <PASS / CONDITIONAL PASS / FAIL>
- Conditions (if any): <list>
- Approval required before production: <Yes/No>

13.5.4.12 10. Sign-Off

Role	Name	Decision	Date	Signature
Performance Engineering Lead	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Architecture Lead	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Test Manager	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Operations Lead	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>
Project Manager	<TBC>	<Approve/Reject>	<DD MMM YYYY>	<signature>

13.5.4.13 11. Appendices

13.5.4.13.1 Appendix A: Run Inventory

Run ID	Date/Time	Scenario	Build	Duration	Status
<RUN-001>	<date/time>	<scenario>	<build>	<duration>	<Pass/Fail>

13.5.4.13.2 Appendix B: Metric Snapshots

- Include representative charts for:
  - Throughput vs latency
  - P95/P99 over time
  - Error rate trend
  - Lambda concurrency/cold starts
  - DynamoDB consumed capacity/throttles

13.5.4.13.3 Appendix C: Environment and Configuration

- Lambda memory and timeout settings: <value>
- Reserved/provisioned concurrency: <value>
- API Gateway throttling limits: <value>
- DynamoDB mode and capacity: <value>
- Cache topology and sizing (if used): <value>

13.5.4.14 How To Use This Page

1. Paste this content into a new Confluence page using wiki markup.
2. Replace all placeholder values with actual run data.
3. Add chart images/links from CloudWatch, Grafana, and X-Ray.
4. Complete the acceptance tables before requesting sign-off.
5. Set status to **Approved** only after all critical criteria are met.

13.6 Private Beta Testing Summary

13.6.1 Test Execution Summary (Private Beta)

Testing is continuously executed to provide objective evidence that the Patient Triage API is functionally operational and clinically safe as designed within the defined Private Beta scope.

Testing covers quality, security, dependency, and infrastructure assurance tools built into an automated deployment pipeline (See Appendix A – Continuous Integration Test Coverage). Testing also includes unit, acceptance, integration, performance, and penetration testing. This is supported by smoke testing to confirm baseline system operability following deployment.

Performance testing confirmed that the system operated within defined response time, availability, and error rate thresholds relevant to its intended use, with no performance related behaviours identified that would introduce new or increased clinical safety risk (see Appendix B –Test Evidence).

An independent penetration test was executed to ensure robust cybersecurity controls across three specific areas (see Appendix B – Test Evidence):

1. Web API Penetration Test
2. AWS Technical Security Assessment
3. Secure Codebase Review

One high severity issue identified in the execution report has been resolved, with remaining items either mitigated or managed via agreed actions on the delivery backlog. Review of these findings confirmed that none introduced unacceptable residual clinical safety risk within the defined scope.

The results of executed testing demonstrate that the API does not alter or compromise the behaviour or clinical outputs of the underlying authored content exported from the Pathways Authoring Tool (PAT), and that identified safety controls operate as intended. No new clinical hazards were identified through testing activities.

13.6.2 Appendix

13.6.2.1 Appendix A – Continuous Integration (CI) Test Coverage

The CI quality workflow performs non-functional checks intended to catch security, hygiene, and standards issues early.

These checks include:

- **Secret scanning** to identify committed secrets or sensitive material
- **Dependency scanning** to identify vulnerable or problematic dependencies
- **File format checks** to enforce expected file formatting rules
- **Markdown format checks** to keep documentation standards consistent
- **English usage checks** to improve documentation and content quality
- **Terraform format checks** to ensure infrastructure code is consistently formatted
- **Checkov security scanning** to assess infrastructure-as-code for common security issues

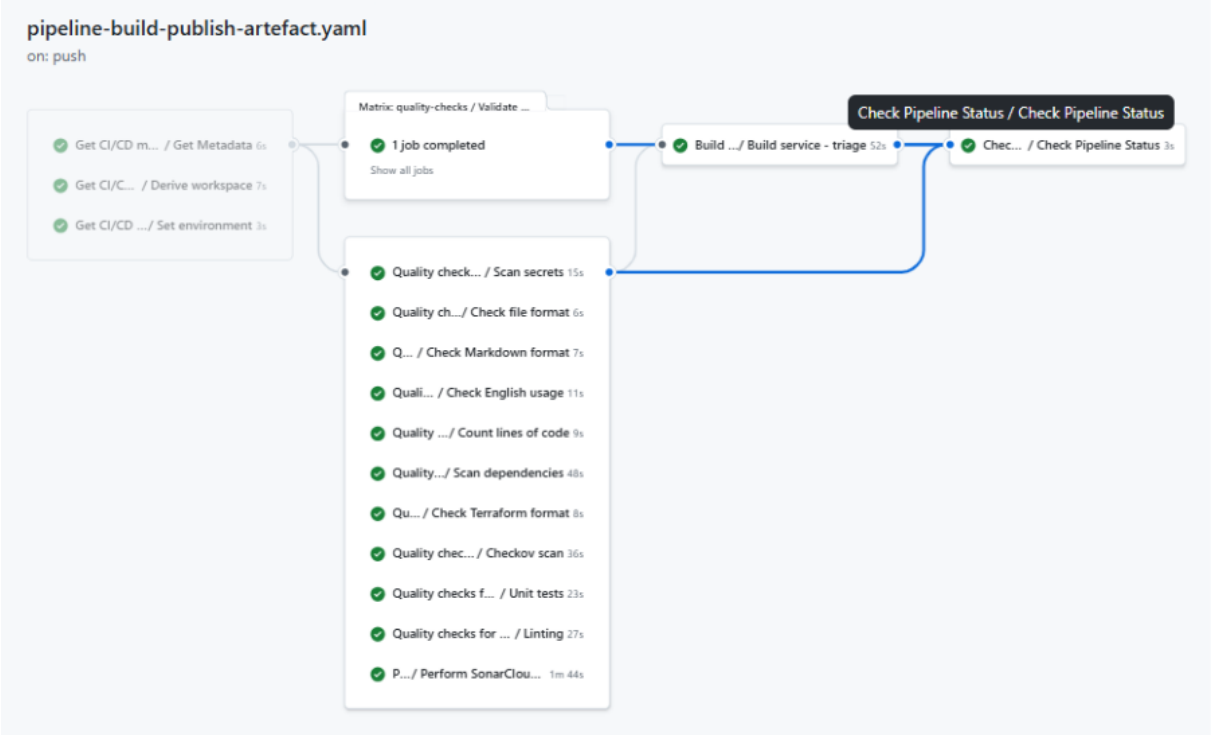
The CI workflow performs the core automated application tests currently enforced by the pipeline.

These include:

- **Unit tests** executed through the project test target
- **Linting** to identify code quality and style issues
- **Coverage testing** to generate test coverage output for analysis

The coverage output is then used as part of the static analysis stage.

Example deployment validation:



13.6.2.2 Appendix B –Test evidence

13.6.2.2.1 Performance test summary

Performance test strategy: https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1336619946/Patient+Triage+Api+-+Performance+Testing+Strategy

Non-functional KPIs: https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1336618331/7.1.6+Key+Performance+Indictors+KPIs

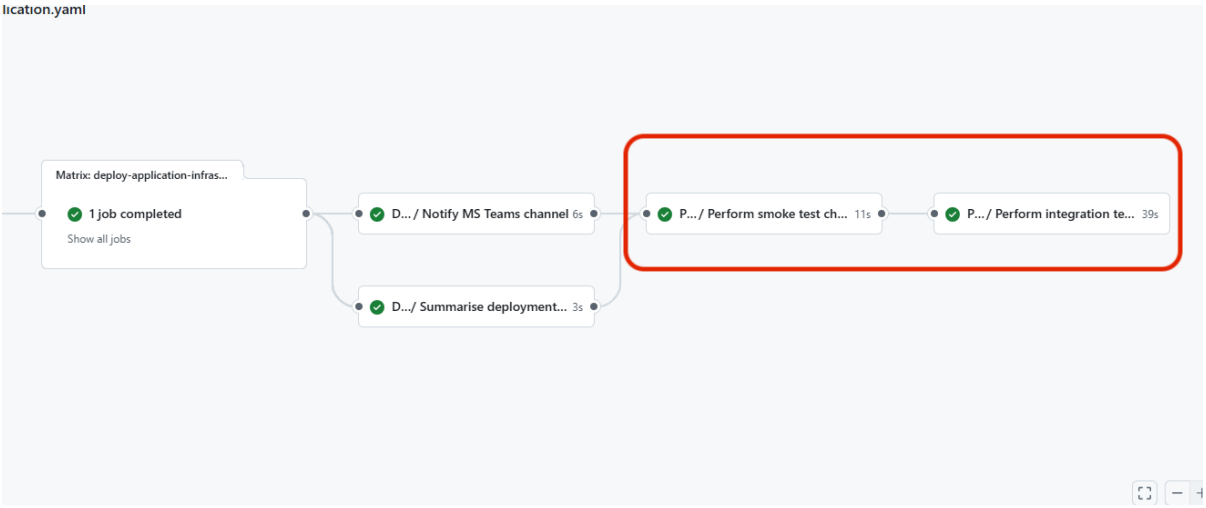
Performance test result and analysis: https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1368820571/Patient+Triage+API+-+Performance+Test+Results+05+05+2026 (see page 182)

13.6.2.2.2 Penetration test summary

Output of issues identified from penetration test execution:

Section	Low	Medium	High	Critical
Web API Penetration Test	1	0	0	0
AWS Technical Security Assessment	2	1	0	0
Secure Codebase Review	3	1	1	0

13.6.2.2.3 Smoke & Integration test summary



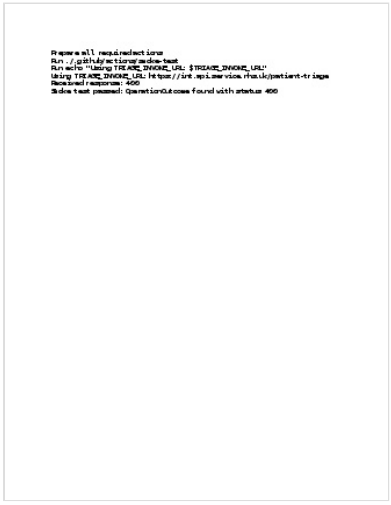
A smoke test run prior to integration testing as part of the automated pipeline depicted above, it covers the following:

- tests that the endpoint is operational

Sample Integration Test Report

Sample test run 12 Jun 2026

Test result PASS



Overview of Integration Testing:

- Runs triage queries through APIM
- Requests are against a sample of a clinical release for 111 Online and the Emergency Prescription pathway
- Requests invoke error scenarios
- Test verifies that the response contains the information expected

Sample Integration Test Report

Sample test run 12 Jun 2026

Test result PASS



13.6.2.2.4 111 Online Testing

Below documents the testing 111 Online have carried out with the Patient Triage API

- End to End testing Triage Dispatcher API End-2-End[71]

- Load testing C1515 Triage Dispatcher API and infrastructure load testing[72]

71. https://nhsd-confluence.digital.nhs.uk/spaces/1O/pages/1386349995/Triage+Dispatcher+API+End-2-End
72. https://nhsd-confluence.digital.nhs.uk/spaces/1O/pages/1373934747/C1515+Triage+Dispatcher+API+and+infrastructure+load+testing

14 Red Lines Assessment

This assessment is based on the red lines as defined on 09 Mar 2026 , on https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-lines.aspx.

14.1 Scope

This assessment covers the Patient Triage API service and ingestion pipeline. Authoring and other services are out of scope.

TODO - Link to Arch diagram.

14.2 SoftEngs Template

TODO - Find out if we need an engineering assessment and what template to use (below is a link to pathways one)

Link to the source on Pathways Sharepoint: Strategy & Process - Red Lines - All Documents[73]

73. https://nhs.sharepoint.com/sites/X26_Pathways/SMT/Shared%20Documents/Forms/AllItems.aspx?id=%2Fsites%2FX26%5FPathways%2FSMT%2FShared%20Documents%2FPathways%5FColleagues%2FProducts%2FRed%20Lines

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Cloud-1	All new services **must** be developed on public cloud services in secure cloud tenants supported by the Cloud COE with: • One set of accounts/subscriptions per product, with separation of production and non-production environments, owned by the product area • Accounts on AWS must be within the AWS Landing Zone[74] in the NHS Digital AWS Organisation • Subscriptions on Azure must be within the NHSE-LZ-Root Management Group structure in the NHS Strategic Azure Tenant Notes: • Supported secure landing pads only exist for Azure and AWS, using **any** other cloud provider for production use cases requires Engineering board approval and substantial amounts of engineering to make them ready for production. • Any proof-of-concept work outside of the supported landing pads on Azure and AWS must be brought for discussion at the Engineering Board in advance of the work starting, and the technologies recorded as "proposed" in the tech radar. • Crown Hosting is not considered "public cloud" in this context so does not satisfy this redline	• Overproduction - building when you could reuse or buy[75] principle • Outsource from the bottom up[76] pattern • Cloud[77] practices • How Cloud-1 and SDLC-6 Red Lines Complement Each Other[78]	Met	Hosted via NHS AWS Landing zone process
Cloud-2	All services **must** be designed as Internet first - HSCN may be provided but only to support legacy connections.		Met	

74. https://nhsd-confluence.digital.nhs.uk/display/AWS/Landing+zone+user+guide
75. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste
76. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/outsource-bottom-up.md
77. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/practices/cloud-services.md
78. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/How-Cloud-1-and-SDLC-6-red-lines-complement-each-other-and-what-is-their-applicability-.aspx

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Cloud-3	All production environments **must** automatically scale down by at least 40% running cost from peak hourly usage to off-peak usage	Inventory — unnecessary resources[79] principle Cloud[80] practices: "services should scale automatically up and down", "infrastructure should always be fully utilised", etc	Met	Prod currently being built. Core stack is serverless.
Cloud-4	Gold and platinum services **must** use zero-downtime blue/ green deployments for all planned deployments	Deployments[81] pattern	Not met N/A	Not yet Gold/ Plat - Planned for. Currently Bronze SLA.
Cloud-5	Disaster recovery: Gold and platinum services **must** test (in isolation) a full restore from production backups of code and data at least every 24 months, i.e. a complete rebuild of the system on a new account, separate to the production system, in order to test that the system can be recreated following scenarios such as total account loss	Test scenarios for cloud-5 and cloud-6[82]	Not met N/A	Not yet Gold/ Plat - Planned for. Currently Bronze SLA.

79. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste
80. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/practices/cloud-services.md
81. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/deployment.md#zero-downtime-deployments
82. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-Lines-Backup-Scenario---Overview.aspx

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Cloud-6	Data recovery: All services **must** have persistent data backed up to an immutable backup service with appropriate policies to prevent deletion, to protect against scenarios such as ransomware attacks, insider attack, etc. Every 12 months, the most recent backup must be tested in order to verify that the production data can be restored and that it is valid & up to date. Please note that immutable backups will not achieve RPO of zero: if that is a requirement for your service, you will need additional measures AWS-based and Azure-based systems **must** use the mandatory blueprints as per cloud backups[83].	Testing: Test scenarios for cloud-5 and cloud-6[84] Solutions: Cloud backups[85]	Not met	Currently in design. Note we need to plan annual tests. Backup & Immutability Strategy[86] Backup Strategy[87]
Cloud-7	Services operating in an active-passive model in production **must** swap active and passive sites at least every 12-months		N/A	Not yet part of plan for multi-region. Possible future need.

83. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/TechRadar-CloudBackups.aspx
84. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-Lines-Backup-Scenario---Overview.aspx
85. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/TechRadar-CloudBackups.aspx
86. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1305319532/Patient+Triage+Api+-+Disaster+Recovery+Strategy#PatientTriageApiDisasterRecoveryStrategy-Backup&ImmutabilityStrategy
87. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1305321143/Patient+Triage+Api+-+Backup+Strategy

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Cloud-8	Any write access by a human to production **must** only be possible via temporary (max 24 hours) elevation of access which **must** be approved by a different human - there should be no standing write access on user's accounts, and no ability for users to self-serve elevated access without approval	Security[88] practices For AWS: CCOE solution is now ready for use[89] For Azure: PIM configuration details to be added	Test	Simple TEAM config set up. Needs testing once envs are deployed. Request Elevated AWS Access[90]

14.2.1 Using Shared Services

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Reuse-1	NHS notify **must** be used for any physical or digital communications for communication to citizens. Currently there is no equivalent service for staff.	Overproduction - building when you could reuse or buy[91] principle	N/A	
Reuse-2	The single source of truth for demographic details **must** be PDS, no address or contact detail overrides can be captured outside of PDS	Overproduction - building when you could reuse or buy[92] principle	N/A	

88. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/practices/security.md#infrastructure-security
89. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Elevated-Access-Management.aspx
90. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1305340843/Request+Elevated+AWS+Access
91. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste
92. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Reuse-3	Citizen facing services requiring user authentication **must** use NHS login authentication – note: Proxy access is a known exception, which is not currently supported by NHS login	Overproduction - building when you could reuse or buy[93] principle	N/A	App restricted API, auth handled by the client.

93. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Reuse-4	If you require IAL3 then you **must** use CIS2, if you don't then you **must** use either CIS2 or NHS.net[94]	Overproduction - building when you could reuse or buy[95] principle **Identity Assurance Level:** Identity Assurance Levels as defined in the NIST Digital Identity Guidelines for Enrollment and Identity Proofing Requirements define the degree of confidence that the person claiming a particular identity is indeed the person to whom the identity was assigned. The NIST guidance categorises IALs into three levels:	N/A	

94. http://NHS.net
95. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste

Ref #	Red line	Related principles / patterns / etc	Status	Notes
		• IAL3: The highest level of assurance, IAL3, requires in-person identity proofing and verification of the individual's identity using multiple pieces of strong evidence by a trained and authorized individual. This level is used for high-risk applications where the consequences of identity misrepresentation are severe. For access to national clinical systems the Identity Assurance Level **MUST** be 3. • IAL2: This level requires evidence of identity and the verification of that evidence. It involves a higher degree of confidence in the claimed identity, often requiring the presentation of government-issued identification and other supporting documents.		

Ref #	Red line	Related principles / patterns / etc	Status	Notes
		• IAL1: At this level there is no requirement to link the applicant to a specific real-world identity. This level is typically used for low-risk applications where the consequences of an identity misrepresentation are minimal.		
Reuse-5	Teams **must** consult with the FDP/CDP teams in Data & Analytics where a data analytics platform is required: this includes when joining multiple large datasets together for analysis. Teams **must not** build additional analytics platforms, though can and should use cloud-native tools such as AWS Athena for simple data analysis.	Overproduction - building when you could reuse or buy[96] principle	N/A	May be scope for this in future phase?
Reuse-6	All national cohorting generation **must** be performed by Cohorting As A Service		N/A	

96. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Reuse-7	When building a citizen-facing or new staff-facing system, you must align with the nhsuk-frontend[97] and NHS Design System[98] to build your product or service user interfaces. Only alter it, extend it or go beyond the scope of the frontend and design system where you have evidenced user or business needs.	If you create new components, you must feed these back to the Design System Community Backlog[99]. Include the needs that informed them, design rationale and research insights. If you need a higher-level toolkit to satisfy user or business needs, use the nhsuk-react-components in line with the UI Working Group decision[100].	N/A	

14.2.2 Software Development Lifecycle

Ref #	Red line	Related principles / patterns / etc	Status	Notes
SDLC-1	All code **must** use the NHS GitHub[101] enterprise for version control and **must** comply with: • Configuration rules: securing-repositories.md[102] • Configuration compliance reporting: engineering dashboards[103] – note this data is collected automatically for organisations within the NHS GitHub enterprise subscription	Overproduction - reinventing the wheel[104] principle Code security[105] Outsource bottom up[106] pattern	Test	Repo hosted on GIthub Need tech team to confirm if met. Only a question around how the engineering dashboard is being displayed

97. https://github.com/nhsuk/nhsuk-frontend
98. https://service-manual.nhs.uk/design-system
99. https://github.com/orgs/nhsuk/projects/21/views/1
100. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/UI-Working-Group---Tool.aspx
101. https://github.com/NHSDigital
102. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/practices/securing-repositories.md#securing-repositories
103. https://dashboarding.engineering.england.nhs.uk/
104. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste

Ref #	Red line	Related principles / patterns / etc	Status	Notes
SDLC-2	Builds of the primary branch **must** use SonarCloud and **must** comply with the 4 Sonar way quality gate conditions and additionally total unit test coverage **must** be over 50%	Accessing Sonarcloud[107] Outsource bottom up[108] pattern	Met	Sonarcloud is enabled. Check if this is met yet.
SDLC-3	All production services **must** be built and deployed to production at least once a quarter even if there is no code change in that period		Not met	Still in build phase.
SDLC-4	All planned deployments to production **must** be automated	Automate everything[109] pattern	Met	
SDLC-5	Services under active development **must** be built daily and must be deployed at least every two weeks (outside of change freeze windows)	Little and often[110] pattern	Met	Assume this is met. What is the boundary of active / inactive development?
SDLC-6	The architect-for-flow[111] pattern **must** be applied: any functional changes relating to any identifiable bounded context (new or existing) **must** be delivered in a loosely-coupled and independently-deployable component specific to that bounded context	• Architect-for-flow[112] pattern • How Cloud-1 and SDLC-6 Red Lines Complement Each Other[113]	Met	

105. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/principles.md#1-eliminate-waste
106. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/outsource-bottom-up.md#the-pattern
107. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Accessing-Sonarcloud.aspx
108. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/outsource-bottom-up.md#the-pattern
109. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/automate-everything.md#automate-everything
110. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/little-and-often.md#the-pattern
111. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/architect-for-flow.md
112. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/patterns/architect-for-flow.md
113. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/How-Cloud-1-and-SDLC-6-red-lines-complement-each-other-and-what-is-their-applicability-.aspx

Ref #	Red line	Related principles / patterns / etc	Status	Notes
SDLC-7	Teams **must** run appropriate vulnerability software and **must** remediate findings within the timeframes set out in cyber standards[114]: • All production code and production cloud accounts for a service **must** use the following mandated vulnerability-management tools: • Dependabot[115] (within the NHSE enterprise) for Dependency scanning • SonarQubeCloud[116] (wit hin the NHSE instance) for Static Application Security Testing of application code • TenableOne[117] (within the NHSE instance) for cloud-account configuration scanning • Compliance means full coverage of the service in the NHSE Engineering Dashboards[118], and no "overdue" findings reported in the dashboards. • If a team cannot make use of the above mandated tooling they **must** seek an exception from the Engineering Board to use equivalent tools: this scenario **must** include providing an automated daily feed of open-vulnerability counts at least daily to the NHSE Engineering Dashboards[119] to support oversight of open vulnerabilities.	Vulnerability Management[120] Tech Radar[121]	Test	Assume this is met, tech team to confirm it. Only a question around how the dashboard is being displayed Only a question around how the engineering dashboard is being displayed

114. https://nhs.sharepoint.com/sites/X24_securitymanagement/Cyber%20Security%20Standards/Forms/AllItems.aspx
115. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Dependabot-Management.aspx
116. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Sonarqube-Cloud-Management.aspx
117. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Tenable-is-ready-to-use.aspx
118. https://dashboarding.engineering.england.nhs.uk/service-metrics
119. https://dashboarding.engineering.england.nhs.uk/service-metrics
120. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/TechRadar-VulnerabilityManagement.aspx

14.2.3 Software Management

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Soft-1	Externally facing (to the team) APIs **must** follow the published API guidance	<< this one needs adjusting API Policies and Best Practice[122] What Good Looks Like[123]	Met	
Soft-2	All externally-facing APIs **must** use the Digital Onboarding Service (DOS)[124] so no SCALs will be used for onboarding.		Not met	Planned for

121. https://radar.engineering.england.nhs.uk/
122. https://digital.nhs.uk/developer/guides-and-documentation/api-policies-and-best-practice
123. https://nhsd-confluence.digital.nhs.uk/display/APM/What+Good+Looks+Like+checklist
124. https://digital.nhs.uk/developer/guides-and-documentation/digital-onboarding

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Soft-3	All software and dependencies should be recorded in a Software Bill of Materials. Software that is marked as being "End of Life" (or otherwise to indicate that it will not receive security updates) **must** not be used in Production systems and **must** be removed. For software which declares a version support policy, only those versions which are receiving security updates are permitted to be used, with a preference for the most recent stable version. For software which does not declare a version support policy, the version in use **must** be updated no less than once every 6 months, and updated to the most recent stable version. It **must** be removed if it is no longer being supported. Patches released for all vulnerabilities **must** be applied in line with Cyber Vulnerability Management standards. Teams can consult with their Cyber Consultants to discuss specific cases.	Teams should ensure they build a Software Bill of Materials through the central approach documented here[125] and contribute to this approach if applicable.	Test	Part of the MDD work. Check with Ali / Andrew if met.
Soft-4	Teams **must** use the maturity review model with the software quality framework at least once a quarter	Team self-assessment maturity model[126]	Not met	We haven't done one of these yet. Once a quarter feels a bit much..?

125. https://github.com/NHSDigital/software-engineering-quality-framework/tree/main/tools/dependency-scan#dependency-scan
126. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/insights/review.md

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Soft-5	Teams **must** align with the Tech Radar[127] • If you're currently using or evaluating any technologies which are missing from the radar, you **must** follow the adding-technologies-to-the-radar process[128] • If you're currently using any AVOID technologies, you **must** have an active migration plan to move away from those technologies within the timeframe specified in the radar • CONTAIN, PROPOSED, or AVOID technologies **must** have prior approval from the Engineering Board to be introduced to production or used to build, deployment, or operation of a live system	Tech Radar[129]	Met	
Soft-6	Teams **must** comply with all mandatory blueprints in the engineering framework, within 6 months of the blueprint being mandated by the engineering board	Blueprints [130]	Test	Assume this is met, check with tech team.
Soft-7	Production services **must not** have any dependencies on services/components or versions of APIs that are within 6 months of being retired, unless the production service itself is due to be retired first		Met	

127. https://radar.engineering.england.nhs.uk/index.html
128. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Radar.aspx#adding-technologies-to-the-radar
129. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Radar.aspx
130. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Blueprints.aspx

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Soft-8	Production services must not have any network dependencies (e.g. communication with an API) relying on security protocols that are within 6 months of being retired by that dependency (e.g. changes to supported TLS versions), unless the production service itself is due to be retired first		Met	

14.2.4 Team-Specific Red Lines

The following red lines are vetted by the ETA but only apply within the specific team indicated.

14.2.4.1 Spine

Ref #	Red line	Related principles / patterns / etc	Status	Notes
Spine-1	The Riak database type is deprecated and **must** be retired in Spine by end of 2027	Tech Radar[131]	N/A	

131. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Radar.aspx

15 Triage Engine API Alpha

- Architectural approach to minimise latency in the engine (see page 254)
- Conceptual Design and Data Requirements (see page 260)
- Triage API architecture review (see page 274)
- Triage API Alpha goals and success criteria (see page 289)
- Triage API Run cost estimate - pre-Alpha (see page 290)
- Future infrastructure and role permissions (see page 293)
- Failure Scenarios (see page 300)
- Service Map (see page 302)
- APIM FMEA (see page 303)

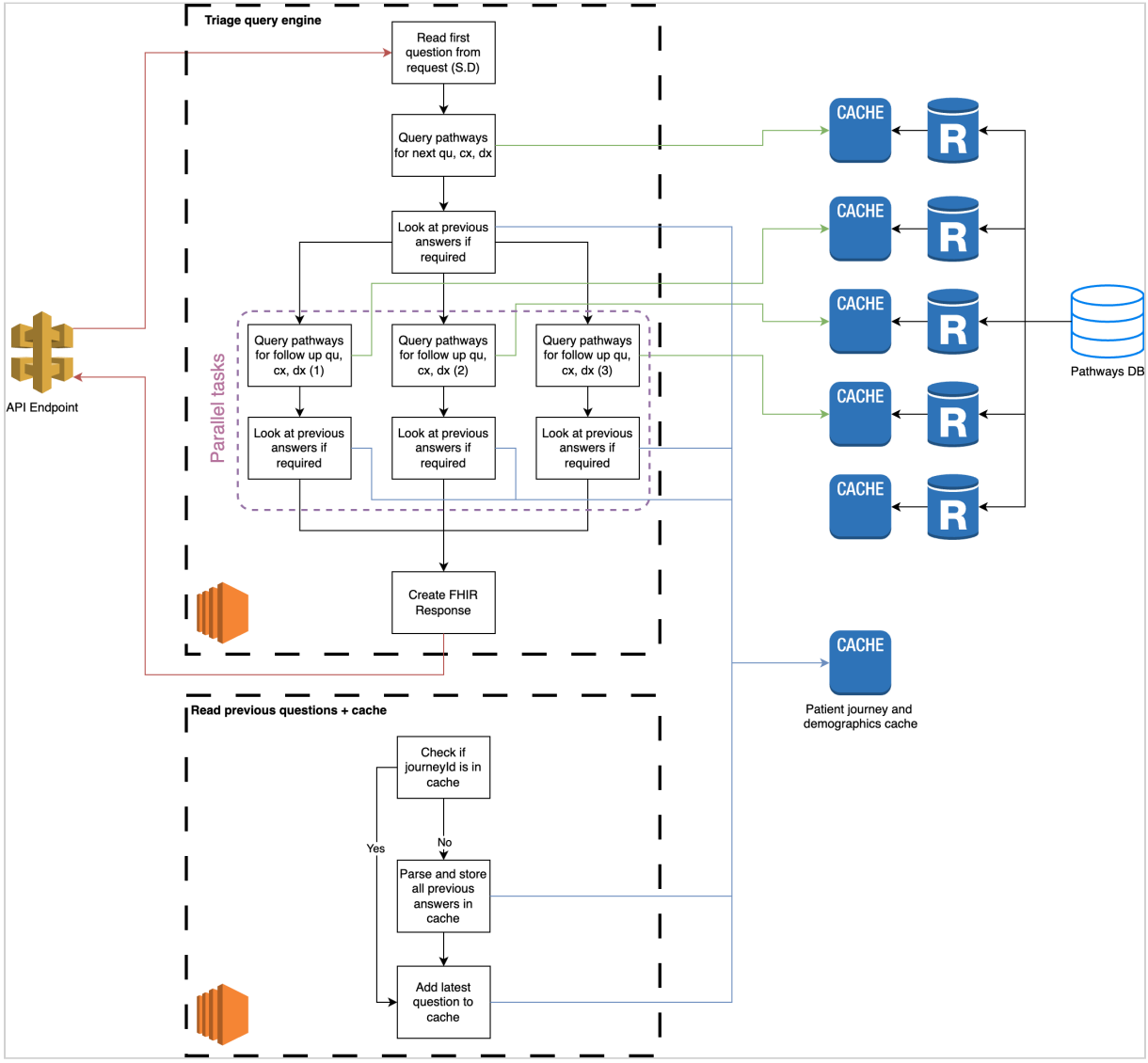
15.1 Architectural approach to minimise latency in the engine

Related ticket: NPT-443[132] - API: Architectural approach to minimise latency in the engine DONE

15.1.1 Overview of potential architecture

The following diagram shows the potential architecture required for implementing the low latency approaches outlined below. This diagram is not a complete architecture of the system but only demonstrates the architecture specific components to reduce latency, e.g. Database with read replicas, cache and separate server instances to handle different tasks.

132. https://nhsd-jira.digital.nhs.uk/browse/NPT-443?src=confmacro



15.1.2 Deconstructing the request

15.1.2.1 High performance library for JSON parsing

For parsing the JSON from the request, we need to use a high performance JSON parsing library.

15.1.2.1.1 For Java

If we do not use any streaming deserialisation to read the first question (not the whole message), Jsoniter would be the best to use. If we do use stream deserialisation, we will have to use Jackson which is capable of both and offers good performance.

	Performance*	Streaming deserialisation

Jackson	Good	Yes
Jsonitor	Better	No

*based on these benchmark tests as a sample (benchmark results as example[133])

15.1.2.1.2 For JavaScript

The native JSON library is good for smaller datasets, but Fast JSON Parse offers the best performance of the three and is designed for larger datasets. If we need streaming deserialisation for our approach, JSONStream is the only one we can use for it.

	Performance	**Reliability**	**Lightweight**	**Streaming deserialisation**
Native JavaScript	Good	Part of JS engine	Native	No
Fast JSON Parse	Best	Reliable, 3rd party library	Lightweight but added overhead	No
JSONStream	Slowest of the 3	Reliable, widely used with node.js	Lightweight for streaming functionality	Yes

15.1.2.1.3 For C#

System.Text.Json and Newtonsoft.Json are both good and widely used options for reading JSON in .NET applications. Utf8Json offers extremely fast and optimised performance, which maybe more appropriate that System.Text.Json and Newtonsoft.Json. All support streaming deserialisation so is not a differentiator.

	Performance	**Reliability**	**Lightweight**	**Streaming deserialisation**
System.Text.Json	Good	Part of .NET framework	Native	Yes
Newtonsoft.Json	Better	Very reliable, widely used	Lightweight	Yes

133. https://github.com/noboomu/java-json-benchmark

Utf8Json	Best	Reliable, less widely used	Lightweight	Yes

15.1.2.1.4 Summary

There will be a need for high performance parsing of JSON regardless of approach, therefore the fastest libraries should be selected. If streaming deserialisation is used in a separate async task, libraries that support it could be used along side the high performance libraries.

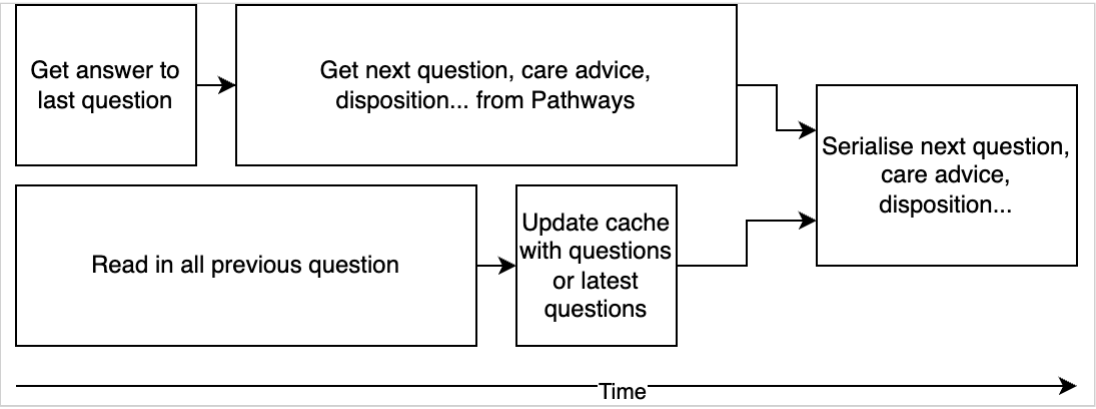
However, it is likely that the payloads, even the largest expected payloads, will not be big enough that deserialisation performance is critical. Microbenchmarking will need to be run to confirm if parsing these payloads will require a significant enough effort to warrant considering streaming deserialisation in our design.

	High performance	**Streaming deserialisation**
Java	Jsonitor	Jackson
JS	Fast JSON Parse	JSONStream
C#	Utf8Json	Utf8Json

15.1.2.2 Split up parsing task and do part asynchronously

After we complete the first exchange with the triage engine and return the first question, we expect to read the last question answered every time. With stronger rules for the request information order, we could make it quicker to parse the first question and query the engine for the next question. While this is happening, the process of parsing all the previous answers to questions could be done asynchronously (this could be a very large number of questions).

If the request is very large, the next question/step may already be returned before finishing parsing the full request. This may be a problem if the engine needs to check a previous answer as the engine will have to wait for the parsing to finish.



15.1.2.3 Cache some expected request information

We could also store the patient demographics, variables, previous questions and answers in a cache with an associated journeyId, in a write through approach to minimise the latency of all transactions with the cache. The triage engine would just add the last question and answer to it's journeyId in the cache. This could reduce the amount of time we spend parsing the requests for information. We could minimise the mount of cache data required with a smaller TTL if we don't expect to need to hold on to the cache data for longer than the triage itself.

Potential downsides include if the user updates the information in the request, e.g. goes back to a previous question and changes the answer, the cache would be out of date. This could be mitigated by making it required to send the actionId in the request to tell the API if the previous questions has been changed, gone back etc.

15.1.3 Triage engine optimisation

15.1.3.1 Skillset entry path lookup

At the start of the triage process, the skillset of the client is used to get the first pathway and start point in that pathway for the start of the triage. For the skillsets listed below, the entry pathway id could be set to "AgeGenderParty" and the entry order number could be determined based on if it's 999 or 111/OOH. This would save calling the database at the start of a triage and the TBranch value could be queried when it is needed. This may be undesirable if these are likely to change for these skillsets as it would require a code change rather than just releasing and update of the pathways data to the triage engine.

Entry Pathway ID	Entry Order Number	Skillset Description
AgeGenderParty	5	OOH Call Handler
AgeGenderParty	5	OOH Clinician
AgeGenderParty	0	999 Call Handler
AgeGenderParty	0	999 Clinician
AgeGenderParty	5	111 Call Handler
AgeGenderParty	5	111 Clinician
AgeGenderParty	5	111 Online

15.1.3.2 Pathways database read optimisation

15.1.3.2.1 Pathways in memory

We will load all the pathways data into memory. We will read the pathways data from memory when actually processing a request. This will mean we are not waiting for a database transaction to return to query the triage engine, making it very fast to get information from the pathways data. This would likely make using read replicas in combination with in memory pathways redundant.

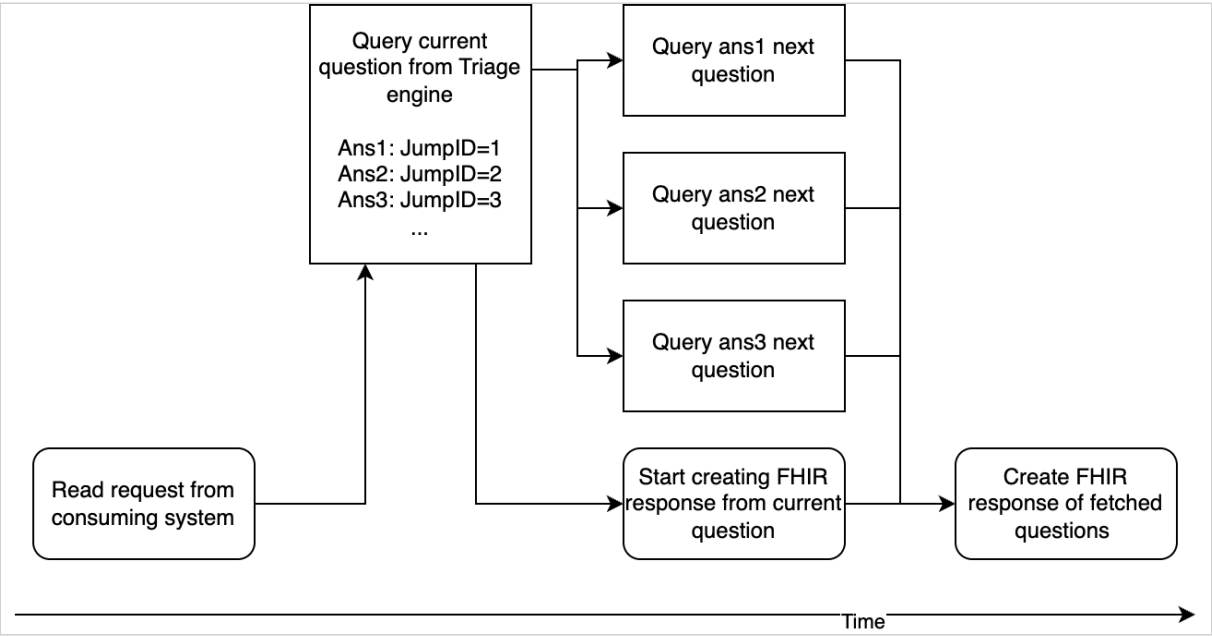
15.1.3.2.2 Read replicas

The triage engine will only need to do reads from the pathways database, hence it is crucial to optimise the engine's ability to read from the database. Read replicas of the source database will reduce the load on the source database. One potential issue could be with read replicas being asynchronous with the source database. This means some database reads could be out of date if there is a new release if they read from a read replica. It would be necessary to wait/force the read replicas to update when a new release is pushed to the pathways database.

15.1.3.3 Fetching the next set of questions ahead of the current question (N+1 Solution)

It's may be necessary to fetch and return the next potential next questions and outcomes along with the current question. This would allow the consuming system to proceed to the next question for the user while sending a request to get the next question + next potential questions after that in the background. There would be little/no wait time for the next question from the user perspective.

To achieve this, we can fetch the potential next questions based on the jump ID of the answers of the current question to be returned. The process for this could be sped up by doing this operation in parallel, i.e. fetch the next question based on the jumpIds of the current question's answers. The timing diagram below shows how this could be executed by the engine.



15.1.4 Constructing the response

15.1.4.1 Template based generation/pre constructed JSON

For constructing the JSON FHIR message response, much of the JSON can be pre constructed. The engine would just need to copy the relevant JSON and set the relevant values

The query form the pathways database could be fed into the FHIR JSON constructor and return the JSON to be sent in the response

15.1.4.2 Parallel JSON construction

The body map has a very large JSON data size (69.5 kB minified). One way this could be mitigated it creating the JSON for each clinicalArea in parallel, then combining them into one JSON object, before adding them to the rest of the FHIR JSON resource

15.2 Conceptual Design and Data Requirements

NPT-441[134] - API: Triage engine conceptual design and data requirements DONE

15.2.1 In-memory OOP approach

15.2.1.1 Loading up the data into memory

We'll need functions to work out how to start triage. In the first case this is a map from skillsets to a node.

`entryPathway(skillset : string): NodeReference`

where

```
NodeReference {
  PathwayID : string
  OrderNumber : integer
}
```

If `NodeReference.PathwayID == "AgeGenderParty"`, a further function is needed to get the starting node according to patient demographics.

`startingNode(skillset : string, ageInHours : integer, gender : string, party : string) : StartingNode`

where

134. https://nhsd-jira.digital.nhs.uk/browse/NPT-441?src=confmacro

```
StartingNode {
  Tbranch : string
  EntryNode : NodeReference
}
```

To find the first question, the pathways, questions and answers must be in memory.

```
Pathway {
  ID : string
  Nodes : set<Question>
}
```

```
Question {
  OrderNumber : integer
  ID : string
  Text : string
  Rationale : string
  Answers : list<Answer>
}
```

```
Answer {
  Text : string
  Rationale : string
  Target : NodeReference
}
```

1. find `Pathway p` such that `p.ID == StartingNode.EntryNode.PathwayID`
2. find `Question q` in `p.Nodes` such that `q.OrderNumber == StartingNode.EntryNode.OrderNumber`
3. supply a view of the question with `q.Id` , `q.Text` , `q.Rationale` , and for each answer `a` its index in the list, `a.Text` and `a.Rationale`

The client can then request the second question by also supplying `q.Id` and the index of `a` .

In this case, the server processes the new request as before, finding the first question, but sees that the `q.Id` matches that provided in the request, finds the `NodeReference` in the selected answer's `Target` and queries the pathway as before to supply the new question.

Not all nodes will be questions though. Some are used to set variables, or to redirect according to the value of variables.

Thus `Question` is one of several kinds of Node

```
abstract TriageNode {
  OrderNumber : integer
}
```

```
QuestionNode : TriageNode {
  ID : string
  Text : string
  Rationale : string
  Answers : list<Answer>
}
```

```
SetVariableNode : TriageNode {
  Variable : string
  Value : string
  Target : NodeReference
}
```

```
ReadVariableNode : TriageNode {
  Variable : string
  Cases : list<Predicate, NodeReference>
  Default : NodeReference
}
```

```
CareAdviceNode : TriageNode {
  Advice : list<string>
  Next : NodeReference
}
```

```
DispositionNode : TriageNode {
  Code : string
  Text : string
  Next : NodeReference
}
```

Along with jumps within a pathway and to a node in another pathway, we may also have jumps: to the body map, to the 1st or 3rd party safety pathways, back again from either of those, to the care advice interface, or to end the triage. Worth noting, an explicit instruction to jump to the care advice interface also ends the triage. The "end triage" jump seems to be used to prevent the care advice interface from being shown.

Thus, `NodeReference` should actually be just one type of navigation target:

```
abstract NavigationTarget {}
```

```
NodeReference : NavigationTarget {
  PathwayID : string
  OrderNumber : integer
  ToString() : string => "{PathwayID}.{OrderNumber}"
}
```

```
SafetyPathwayJump : NavigationTarget {
  Party : string
  ReturnTo : NodeReference
}
```

```
SafetyPathwayReturn : NavigationTarget {}
```

```
UserInterfaceJump : NavigationTarget {
  Target : UI
}
```

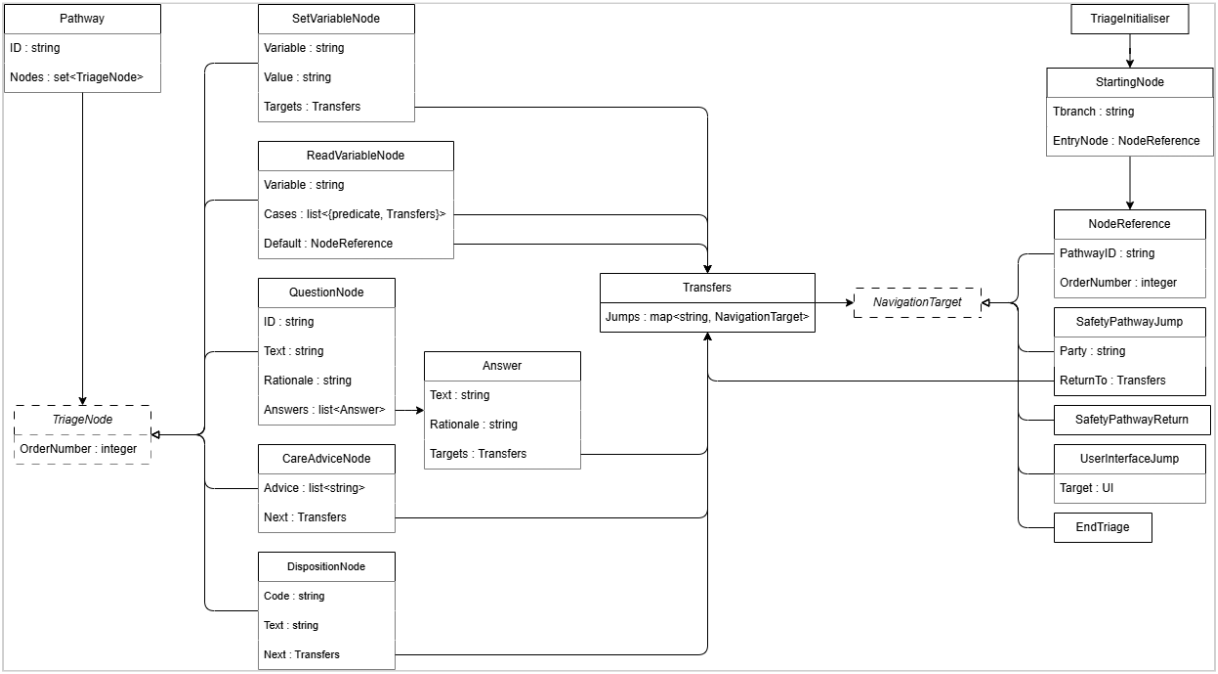
```
EndTriage : NavigationTarget {}
```

```
enum UI {
  BodyMap
  CareAdvice
  PaCCS
}
```

Finally, we must accommodate the 4 transfer categories determined by skillsets.

```
Transfers {
  Jumps : map<string, NavigationTarget>
}
```

Yielding an initial design like:



Taking the most recent release files at the time of writing (48.2.2), the provenance of these data is:

	Pathway.ID	pathways.csv : PW_ID
	Pathway.Nodes	(see the TriageNodes)
TriageNode	**TriageNode.OrderNumber**	triagelogic.csv : OrderNo
	SetVariableNode.Variable	TxQuestions.csv : QuestionText (where ControlType is "3")
	SetVariableNode.Value	TxAnswers.csv : AnswerText (where QuID matches that in TxQuestions.csv)
	SetVariableNode.Targets	triagelogic.csv : JumpID, T1, T2, T3, T4
	ReadVariableNode.Variable	TxQuestions.csv : QuestionText (where ControlType is "4")
	ReadVariableNode.Cases.Predicate	TxAnswers.csv : AnswerText (where QuID matches that in TxQuestions.csv), excluding "else"
	ReadVariableNode.Cases.Transfers	triagelogic.csv : JumpID, T1, T2, T3, T4
	ReadVariableNode.Default	triagelogic.csv : JumpID, T1, T2, T3, T4 (where AnswerText in TxAnswers.csv is "else")

	QuestionNode.ID	TxQuestions.csv : QuID
	QuestionNode.Text	TxQuestions.csv : QuestionText
	QuestionNode.Rationale	TxQuestions.csv : Qu_Rationale
	QuestionNode.Answers	(see Answer)
	CareAdviceNode.Advice	CareAdviceItems.csv : CxText
	CareAdviceNode.Next	triagelogic.csv : JumpID, T1, T2, T3, T4
	DispositionNode.Code	dispositions.csv : DispoCode
	DispositionNode.Text	dispositions.csv : DispoText
	DispositionNode.Next	triagelogic.csv : JumpID, T1, T2, T3, T4
	Answer.Text	TxAnswers.csv : AnswerText
	Answer.Rationale	TxAnswers.csv : AnswerRationale
NavigationTarget	**NodeReference.PathwayID**	triagelogic.csv : JumpID, T1, T2, T3, T4
	NodeReference.OrderNumber	triagelogic.csv : JumpID, T1, T2, T3, T4
	SafetyPathwayJump.Party	triagelogic.csv : JumpID (-3 means 3rd party, -6 means 1st party)
	SafetyPathwayJump.ReturnTo	triagelogic.csv : T1, T2, T3, T4
	UserInterfaceJump	triagelogic.csv : JumpID (-2 means body map, -5 means care advice)

15.2.1.2 Processing requests

To start a triage, we need to look up where in the pathways to begin. The `TriageInitialiser` will have this responsibility, as described earlier.

```
TriageInitialiser {
  - lookupTables
  entryPathway(skillset : string) : StartingNode
  startingNode(skillset : string, ageInHours : integer, gender : string, party :
string) : StartingNode
}
```

(Note: the `entryPathway()` call may be omitted.)

It may be sensible to have an object acting as the state of knowledge about the triage as the engine is processing it, mirroring the tech spec's `TriageRecord` . If so, the `StartingNode.Tbranch` needs to be recorded, and the "current" node set to the `StartingNode.EntryNode` (as a `NavigationTarget`).

```
doTriage(skillset, ageInHours, gender, party):
  triageRecord <- new TriageRecord()
  startingNode <- triageInitialiser.startingNode(skillset, ageInHours, gender, party)
  triageRecord.currentNode <- startingNode.entryNode
  triageRecord.transferPolicy <- startingNode.tbranch
```

The engine then enters a loop, examining the current node and the incoming request to see if it can automatically progress further, or if it needs to return details of what it has discovered so far.

Each kind of node requires different things of the engine.

```
1  doTriage(skillset, ageInHours, gender, party, answers):
2    triageRecord <- new TriageRecord()
3    startingNode <- triageInitialiser.startingNode(skillset, ageInHours,
   gender, party)
4    triageRecord.currentNode <- startingNode.entryNode
5    triageRecord.transferPolicy <- startingNode.tbranch
6
7    payload <- new TriageResult
8    repeat:
9      switch on triageRecord.currentNode:
10       when a NodeReference n:
11         triageNode <- pathways[n.pathwayId][n.orderNumber]
12
13         switch on triageNode:
14           when a QuestionNode q:
15             if q.ID is in answers:
16               answerNum <- answers[q.ID]
17               triageRecord.currentNode <- q.answers[answerNum]
18               continue
19             else:
20               payload.question <- q
21               return payload
22
23           when a SetVariableNode s:
```

```
            triageRecord.variables[s.variable] <- s.value
            triageRecord.currentNode <- s.target
            continue

          when a ReadVariableNode r:
            value <- triageRecord.variables[r.variable]
            matched <- false
            for each {predicate, target} in r.cases:
              if predicate.test(value):
                triageRecord.currentNode <- target
                matched <- true
                break
            if not matched:
              triageRecord.currentNode <- r.default
            continue

          when a CareAdviceNode c:
            payload.careAdvice <- c.advice
            triageRecord.currentNode <- c.next
            continue

          when a DispositionNode d:
            payload.disposition.code <- d.disposition.code
            payload.disposition.text <- d.disposition.text
            triageRecord.currentNode <- d.next
            continue

      when a SafetyPathwayJump s:
        triageRecord.safetyReturn <- s.returnTo
        triageRecord.currentNode <- safetyPathways[s.party]
        continue

      when a SafetyPathwayReturn s:
        triageRecord.currentNode <- triageRecord.safetyReturn
        continue

      when a UserInterfaceJump u:
        switch on u.target:
          when BodyMap:
            return new BodyMapTriageResult
          when CareAdvice:
            careAdvice <- retrieveFinalCareAdvice()
            payload.careAdvice <- careAdvice
            return payload
          when PaCCS:
            // no idea yet!
```

15.2.2 Off-the-shelf neo4j approach

Clarification: neo4j is "off-the-shelf", but the triage engine and its representation within neo4j would still need to be designed and built.

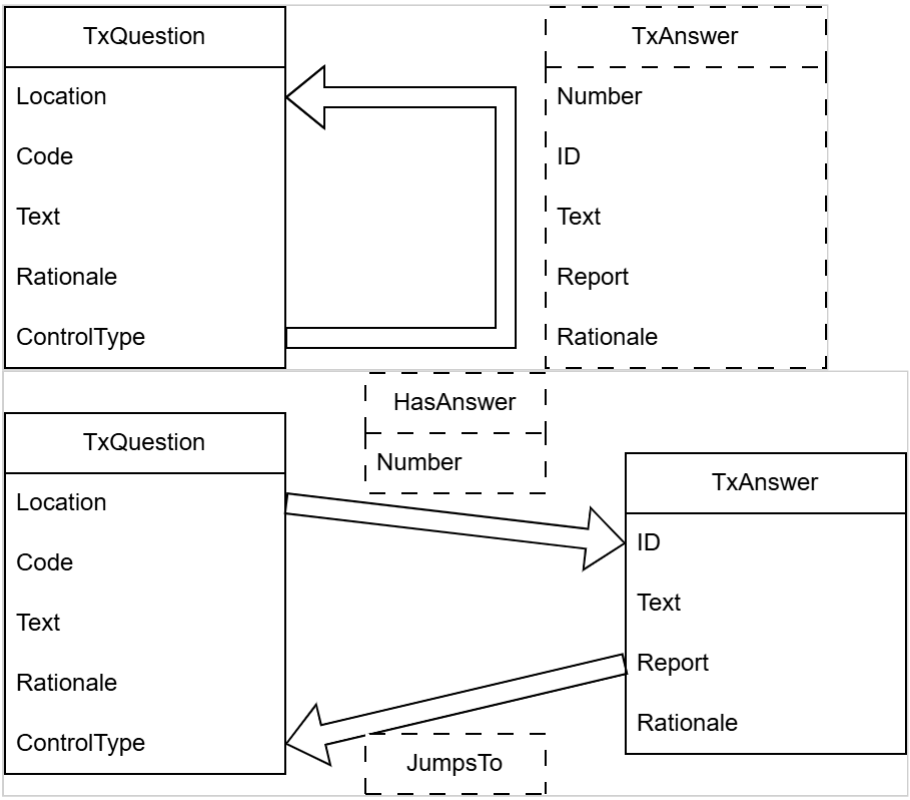
15.2.2.1 Loading the data into neo4j

The graphical structure in neo4j could closely match that shown in VPAT. This implies an overloaded "txQuestion" type whose function is determined by its control-type.

E.g.

```
CREATE (:TxQuestion {
  Location: 'PW888.30400',
  Code: 'Tx100001',
  Text: 'Can you see an object in their mouth?',
  Rationale: "To find out if an object can be seen in the individual's mouth.",
  ControlType: 1
})
```

Answers could either be nodes of their own, relations between questions, or have content across both.



E.g., with content across both:

```
CREATE (:TxAnswer {
  ID: 4541,
  Text: 'yes',
  Report: 'An object was visible in the mouth.',
  Rationale: "This means that an object is clearly visible in the individual's mouth."
})
```

```
MATCH (q:TxQuestion { Location: 'PW888.30400' }), (a:TxAnswer { ID: 4541 })
CREATE (q)-[:HasAnswer { Number: 1 }]->(a)
```

```
MATCH (a:TxAnswer { ID: 4541 }), (q:TxQuestion { Location: PW888.30500 })
CREATE (a)-[:JumpsTo]->(q)
```

15.2.2.2 Processing requests

The 111O approach involves communicating the pathway and order number of questions directly, allowing for immediate querying of the targets neighbours. As we intend to communicate the Q&A history instead, our approach will need to navigate the graph following all the answers provided. As not all nodes will be questions that could have answers supplied, we cannot generate a single query that will apply the provided answers in sequence. It may however be possible to apply *as much as possible* until an unanswered question is encountered.

```
OPTIONAL MATCH (q1:TxQuestion { Code: 'tx1' })
OPTIONAL MATCH (q1)-[:HasAnswer { Number: 7 }]->()-[:JumpsTo]->(q2:TxQuestion { Code:
'tx2' })
OPTIONAL MATCH (q2)-[:HasAnswer { Number: 2 }]->()-[:JumpsTo]->(q3:TxQuestion { Code:
'tx3' })
OPTIONAL MATCH (q3)-[:HasAnswer { Number: 3 }]->()-[:JumpsTo]->(q4:TxQuestion)
OPTIONAL MATCH (q4)-[option:HasAnswer]->(ans:TxAnswer)
RETURN q1, q2, q3, q4, option, ans
```

With this example, if a non-user-question node is actually between `tx2` and `tx3` then `q1` and `q2` will be returned, and the engine can process the instruction before picking up where it left off. This would reduce the number of queries substantially.

The process for establishing the next step in the triage then involves maintaining a farthest node along the triage, and repeatedly hitting neo4j with a query like the one above to retrieve instructions which may need to be applied - updating the farthest known node as it goes - until we detect an unanswered question or the end of triage.

15.2.3 Lambda with DynamoDB approach

From discussion with AWS on 28 Jul 2025 a third approach was posited with specific technologies and it was agreed that our first internal release will target this architecture for benchmarking and analysis.

15.2.3.1 Loading up the data into DynamoDB

When a pathways release is published, the archive (e.g., `NHSPathways48.2.2ReleaseData.zip`) can be placed into an S3 bucket, triggering an event `s3:ObjectCreated:*`. A lambda can be

triggered from this event to extract the archive, process the CSV files inside, and publish the data into DynamoDB in a consumable structure.

The bulk of the work is inspecting particular nodes of the graph, so it seems sensible to have a partition key of the node coordinate. We could have columns for each other field:

Coord	QID	QTxt	QRsn	ANum	ATxt	ARsn	Ctrl	Flag	Jump	T1	T2	T3	T4
PFC3.5	TX220074	What is the reason for the contact?	To find out the reason for the contact and where to send it.	1	an injury or other health problem	This means a new health problem or any existing problem that has worsened. An increase in the use of medication may hide an existing problem that is worsening. This means an individual who has swallowed or breathed in something potentially poisonous but is symptom-free. This does not mean an individual who is symptom-free but requires information.	1		1400				
PFC3.5	TX220074	What is the reason for the contact?	To find out the reason for the contact and where to send it.	2	labour or childbirth	This means waters breaking, increasingly frequent contractions, seeing or feeling the baby coming. This also means that a baby has just been born. DO NOT SELECT THIS ANSWER if the individual has gone into labour following an injury to the abdomen, lower back, groin or genitals. Select 'an injury or other health problem' instead.	1		1000				

Coord	QID	QTxt	QRsn	ANum	ATxt	ARsn	Ctrl	Flag	Jump	T1	T2	T3	T4
PFC3.1400	TX221788	STRING:to'ImmediateThreatsToLife(SPA/OOHversion)'-'ABC'menu		1	present				-1	PA128.0	PA128.0	PA128.0	PA128.0

but this leads to extra duplication (e.g. of question content), where a more simple coord to JSON affords more flexibility.

Coord	Data
PFC3.5	`{"QID":"TX220074","QTxt":"What is the reason for the contact?","QRsn":"To find out the reason for the contact and where to send it.","Ctrl":1,"A":[{"Txt":"an injury or other problem","Rsn":"This means a new health problem or any existing problem that has worsened.\nAn increase in the use of medication may hide an existing problem that is worsening.\nThis means an individual who has swallowed or breathed in something potentially poisonous but is symptom-free.\nThis does not mean an individual who is symptom-free but requires information.","Jump":1400},{"Txt":"labour or childbirth","Rsn":"This means waters breaking, increasingly frequent contractions, seeing or feeling the baby coming. This also means that a baby has just been born. DO NOT SELECT THIS ANSWER if the individual has gone into labour following an injury to the abdomen, lower back, groin or genitals. Select 'an injury or other health problem' instead.","Jump":1000}]}`
PFC3.1000	`{"QID":"TX221408","QTxt":"STRING:LabourOrChildbirth","Ctrl":3,"A":[{"Txt":"present","Jump":1400}]}`

Coord	Data
PFC3.1400	`{"QID":"TX221788","QTxt":"STRING:to'ImmediateThreatsToLife(SPA/OOHversion)'-'ABC'menu","Ctrl":3,"A":[{"Txt":"present","Jump":-1,"T1":"PA128.0","T2":"PA128.0","T3":"PA128.0","T4":"PA128.0"}]}`

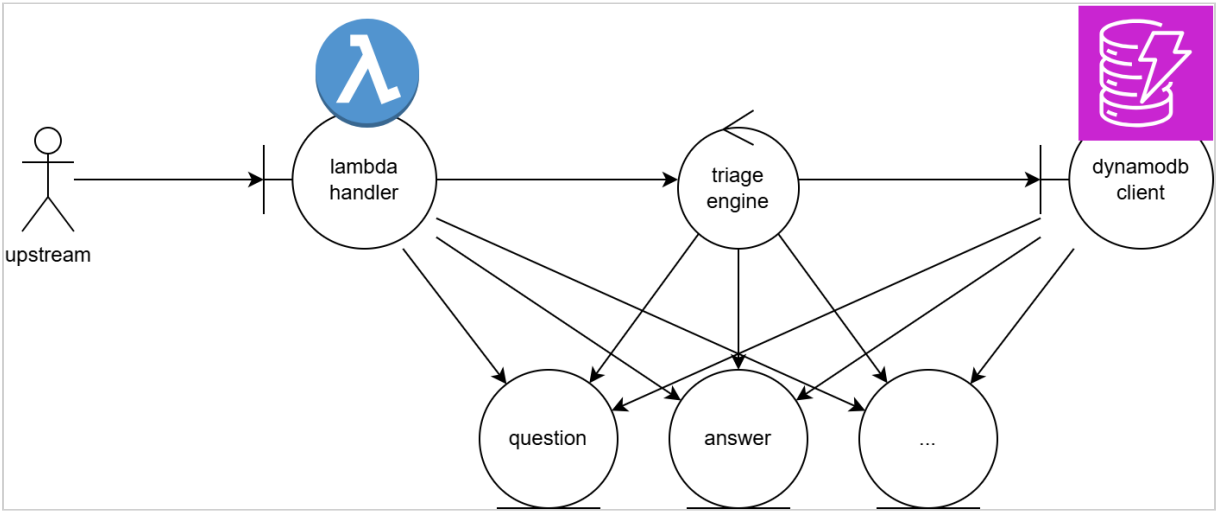
We also need to know where to start. The inputs to establish the first node of a triage are the age, gender and person of the patient, and the skillset of the operator. With 5 age categories, 2 genders (unknown defaulting to female), 2 persons (1st and 3rd) and 16 skillsets, we can fully enumerate the starting nodes with 5*2*2*16=320 rows. Though it could be further optimised, at this scale there's not much point. Using the skillset as the partition key will work.

skillset	age	gender	person	coord	branch
111 Clinician	Adult	Female	1st party	PFA1.5	T4
Dental	Child	Male	3rd party	PW1809.0	T3
...					

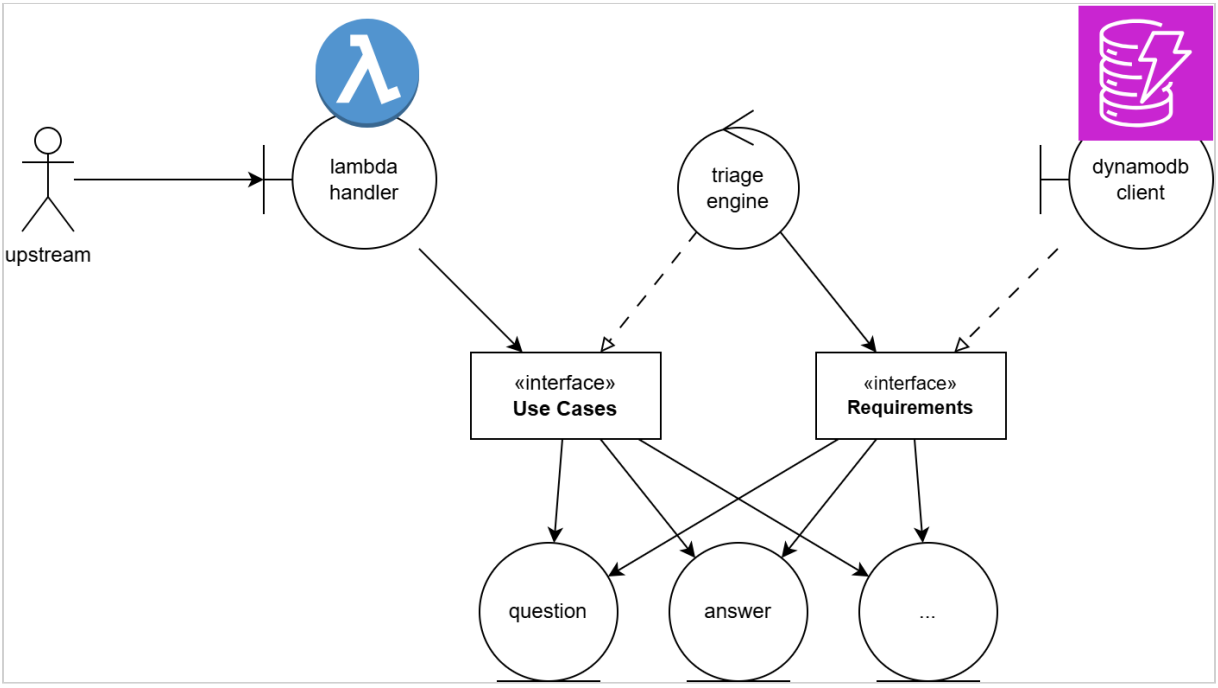
Other tables will also be needed but the principle will be to process the pathways data into a form where we get what we need from a single query, inasmuch as is possible.

15.2.3.2 Processing requests

The important part of processing requests in this design, particularly as we keep in mind that some technology choices may change after we've run some performance analysis, is to keep a clear separation of concerns and to consider carefully what the concerns are.



Rather than coupling the triage engine to lambda and dynamodb, we will apply a lightweight ports-and-adapters approach to separate business logic from infrastructure and allow plug-and-play changes in the future.



The triage engine's responsibility will be to interpret the contents of a node and decide whether a new node needs to be read or if we should return data to the user.

1. Upstream sends a FHIR message which is received, eventually, by the lambda
2. Lambda handler parses this message and invokes the use case
3. Triage engine implements the use case and receives that invocation
4. Triage engine invokes Requirements asking for the starting node, given the contents of the message
5. DynamoDB client implements the Requirements and receives it
6. DynamoDB client sends a request to a DynamoDB table holding the 320 starting nodes

7. DynamoDB returns the relevant row
8. DynamoDB client parses the response and maps it to the structure defined by the Requirements interface and returns it
9. Triage engine invokes Requirements asking for the node defined in the previous response
10. DynamoDB client sends a request to a DynamoDB table holding the triage logic for the node at the specified coordinate
11. DynamoDB returns the relevant row
12. DynamoDB client parses the response and maps it to the structure defined by the Requirements interface and returns it
13. Triage engine decides what to do with the returned node, which could be:
    a. return an unanswered question to the user, with any collected care advice
    b. jump to step 9 with the target of an answer already provided to the question
    c. set a variable and jump to step 9 with the node's target
    d. read a variable and jump to step 9 with the target of the passing rule
    e. collect care advice and jump to step 9 with the node's target
    f. return a triage outcome to the user

15.2.4 Recommendation

We will initially target option 3 with dynamodb holding the pathways data.

15.3 Triage API architecture review

Note: we need to establish the name for the api.

To do:

- Add a link to it in API register[135]
- Fill out the page - deleting anything you don't need, such as this tip. When you edit this page there are number of hints and further help.

This page is the API architecture review checklist for the above API.

It captures (or provides links to) details of key architectural decisions for the API.

Links might be to documentation on other Confluence pages, or possibly code artefacts, such as the API's OAS file.

As per the API delivery process[136], the decisions are split into two sections:

135. https://nhsd-confluence.digital.nhs.uk/display/APM/API+register
136. https://nhsd-confluence.digital.nhs.uk/display/APM/Deliver+your+API

1. in the discovery phase: **architecture** - big architectural decisions
2. in the alpha phase: **design** - detailed design decisions

15.3.1 Status codes key

TO DO

IN PROGRESS

DONE

N/A

15.3.2 Architecture checklist (discovery phase)

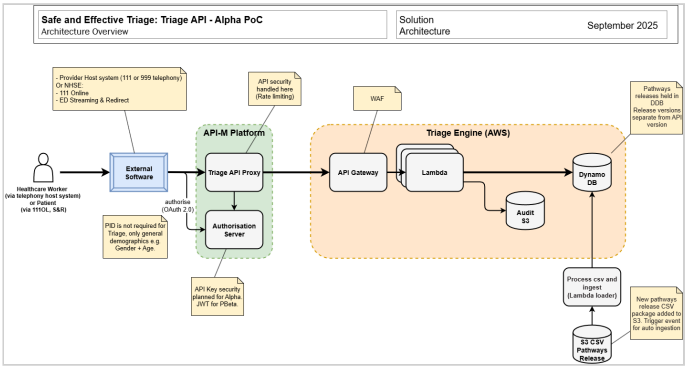
If this API is going to TRG for review, it might make sense to complete an Solution Design Options (SDO) and / or Key Architectural Decision (KAD) document and link out to that.

- Response - This has been to TRG as part of GDS phase transition moving to Alpha. Link to SDO here: SDO SaET Pathways Transformation - API v1.0.pptx[137]. TRG approval was granted in session 6th August. SDO2918 and SDO-2919 covering Triage outcome model and Triage API.

137. https://nhs.sharepoint.com/:p:/s/msteams_9646b6/EeA_3QqNcr9CtpCTotrRkIsB5rPfNp4-r_Gp8-AyxJcOkw?e=HkVL2s

Activity	Status	Comments / Evidence
Explain the context / problem / opportunity space for your API.	DONE	**The purpose of the API**: To provide real-time 'safe and effective clinical triage' over an api. The api will deliver triage questions and receive questionnaire responses until a structured triage outcome is reached. The outcome can then support the onward triage journey and downstream processes. **What benefits will it bring**: • Lower effort clinical content releases • Better ability to respond to change • Improved data and analytics • Easier onboarding of and integration for suppliers • Improved compliance • Enablement of future capabilities **Who are the expected users**: Host system suppliers operating within 111 and 999. This includes 111 Online and the ED Streaming and redirection service. **What use cases do you have for your API**: Running end to end clinical triage journeys **Planned Go-live Date:** Currently in Alpha, complete by: November 2025 Exit discovery presentation is here with more background context: SaET-HowWeConnect-RefinementPhase-Report.pptx[138]
Consider solution / architecture options to address the problem / opportunity space, and select a preferred option. Everything below this activity is based on the selected option, and assumes the selected option includes an API.	IN PROGRESS	*<provide evidence of your options, for example by completing a copy of our API architecture options template[139] and linking out to that>* Will need to go back and retrospectively complete the options template if required. A few relevant ADRs: • ADR-API-002: Triage API hosting options[140] - Serverless AWS approach and not locally deployed containers • ADR-API-003: Triage API communication format options[141] - Use of FHIR or not. TRG API SDO covers the overview.
Identify the key use cases / functions for your API.	IN PROGRESS	This API will allow external systems to: • Self-triage – e.g. individuals using automated online triage tools • Triage of patient by non-clinical staff[142] • Triage of patient by clinical staff[143]

138. https://nhs.sharepoint.com/:p:/s/msteams_9646b6/Ee4p-fOzfLBOhB3-vvcD8EoBY7FUb1kF4Sgr1zSJywFrJw?e=q4FDSk
139. https://nhsd-confluence.digital.nhs.uk/display/APM/API+architecture+options+template
140. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1087781594/ADR-API-002+Triage+API+hosting+options
141. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1102731443/ADR-API-003+Triage+API+communication+format+options

Activity	Status	Comments / Evidence
Identify the end user - healthcare worker, patient or not present.	DONE	The end user is a **healthcare worker** / **patient**
Determine the end-to-end architecture - how data flows and where processing occurs. Illustrate it using a context diagram.	DONE	
If your API routes requests to multiple backends, figure out how it will do this. • See Routing requests to multiple backends[144].	DONE	Single back end
Determine where the various components of your architecture (including your back-end, if new) will be hosted. • We have some hosting patterns - we can help you with this. https://nhsd-confluence.digital.nhs.uk/display/APM/Transforming+data+on+the+API+platform+and+containers[145]	DONE	AWS self hosted

142. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1054580779/Use+Case+-+Triage+of+patient+by+non-clinical+staff+via+Triage+API
143. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1071917461/Use+case+-+Triage+of+patient+by+clinical+staff+via+Triage+API
144. https://nhsd-confluence.digital.nhs.uk/display/APM/Routing+requests+to+multiple+backends
145. https://gbr01.safelinks.protection.outlook.com/?url=https%3A%2F%2Fnhsd-confluence.digital.nhs.uk%2Fdisplay%2FAPM%2FTransforming%2Bdata%2Bon%2Bthe%2BAPIM%2Bplatform%2Band%2Bcontainers&data=05%7C01%7Cpaul.fisher18%40nhs.net%7C576a3f84242849e79d3708db14cfca35%7C37c354b285b047f5b22207b48d774ee3%7C0%7C0%7C638126654897068216%7CUnknown%7CTWFpbGZsb3d8eyJWIjoiMC4wLjAwMDAiLCJQIjoiV2luMzIiLCJBTiI6Ik1haWwiLCJXVCI6Mn0%3D%7C3000%7C%7C%7C&sdata=fmfLXuxADYr8tJHeq39tTxppBJAf2B2Ltq8Jpl5omeM%3D&reserved=0

Activity	Status	Comments / Evidence
Decide on the API consumer security pattern. • See API consumer security[146]	DONE	This API uses the following security pattern(s): • application-restricted access - using API key authentication (as per API consumer security - Application restricted- API key authentication[147]) • This pattern is intended for Alpha phase for speed and ease of implementation. Alpha phase will cover a small subset of clinical pathways. • application-restricted access - using signed JWT authentication (as per API consumer security - Application restricted API- signed JWT authentication[148]) • This pattern is intended once we proceed to Private Beta. • ~~user-restricted access - using NHS Identity for end user authentication (end user is a healthcare professional) (as per API consumer security- User-restricted-NHS CIS2[149])~~ • ~~user-restricted access - using NHS Login for end user authentication (end user is a citizen) (as per API consumer security- User-restricted-NHS login[150])~~ This pattern is appropriate because: • The API is read-only. There is no facility to add, update or delete data. • PID is not required for Triage • The API is stateless. The host system is responsible for managing the Triage Journey state. • A call to the api is to determine the next best activity - ask another question, or triage outcome. There is no single call that can be made to elicit large scale export of the algorithm. It would need continuous cycling of inputs to spider the algorithm. Rate limits and WAF will be in place to mitigate this including the app-restricted protections. • Telephony call handlers do access sensitive patient data within the host system and it is the host system responsibility to authenticate users for this. The above elements taken together mean this is a low risk api and there is limited benefit in imposing user-restricted access. • For self triage route - if using 111OL via the NHS App then the login is passed to 111 Online to pre-fill some elements if there is a referral.

146. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security
147. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-Application-restrictedAPI-APIkeyauthentication
148. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-Application-restrictedAPI-signedJWTauthentication
149. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-User-restricted-NHSCIS2
150. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-User-restricted-NHSlogin

Activity	Status	Comments / Evidence
		• For telephony, call handlers are not normally employed by NHS but by the healthcare provider meaning they typically do not have nhs.net accounts. Smartcard usage is mixed depending on provider. Rollout of NHS Identity for telephony providers would require significant resource and management. This could impact uptake of the API. ADR writeup covering this in progress[151]
Decide how to do backend security. Security for API Backend[152]	IN PROGRESS	The API proxy connection to the backend is secured using **TLS-MA** ***Assume this will be TLSMA - to discuss in review meeting.***
Decide on the integration pattern for the API. • See Integration Pattern Book[153] • Your APIs should be synchronous under the majority of situations	DONE	This API uses the following integration pattern(s): • REST - synchronous
Decide whether to use FHIR. • See Is my API a FHIR API?[154]. • See API Landscape[155].	DONE	This API is FHIR-conformant. In particular: • It uses **FHIR R4** • It uses the following FHIR R4 global base resources: FHIR Task stateless approach[156] • It has the following local profiles in FHIR UK Core: **TBC**
If you're not using FHIR but you should be, or if you're using an older version fo FHIR, explain why. You'd better have a damn good reason.	DONE	N/A
Decide on the payload type(s). • See https://developer.nhs.uk/apis/fhir-policy/serialisation.html. • Consider large payloads - see Large payloads[157].	DONE	The API payload is **JSON** Will any payload exceed 10MB in size? **NO** FHIR Task stateless approach[158]

151. https://nhsd-confluence.digital.nhs.uk/x/XW4-RQ
152. https://nhsd-confluence.digital.nhs.uk/display/APM/Security+for+API+Backend
153. https://digital.nhs.uk/about-nhs-digital/our-work/nhs-digital-architecture/integration-patterns-book
154. https://nhsd-confluence.digital.nhs.uk/pages/viewpage.action?pageId=216171050
155. https://nhsd-confluence.digital.nhs.uk/display/APM/API+Landscape
156. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1102713093/FHIR+Task+stateless+approach

Activity	Status	Comments / Evidence
Decide the API's service level. These specify SLAs around things like availability and incident fix times. • See Service levels for APIs[159].	IN PROGRESS	Detailed discussions in progress, including DOSM (Digital Operations and Service Management). Assumed Platinum for live but if Private Beta uses 111 Online then there may be scope to use bronze in this phase. During private beta, the service level for this API is **BRONZE**. From public beta and beyond, the service level for this API is **PLATINUM**.

157. https://nhsd-confluence.digital.nhs.uk/display/APM/Large+payloads
158. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1102713093/FHIR+Task+stateless+approach
159. https://nhsd-confluence.digital.nhs.uk/display/APM/Service+levels+for+APIs

<table>
<tr><th>Activity</th><th>Status</th><th>Comments / Evidence</th></tr>
<tr><td>Determine what path-to-live environments you'll need to meet your API consumers' testing needs.</td><td>TO DO</td><td>Assumed for Alpha and then Private-beta. For production there may be scope for a training environment?
The API includes the following path-to-live environments:
<table>
<tr><th>Environment</th><th>Purpose</th><th>API consumer security pattern(s)</th><th>Implementation notes</th></tr>
<tr><td>Production</td><td>Live environment</td><td>As per production</td><td></td></tr>
<tr><td>Pre-production (Reference)</td><td>Production reference.</td><td>As per production</td><td></td></tr>
<tr><td>Integration test</td><td>Formal integration testing</td><td>As per production?</td><td>TBC which backend environment does this connect to?
TBC how will test data be managed?</td></tr>
<tr><td>Development</td><td>Development</td><td></td><td></td></tr>
</table></td></tr>
</table>

<table>
<tr><th>Activity</th><th>Status</th><th>Comments / Evidence</th></tr>
<tr><td></td><td></td><td>
<table>
<tr><th>Environment</th><th>Purpose</th><th>API consumer security pattern(s)</th><th>Implementation notes</th></tr>
<tr><td>Sandbox</td><td>Early developer investigation</td><td>Open access</td><td>Canned responses to a small number of scenarios.
Response fragments shared with API specification.
Deployed to AWS as per API platform standard approach.
Stateless.</td></tr>
</table>
</td></tr>
<tr><td>Decide the granularity and name(s) for your API(s).
See API naming and grouping endpoints[160]</td><td>TO DO</td><td>TBC. Naming has been identified as something we need to decide on.
The functionality described on this page is grouped into coherent APIs as follows:
<table>
<tr><th>Plain English name (as per the API catalogue[161])</th><th>Physical name (URL)</th><th>Functional scope</th></tr>
<tr><td>Start or continue a triage</td><td>https://api.service.nhs.uk/triage/FHIR/R4/triage</td><td></td></tr>
<tr><td>End a triage before reaching an outcome</td><td>https://api.service.nhs.uk/triage/FHIR/R4/triage/early-exit</td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
</table>
</td></tr>
</table>

<table>
<tr><th>Activity</th><th>Status</th><th>Comments / Evidence</th></tr>
<tr><td>Check that your API doesn't overlap with another API owned by another team.
The above naming exercise, and a scan of the API catalogue[162], will help with this.
Silo teams building overlapping APIs has historically been one of our biggest issues.</td><td>TO DO</td><td><confirm or discuss as appropriate>
Depends on naming. Not expected to clash.</td></tr>
<tr><td>Estimate the expected volumes for your API.</td><td>TO DO</td><td>TBC
We estimate that the API will experience the following volumes:
<table><tr><th>Metric</th><th>Private beta</th><th>Long term</th></tr><tr><td>Number of connected systems</td><td>TBC</td><td>TBC</td></tr><tr><td>Peak load transactions per second (TPS) per connected system</td><td></td><td></td></tr><tr><td>Peak load transactions per second (TPS) overall</td><td></td><td>30 TPS</td></tr><tr><td>Transactions per month per connected system</td><td></td><td></td></tr><tr><td>Transactions per month overall</td><td></td><td></td></tr></table>
<table><tr><th>Service</th><th>Estimated triages per annum</th><th>Average number of questions per triage</th><th>Estimated transactions per second (TPS)</th></tr><tr><td>Ambulance 999</td><td>5,099,768</td><td>19</td><td>4</td></tr><tr><td>111 Telephony</td><td>19,514,394</td><td>20</td><td>22</td></tr><tr><td>111 Online</td><td>7,794,040</td><td>11</td><td>5</td></tr><tr><td>Streaming & Redirection</td><td>358,579</td><td>17</td><td><1</td></tr><tr><td>Total</td><td>~ 33,000,000</td><td></td><td>32</td></tr></table></td></tr>
</table>

160. https://nhsd-confluence.digital.nhs.uk/display/APM/API+naming+and+grouping+endpoints
161. https://digital.nhs.uk/developer/api-catalogue
162. https://digital.nhs.uk/developer/api-catalogue

Activity	Status	Comments / Evidence
Do you have PID or sensitive data in your URL Path or Query parameters? As key pieces of data, such as NHS number, is a key element of any RESTful design, the platform is geared up to have sensitive data in this. However, this should be kept to a minimum because of systems (e.g. client browsers) that have access to the full values could log such data	TO DO	<discuss as appropriate - there are a number of pages in the Producer Zone, for instance Privacy considerations in your design[163]> No PID expected in the API or response.

15.3.3 Design checklist (alpha phase)

15.3.3.1 Consistency checklist

The API Management Platform is going to be a key entry point into NHS Digital, and as a consequence will highlight potential inconsistency between different approaches. Some of these consistencies can be hidden behind the Platform façade, others simply won't be cost effective to change. See the page on Consistency[164] for a run down of the wider view of this issue.

The following checklist is designed to help you identify inconsistencies early on.

15.3.3.1.1 Primary considerations

For detailed guidance on what is meant in these areas see: Designing your API[165]

Area	Consideration		If No:
API Name	Does it fit into the wider APIM Namespace / domain model?	Y/N	*For guidance on naming see:* API naming and grouping endpoints[166]
Session	Can Apigee manage the session with your API Consumer? Does the API Consumer call other APIs outside of Apigee?	Y/N	This has a number of small complications, so if your existing API has a custom call to start the session - or lots of data is pre-configured during authentication or authorisation this might need some extra design work

163. https://nhsd-confluence.digital.nhs.uk/display/APM/Privacy+considerations+in+your+design
164. https://nhsd-confluence.digital.nhs.uk/display/APM/Consistency
165. https://nhsd-confluence.digital.nhs.uk/display/APM/Designing+your+API
166. https://nhsd-confluence.digital.nhs.uk/display/APM/API+naming+and+grouping+endpoints

Area	Consideration		If No:
Update	PUT is for updating entire FHIR resources PATCH is being used where there are small changes to a large FHIR resource	Y/N	
Atomic operations	Always use small atomic (stateless) requests - batching multiple items should be a last resort. Batching encourages poor design, reduces scalability and increases the 'blast radius' of any failures	Y/N	• Are the requests related by a transaction? This could (would) then still be atomic - but it depends on the size of the batch. • Is there a cost to enabling atomic over batch?
Data minimisation	Do your endpoints return the minimum amount of data possible? (To align with GDPR and other design principles)	Y/N	If not, could you: • Default to a smaller subset, and allow parameter to expand for specific requests? • Could you have different access modes, so the lower security connections receive less data
Testing	Are your current unit and integration tests all automated?	Y/N	
	Can you perform the testing via Apigee and your proxy?	Y/N	This is important to ensure the end to end service is fully tested.

15.3.3.1.2 Secondary considerations

These are the next level down of detail, and ideal for exploring in Alpha and Private Beta.

Area	Consideration		Response
Terminology	Are there key terms in your API that don't fit with other NHS Digital services?		
URL Path	Are there terms that might be confusing, stand out from other APIs? Is the path very long? See policy APIM-LAND-05 on API Landscape[167] for details of requirements for FHIR API URL Paths.		

167. https://nhsd-confluence.digital.nhs.uk/display/APM/API+Landscape

Area	Consideration		Response
Endpoint names	Use nouns instead of verbs (RESTful pattern) Are they intuitive? For FHIR APIs, endpoint names must align with the FHIR resource model as described at https://www.hl7.org/fhir/http.htmland https://www.hl7.org/fhir/resourcelist.html (the endpoint name must be capitalised) For non-FHIR APIs use spinal-case		*If there are inconsistencies here, mapping of these in Apigee is easy. However, what impacts are there on your existing users?*
Query Parameters	Are these consistent with other API Platform query parameters? FHIR APIs must be conformant with https://www.hl7.org/fhir/search.html		*If there are inconsistencies here, mapping of these in Apigee is easy. However, what impacts are there on your existing users?*
Field Names	Are the field names in the **payload** datasets the same as FHIR?		*Even for non-FHIR APIs, should developers have to deal with a different field name for basic data such as first name and last name?*
REST Conventions	Does your HTTP Verb usage align with RESTful principles? For FHIR APIs, does this also align with https://www.hl7.org/fhir/http.html		
	Do your HTTP Status Codes align with RESTful principles?		
	Do your FHIR APIs align with guidance found in the FHIR specification for HTTP status codes		*tbc guidance on this side*
Headers	For API Consumers - are they consistent with other API Management Headers?		*You decide as an API Publisher which platform headers are mandatory, see* HTTP headers for your API[168] *for detailed guidance*
	For your API backend (less important as you are managing this)		

168. https://nhsd-confluence.digital.nhs.uk/display/APM/HTTP+headers+for+your+API

Area	Consideration		Response
Environments	Does the way you use environments align with the API Management platform, and the Path To Live?		See https://confluence.digital.nhs.uk/display/APM/Architecture#Architecture-environments
API Versioning	The private beta hasn't needed to deal with versioning, but we are looking to use accept-headers to support this in the future, though the current NHS Digital policies include the API major version in the URL path.		
Error Messages	How you return error messages should be consistent with other interfaces (e.g. FHIR APIs will return OperationOutcome resources to express error detail)		See additional information pages here Handling errors[169] Consistency between APIs[170] RESTful HTTP design points[171]
Test Data	What is your approach to test data for External Developers?	-	
S Flag	Does your API need to consider the Sensitive flag status[172]	Y/N	No.

15.3.3.1.3 Detailed design considerations

The following are very detailed elements about your APIs, but as with the other points above, they can introduce breaking changes into your design. The design pattern in place on the Platform has come out of a combination between the FHIR standard, Apigee platform design, Spine as a key backend across the platform, and input from across the Tribe and NHS Digital.

Area	Consideration		
ETag	FHIR standard mandates the use of the ETag for version checking		See Does my API need to use ETag Headers[173]

169. https://nhsd-confluence.digital.nhs.uk/display/APM/Handling+errors
170. https://nhsd-confluence.digital.nhs.uk/display/APM/Consistency+between+APIs
171. https://nhsd-confluence.digital.nhs.uk/display/APM/RESTful+HTTP+design+points
172. https://digital.nhs.uk/services/national-back-office-for-the-personal-demographics-service/management-of-nhs-numbers-and-pds-records
173. https://nhsd-confluence.digital.nhs.uk/display/APM/Update+locking+with+ETag

Area	Consideration		
Postman collections	There are other Postman collections across the platform, does yours follow the same conventions ?		
Datetime	Date and time field values should follow the ISO-8601[174] standard, read the details on how to do this: RESTful HTTP design points[175]		

15.3.4 Governance

This section includes information related to the architecture and design reviews that occur as part of the API delivery process[176].

15.3.4.1 API architecture review

Date	TO DO
Attendees	TO DO
Actions	TO DO

15.3.4.2 API design review

Update this page with the latest, could be important points for the later TRG / LSB governance steps

Date	TO DO
Attendees	TO DO
Actions	TO DO

15.3.4.3 Post implementation review

Update with any final changes, and flag up any key future work

174. https://en.wikipedia.org/wiki/ISO_8601
175. https://nhsd-confluence.digital.nhs.uk/display/APM/RESTful+HTTP+design+points
176. https://nhsd-confluence.digital.nhs.uk/display/APM/Deliver+your+API

Date	TO DO
Attendees	TO DO
Actions	TO DO

15.4 Triage API Alpha goals and success criteria

15.4.1 Alpha technical PoC build goals

Validate logical architecture

- Build and test the chosen architecture (APIM proxy, AWS backend - API gateway, Lambda, DynamoDB)
- Confirm components work together and are fit for purpose
- Produce something live-like that can provide meaningful performance metrics

Validate API specification

- Progress from paper based draft and implement a RESTful FHIR API to the initial specification
- Test against clinical pathways of varying complexity to achieve good Pathways technical specification coverage

Measure API performance

- Establish baseline performance metrics (latency, throughput, error rates) covering NFRs
- Assess whether the design meets real-world demands.

Provide a working prototype for external testing

- Deliver an API that consumers can test with in a controlled way.
- Gather feedback from potential integrators and stakeholders.

15.4.2 Success criteria

Architecture design validation

- All key components of the architecture function together reliably with no critical blockers. (APIM, AWS Lambda Triage Engine, DynamoDB data store)

Triage API Specification validation

- The API interface works as designed
- The chosen FHIR resources, codes, and interaction patterns are appropriate
- API passes base FHIR schema validation checks

Performance evidence

- Latency is within an acceptable range (e.g. sub-300ms for key interactions)
- System can scale to expected request volumes without degradation
- Data access patterns (DynamoDB read/write) meet requirements.

Architecture readiness

- We have enough evidence to confirm whether DynamoDB is viable or if alternatives (e.g., in-memory store) should be explored

Operational learnings captured

- Logs and monitoring tested so future Beta can build on them.

Consumer feedback *(in scope for alpha phase but not alpha build?)*

- At least one external partner provides feedback on specification.
- At least one external partner to successfully integrate and provide feedback on API

15.5 Triage API Run cost estimate - pre-Alpha

15.5.1 Architecture key components (to feed into cost calc)

- [x] API Gateway
- [x] Lambda
- [x] Dynamo DB
- [x] Cloudwatch / cloud trace. Alarms
- [x] WAF
- [ ] S3 and ingestion pipeline
- [ ] Networking
- [ ] Step functions
- [ ] CI/CD costs - are they in scope? - Still expect the cost to be negligible. Could bring in scope for next cost revision once we have build data.

15.5.1.1 Environments

Non-prod environments will have low traffic and therefore low cost. Not added to cost model.

Environment design still to be confirmed.

15.5.1.2 TPS calc

Seconds per month

1 day = 24 × 60 × 60 = 86,400 seconds

30-day month = 86,400 × 30 = 2,592,000 seconds

Requests per month

1 TPS → 1 × 2,592,000 = 2,592,000 requests/month (2.6M)

10 TPS → 10 × 2,592,000 = 25,920,000 requests/month (26M)

50 TPS → 50 × 2,592,000 = 129,600,000 requests/month (130M)

100 TPS → 100 × 2,592,000 = 259,200,000 requests/month (260M)

15.5.2 Usage assumptions, scenarios

- Alpha usage - minimal, testing. Load testing the only spike that could incur cost.
- Beta - thin slice of users e.g. with 111O
- Live / target state - All triages via API. Upper scale estimates of volume.

15.5.3 Caveats

- Focus for this cost estimation is on high impact costs. There is more detailed modelling that could be done but assume this would add minimal cost, e.g. storage.
- Traffic assumptions based off averages rather than real world usage. Better modelling of traffic spikes needed via IDT data.
- Consumption usage (eg Lambda) based off rough estimates, need to see usage from the alpha to better refine
- NHS gets one gov OGVA discount ~ 23%
- Triage API only. Doesn't include mapping to new TOM (integration with terminology server) or FDP integration for analytics etc.

15.5.4 Exclusions

- No caching in API Gateway (CF). This will bring costs down for Dynamo.
- Not costed for step functions routing between api versions. Not cost effective or performant. Assumed route within Lambda or use express step functions.
- Not costed for networking as this is a serverless first stack. Cross region replication would incur a cost, not considered yet.
- Audit log storage. Storage is low cost.
- Backup storage. Low cost.
- DR BC for London + Ireland pilot light not yet costed.
- CI/CD costs not accounted for

15.5.5 Cost table

Note all these figures need validating with real usage data from the Alpha poc.

Usage	Alpha (low)	Beta (low-medium)	Live (all triages - very high)	Notes
Transactions per second (TPS)	1 TPS 2.59M p/ month	10 TPS 25.9M p/ month	50 TPS 129M p/ month	Initial estimates were 30 TPS for live usage so 50 TPS is far over but gives us a ceiling cost.
Payload size (Affects API GW and Lambda duration)	3kb	3kb	3kb	Payload size options analysis[177] Max payload estimate is ~ 3kb. The average would be lower but estimating on the upper scale here.
Ave lambda execution time and memory allocation	50ms / 128mb	50ms / 128mb	50ms / 128mb	Needs calculating based on Alpha performance. Pure guess at this point.
DynamoDB pattern (on demand / provisioned)	on-demand	on-demand	on-demand	Once we have detailed metrics and usage patterns we can look into revising the cost with provisioned which will be cheaper. Read cost will be far cheaper with APIGW caching enabled.
Logging retention	1 month	3 months	3 months	Operational logs. Audit logging requirements still to be designed. Calc 2kb per request. 1 TPS - 5GB 10 TPS - 50GB 50TPS - 250GB Based off aws cloudwatch. End state solution should be with NHSE ODIN or Splunk.

177. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1087792387/NPT-003+Options+Analysis

Firehose logging for CSOC/ITOC	Not needed	Pre-prod, prod	Pre-prod, prod	
Ingestion pipeline usage	Ad-hoc	Low (once a week)	Once daily	S3, ETL compute with Lambda, Dynamo writes, logging. Negligible cost per run (<$1).
Dynamo storage (Version retention)	Subset of a single version 50mb	2 versions at 50mb each 100mb	50 versions 2.4gb	This cost is negligible. Main cost for dynamo is access pattern (reads/ write) rather than storage.
No. of environments	1 (Dev)	5 (Dev, test, management, prod, DR)	5 (Dev, test, management , prod, DR)	Non-prod environments will have low traffic and therefore low cost. Not added to cost model. Environment design still to be confirmed.
Features	MVP alpha	1 or 2 limited use cases	Multi-channel (several versions in active use)	
DR/BC	London	London	London + Ireland pilot light	We get Multi AZ out of the box with serverless stack. Multi region not yet designed so difficult to model. Not included in cost.
Estimated Cost per month (not including 23% OGVA 2 discount)	**37.67 USD**	**176.27 USD**	**805.59 USD**	Public link to costing v1[178]

15.6 Future infrastructure and role permissions

15.6.1 Overview

Set the context and overview of this page and why its needed.

178. https://calculator.aws/#/estimate?id=c9d971894f0347c9f56648150cd5cafb84d02c2f

15.6.2 Principles

What principles we will adopt when designing and operating infrastructure and setting roles.

- Least privilege - Access shall be granted strictly to what is necessary for a user to perform their role. This minimises risk by reducing exposure to unnecessary services or resources.
- Immutable pipelines - Application changes are deployed through automated CI/CD pipelines. We avoid manual changes (e.g through the console) to ensure traceability, consistency and audit. This also reduces risks of human error, and enables deployment control.
- Role based access / Environment access scoped to roles - Permissions will be tailored to specific environments making sure appropriate access is managed more easily. This reinforces least privilege.
- Centralised access management - Access will be managed centrally following organisational patterns. This uses infrastructure as code (terraform) and AD SSO. This also provides traceability, consistency and audit.
- Break-glass support - There may be unforeseen scenarios that require elevated access in environments. We will use the organisational Temporary Elevated Access Management (TEAM[179]) pattern to support this. This aligns with Cloud-8 of engineering red-lines[180].

15.6.3 Architecture / tooling

- Terraform based access control - Access is centrally managed through a Terraform repository[181] that defines the AWS account structure, Azure AD group mappings and IAM permission sets.
- Permission sets - Permission Sets act like IAM roles and are assigned per environment. They reference reusable IAM policies
- AWS SSO - Azure AD groups are linked to AWS IAM Identity Center. Staff roles are mapped to Azure AD groups. Group ownership is delegated and managed via https://mygroups.microsoft.com (TODO - this has recently changed, update with new process)

15.6.4 AWS Accounts

Proposed accounts for hosting the Triage API and related components:

Name	Purpose	Details
DEV - Developer	Development	Provisioned - CHR-2813 - 998245890657

179. https://nhsd-confluence.digital.nhs.uk/x/ljXFOw
180. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-lines.aspx#cloud-8
181. https://github.com/NHSDigital/aws-prod-sso-config

Name	Purpose	Details
TEST - Test	• Testing new features • integration testing with external systems • Testing new releases	Provisioned - CHR-2970 - 938674806470
MGMT - Management	Used to manage IAM policies that are attached to the Permission Sets	To request
PROD - Production	Live service environment	Provisioned - CHR-2970 - 744389574564
DR - Disaster Recovery	Immutable backups	To request

15.6.5 External environments

Still to be defined but proposed:

Name	Purpose	Hosting account
Sandbox	Open access, example API with canned responses.	TEST
Integration	• Testing new features • integration testing with external systems • Testing new releases	TEST
Production	Live service environment	PROD

15.6.6 Role definitions

An outline of the staff roles and the access levels across AWS environment.

Role	Tasks	Notes
Admin	Full access	Full access

Role	Tasks	Notes
Developer, including Tester	Writing/maintaining code, submitting pull requests for bugs or application features Creating and running tests on application features	Developers (& testers) can write in **DEV**, but only read in **TEST** and **PROD**.
Operations / infrastructure	Manage the Terraform modules and infrastructure-as-code deployments Monitor system health Applying security controls	In **TEST**, **PROD** and **MGMT** users typically have **Read-Only** access, but for critical tasks **eoRW** is provided.
Support and Billing	Monitor usage spend across environments Read only access	

15.6.6.1 Access key

- **RO** – Read Only
- **RW** – Read/Write
- **eRW** – Elevated Read/Write
- **eoRW** – Elevated (optional approval) Read/Write

> *Note:* `UserRole` *refers to the* ***staff role****, not the AWS IAM role. Staff may assume multiple roles as needed.*

The method to **elevated access** in AWS: AWS TEAM (Temporary Elevated Access Management)[182]

15.6.7 Role Access Matrix by Environment

User role	Dev	Test	Prod	Mgmt	Notes
Admin	eRW	eRW	eRW	eRW	Admins have full access to all environments, with elevated access where required.
AWSAdministratorAccess	RW	RW	RW	RW	Admins have full access to all environments.

182. https://nhsd-confluence.digital.nhs.uk/spaces/AWS/pages/1002780054/AWS+TEAM+Temporary+Elevated+Access+Management

User role	Dev	Test	Prod	Mgmt	Notes
Developer	RW	RO/eoRW	RO	RO	Developers (& testers) can write in **DEV**, but only read in **TEST** and **PROD**.
Infrastructure	RW	RO/eoRW	RO/eoRW	RO/eoRW	In **TEST**, **PROD** and **MGMT** users typically have **Read-Only** access, but for critical tasks **eoRW** is provided.
Support and Billing	RO	RO	RO	RO	Support and billing users have Read-Only access to all environments for monitoring purposes. AWS Managed role.

15.6.7.1 Role-Based Access to AWS Services

Each role has specific access to AWS services across categories like **Compute**, **Data**, **Networking**, **Security**, **Monitoring**, and **Management**.

Note these are suggestions and will need scoping to actual requirements at build time.

Role	Compute	Data	Networking	Security	Monitoring	Management
Admin	Full access	Full access	Full access	Full access	Full access	Full access
Developer	Lambda	DynamoDB, S3	VPC, Route 53, CloudFront, Certificate Manager	WAF	CloudWatch, X-Ray	API Gateway, EventBridge, SSM, OpenSearch, SQS/SNS, Secrets Manager, Billing Console, Cost Explorer

Role	Compute	Data	Networking	Security	Monitoring	Management
Infrastructure	Lambda	DynamoDB, S3	VPC, Route 53, CloudFront, Certificate Manager	IAM, KMS, CloudTrail, Security Hub, GuardDuty, Amazon Inspector, WAF, AWS Firewall Manager	CloudWatch, X-Ray	API Gateway, SSM, OpenSearch, SQS/SNS, CloudFormation, Trusted Advisor, SSM, Health Dashboard, Resilience Hub, Well-Architected Tool, EventBridge, IAM Identity Center, SQS/SNS, Secrets Manager, Billing Console, Cost Explorer
Support	—	—	—	Trusted Advisor	CloudWatch	Support Center
Billing	—	—	—	—	—	Billing Console, Cost Explorer

> *Note: All permissions should be **scoped to environment (dev/test/prod)** and follow the **principle of least privilege**. Elevated access should be **temporary** and **audited** where possible.*

15.6.7.2 Permission sets

Permission sets are the role templates used in IAM. A permission set will have a collection of permissions specific to a role and environment. These permissions will be mapped to AD Groups for easy user management.

Examples include:

- `SaET-RW-Developer` – Read/Write access for developers in DEV
- `SaET-RO-Infrastructure` – Read-Only access for infrastructure teams in TEST, PROD, and MGMT
- `SaET-RO-Billing` – Read-Only access to billing and cost explorer tools

15.6.8 SSO Permission Sets

The repository `aws-prod-sso-config`[183] will define the **permission sets** that are mapped to the policies in the appropriate environment.

Permission sets	Dev	Test	Prod	Mgmt	DR	Notes
SaET-Admin	✅	✅	✅	✅	❌	Full access. DR account restricted for immutable backups.
SaET-RW-Infrastructure	✅	❌	❌	❌	❌	Read/Write Infrastructure role for Dev (full access to compute, data)
SaET-RW-Developer	✅	❌	❌	❌	❌	Read/Write Developer role for Dev (Lambda, DynamoDB, S3)
SaET-RO-Infrastructure	❌	✅	✅	✅	❌	Read-Only Infrastructure role
SaET-RO-Developer	❌	✅	✅	✅	❌	Read-Only Developer role
SaET-RO-Billing	✅	✅	✅	✅	❌	Read-Only Billing role for read-only access
SaET-RO-Support	✅	✅	✅	✅	❌	Read-Only Support role for read-only access

15.6.8.1 References

- FtRS DoS AWS User Role Permissions[184]
- AWS Single Sign-On with Azure AD Identity Provider[185]
- DOS Permission Sets[186]
- AWS Single Sign On[187]
- AWS Admins [188]TEAM documentation[189]
- AWS Admins SSO Documentation[190]
- Engineering red lines - Cloud-8[191] (TEAM)

183. https://github.com/NHSDigital/aws-prod-sso-config/tree/main
184. https://nhsd-confluence.digital.nhs.uk/x/3SW5Pg
185. https://github.com/NHSDigital/aws-prod-sso-config/tree/main
186. https://github.com/NHSDigital/aws-prod-sso-config/tree/main/terraform/programmes/DOS
187. https://nhsd-confluence.digital.nhs.uk/spaces/AWS/pages/529502405/AWS+Single+Sign+On
188. https://nhsd-confluence.digital.nhs.uk/x/ljXFOw
189. https://nhsd-confluence.digital.nhs.uk/x/ljXFOw
190. https://nhsd-confluence.digital.nhs.uk/x/xZCPHw
191. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-lines.aspx#cloud-8

15.7 Failure Scenarios

15.7.1 Context

Informal holding page for technical failure scenarios and potential mitigations.

Will be used to build into threat model and hazard log.

Scenario	API Side	Provider side	Cyber	Clinical	Technical

15.7.2 Infrastructure

- Single AZ outage – multi az config, load balanced out of the box across 3x AZ.
- AWS london region – warm/cold standby in Ireland region. DNS failover. Warm or cold standby.

Single service outage – API GW, Lambda, DDB. Hitting service quotas. Apigee (APIM) – Is this possible for the pillar services, not sure it is. APIM outage is a possibility. Could implement our own API GW same as Notify.

15.7.3 Connectivity

- Local internet outage – connectivity failover – multiple redundant connections – 2x Fibre, 1x 5G?
- National network outage – is this a thing? DNS caching, multi AZ should be resilient.

15.7.4 Security

- Apigee failure – URL bypass?, JWT failure

15.7.5 App / logic bug

- Version control downward negotiation, regression testing

15.7.6 Data – corrupted resource

- Too costly for heavy schema validation

- Conformance testing at onboarding
- Automated monitoring for failure patterns
- Version mismatch – inc v in response headers

15.7.7 Performance

- Latency – observability& alerting, caching of static components, autoscaling
- Rate limiting – usage limits, retry w/ expo backoff, tiered limits per supplier/provider?

15.7.8 Testing / Deployment failure

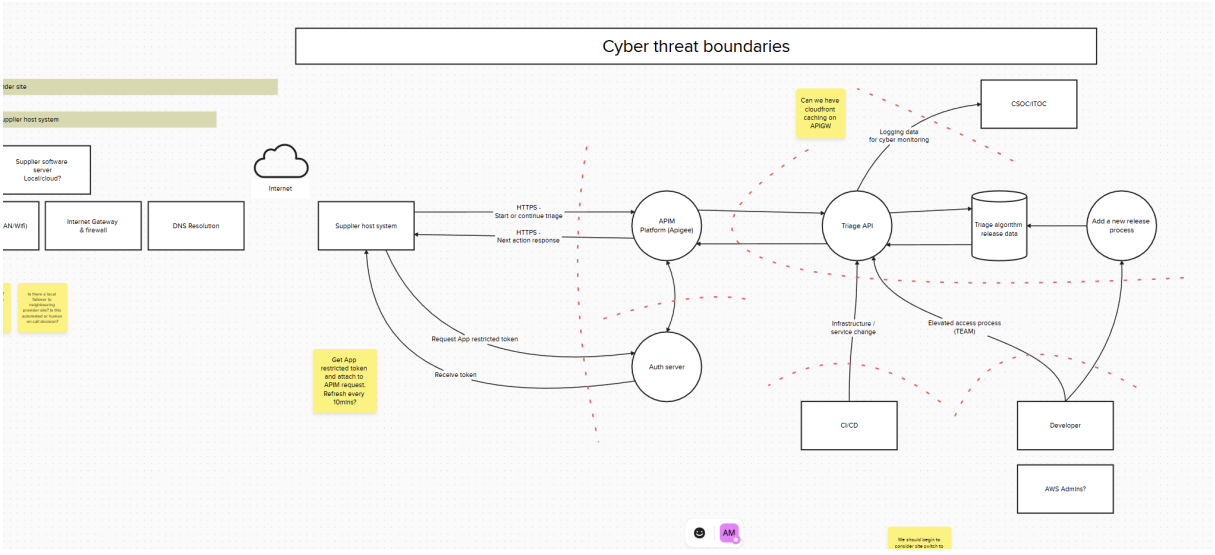
- Staging and int testing, synthetic traffic, canary deployments

15.7.9 Integration

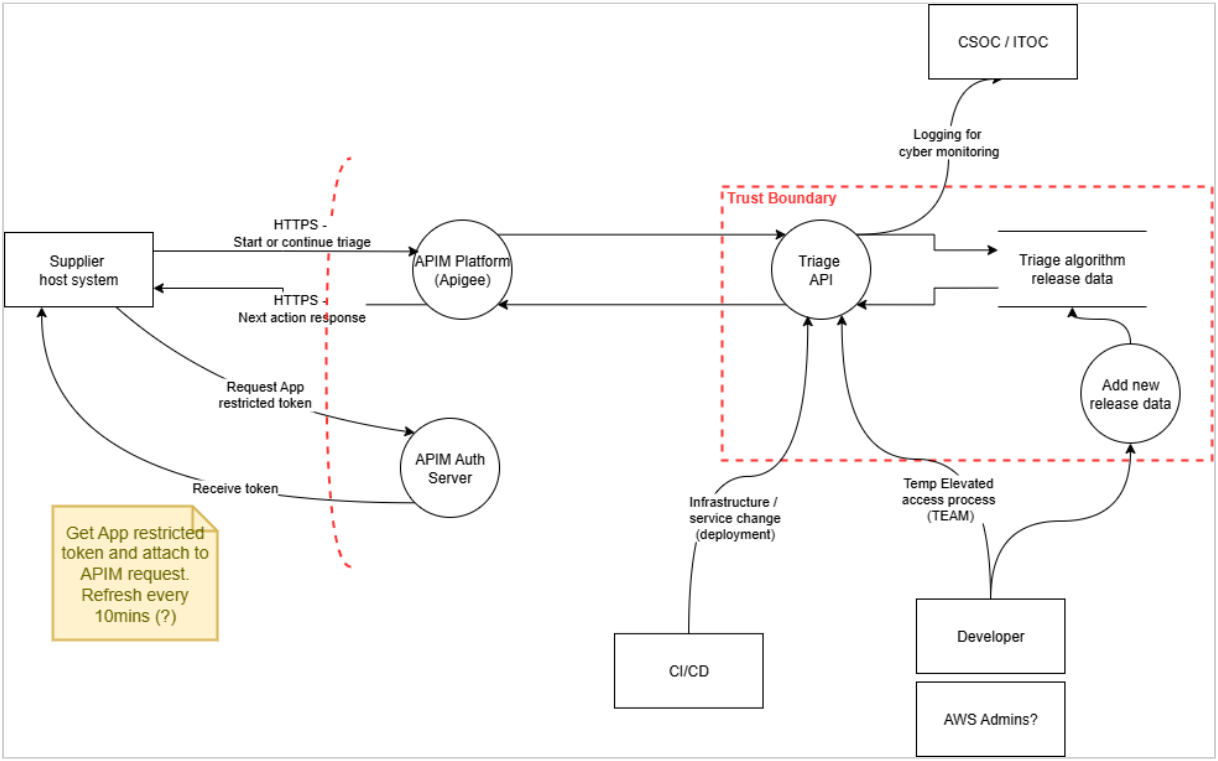
- DOS lookup / Outcome mapping error – ftrs
- Reporting queue – IDT, Use queue strats eg DLQ

15.7.10 Process & human failures

- Incorrect config – onboarding checklist, sandbox env
- Bad local logic – cache expiry, version lockout



Separate the boundaries per interaction as the controls may be different depending on the actor



15.8 Service Map

As-is Index of stages and journeys highlighting key areas of the user's journey impacted by the MVP API.



Link to PDF: MVP API impact of change on As-is.pdf[192]

Link to Mural: https://app.mural.co/t/nhsdigital8118/m/nhsdigital8118/1755007496228/a3e24eff02d03d97a78bbf3455b3fc9a6701d25d?sender=uccd1f4b324f5a5e2cd650604

15.9 APIM FMEA

15.9.1 Redundancy pattern options

15.9.1.1 Active/Passive multi-region via Apigee only

- Two Apigee runtimes in distinct GCP regions; DNS health-checks flip traffic away from unhealthy instance
- Pathways API Backend mirrors in AWS multi-region: (London/Ireland) Active/Active or Active/Warm
    - Pathways API runs on serverless architecture so dual hosting run costs should be minimised

Unknown: APIM RTO/RPO do these meet Pathways platinum expectations? Citation needed.

192. https://nhs.sharepoint.com/:b:/r/sites/msteams_9646b6/Shared%20Documents/General/Workstreams/Improving%20how%20we%20Connect%20-%20Triage%20API%20Alpha/Service%20Design/MVP%20API%20impact%20of%20change%20on%20As-is.pdf?csf=1&web=1&e=Jyz0rH

15.9.1.2 Dual path “break-glass”

- Controversial, may struggle to get agreement at TRG. Removes single-point-of-failure if APIM goes down but sets a precedent that other services may want to follow
- May be a hard requirement for some 1s and 9s providers to accept the API solution
- Normal path for request traffic via APIM; *pre-approved* circuit-breaker to route select providers directly to AWS Gateway (mTLS + signed JWT + IP safelist) during APIM outage.
- Requires TRG / clinical / other governance and audit stance.
    - Only use under change authority? Change authority not automated and could take time to come through. Possibly worthless unless authorised to be activated automatically.

15.9.1.3 Edge-cache acceleration for a static question set / mod 0

- Is there an option to cache immutable content at Apigee or CDN to reduce latency, reduce backend spikes
- Provide resilience for prioritised pathways against Triage API issues

15.9.1.4 Other questions:

(some of this should already be published online in apigee documentation)

- How is Apigee set up for high availability
- How do edge nodes work, in the context of authentication
- What about config propagation and edge inconsistencies?
- How do we detect, contain, degrade gracefully, and recover?
- Observed performance inconsistencies. Could these manifest in the live service?

Failure modes (build this out into a table)

- Mode (how could it go wrong)
- Effect (impact)
- Score severity
- Causes (how could it occur)
- Score occurrence
- Current Controls
- Score detection
- Action recommended to reduce occurrance or improve detection
- Who is responsible
- Actions taken

Failure mode	Details	Causes	Current Controls	Recommended actions	Responsible	Actions		
Route53/DNS outage								